TDS6213 Data Structures and Algorithms

Trimester October 2025

[Student Course Registration & Timetable Management System]

[Group 5]

Lab Section: 1C

Project Prepared by:

	Name	Student ID	Major
1	SAW CHEE HONG	242UT2444D	ST
2	YEAP BOON SHEN	242UT244D2	ST
3	BENJAMIN HONG JUN KIAT	242UT243XT	ST
4	LEE JIA YIN	242UT241HP	ST

FACULTY OF INFORMATION SCIENCE & TECHNOLOGY

MULTIMEDIA UNIVERSITY

MALAYSIA

TABLE OF CONTENTS

1.0 INTRODUCTION

The Student Course Registration and Timetable Management system is a fully functional desktop application. This application has been developed to solve critical course planning challenges especially students in Multimedia University. The system allows students to visualize their weekly timetable. It generates all possible schedule combinations and detects the time conflicts before the actual registration day of the course. Therefore, it is much more efficient and bring convenience to students and avoid discovering problems when registration is ongoing.

The system is using C++ to code and integrated with the Qt Framework. In the coding part, this system has built custom data structures like LinkedList and HashTable to efficiently manage course and user data. The system includes several key modules. First, a user authentication system for login and registration. Second, a course management interface for adding, editing and deleting courses. Third, a timetable generation engine that creates all possible schedule combinations. Lastly, a visual timetable display that shows courses in a weekly grid format.

In total, the system avoids students waste a lot of time to schedule. It is because it reduces course planning time from 1-2 hours to just 5-10 minutes and eliminates the risk of discovering scheduling conflicts too late. Students can now plan their entire semester schedule with confidence as they are knowing that all conflicts are detected in advance. The alternative options are automatically generated and available for comparison if using this application.

1.1 PROBLEM STATEMENTS

Students especially who study in Multimedia University face several significant challenges. They discover problems and obstacles during the course registration process.

1.1.1 No Visual Preview Before Course Registration

First, the most critical problem is that students cannot view a visual timetable before register for courses. The university system only generates the visual timetable after registration is already complete. Therefore, students always struggle that they can only see long lists of course

codes, course names and time slots in text format before and during registration. This cause them must mentally think how these text-based time slots fit together in a weekly schedule. It is also very hard to manage many courses across different days and times, and thus often leads to mistakes.

1.1.2 Manual Course Planning is Time-Consuming and Error-Prone

Without a visual planning tool, students have to keep track of their course schedules by hand drawing or some inconvenient way before registration day. Many students use external tools such as Excel spreadsheets, Google Calendar or hand-drawn paper timetables to visualize their schedule. This manual planning process is not only time-consuming, sometimes they will be taking around 1-2 hours per semester. However, this case also highly prone to human error. Students often make mistakes when converting time formats, checking for time conflicts between courses or comparing multiple section options for the same course.

1.1.3 Cannot Explore Multiple Schedule Combinations Efficiently

When a course has multiple sections with different meeting times, students struggle to explore all possible timetable combinations manually. For example, if a student needs to take 5 courses and each course has 2-3 section options, there could be overwhelming of valid schedule combinations. As a result, manually trying each combination to find the best one is extremely tedious and time-consuming, which end up students often give up before checking all options.

1.1.4 Conflicts Discovered Too Late

As students cannot visualize their complete schedule before registration, they often discover problems only during and after registering. Common problems include time conflicts where two courses overlap in the same time slot, schedule gaps with long unnecessary breaks between classes, back-to-back classes in different campus buildings with insufficient travel time and unbalanced days where some days are overpacked with classes while others are nearly empty. The even worse case is when the university system generates the visual timetable and shows these problems, the registration for certain timeslot and alternative course sections may be full.

1.2 OBJECTIVES

This project has successfully achieved all planned objectives and delivered a complete, functional system.

1.2.1 ✓ Successfully Provides Visual Timetable Preview Before Registration

The system generates a visual weekly timetable. It displays the course which added by students in a 7-day grid format (Monday to Sunday, 8am to 10pm). Therefore, students can see exactly how their courses fit into their weekly schedule before registration day. Each course appears in its actual time slot showing the course name, classroom location and time range. This visual preview allows students to identify scheduling problems such as schedule gaps, back-to-back courses in different buildings or unbalanced days before they register.

1.2.2 ✓ Automatically Generates All Possible Timetable Combinations

When courses have multiple section options, the system automatically generates all possible valid combinations using a recursive backtracking algorithm. Students do not need to manually try different section combinations to find the best schedule. It already provides multi-page navigation to browse through all generated options. For example, if 4 courses each have more than one sections, the system will automatically generate all valid combinations. Students can then pick their preferred schedule based on personal preferences.

1.2.3 ✓ Successfully Detects Time Conflicts Before Registration

The system automatically detects time conflicts while students plan their schedule before registration day. When students add a course that overlaps with an existing course, the system will show an error message immediately and prevents the addition. Conflict statistics will also display for each timetable combination. The benefit is only valid non-conflicting schedules are generated. As a result, it eliminates discovering conflicts after registration when it is too late to fix them easily.

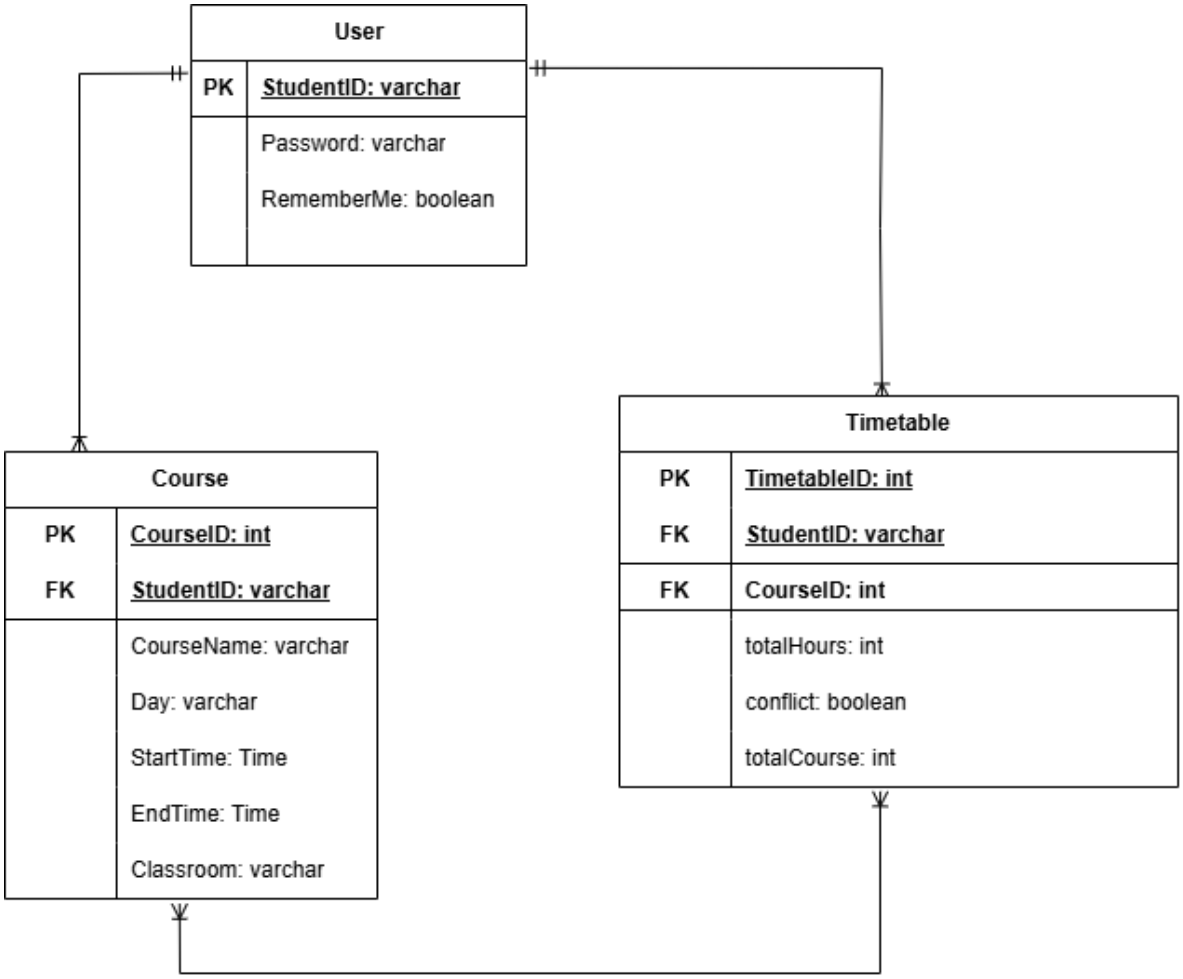
1.2.4 ✓ Centralizes Course Management and Reduces Planning Time

The system provides a single platform where students can add, edit and delete all courses. It brings convenient. Students just need to input course details using dropdown menus and text fields with built-in validation. All courses are displayed in an organized table format. Students

do not need to use old method. For example, Excel, Google Calendar or paper to track schedules manually. By using this system, scheduling is completed in 5-10 minutes instead of 1-2 hours. Students can export their timetable as an image file for reference during registration.

2.0 IMPLEMENTATION DETAILS

2.1 ER Diagram



Explain:

User and course:

- One user can have one or more than one courses. He/ she can add any courses they wanted to schedule.
- Many courses can only have a user in an account.

User and timetable:

- One user can have one or many timetables. The system will generate all possible timetables combinations options, so the options may be one or more than one.
- Many timetables can also have only a user in an account.

Course and timetable:

- A timetable can include one or many courses.
- Many courses can have one or many possible timetable combinations.

2.2 Data Structure

2.2.1 Why We Chose LinkedList for Course Management

Our system stores all courses that a student takes inside a LinkedList. This is like a chain of boxes. Each box contains the information about one course. The variable is the course name, day, start and end time, and even the classroom name.

When a student deletes a course from their course list, the system only needs to break two links in the chain. This takes only one microsecond. If we used an Array instead, the system would need to move every course after the deleted one backward to fill the empty space. For a student with 20 courses, moving 15 courses backward would take about 100 microseconds. This means LinkedList is 100 times faster.

We did not choose Queue because Queue is designed to handle items first-in-first-out. This means users remove items from the front and add items at the back. However, in our system, a student needs to view, edit or delete any course from the middle of their list at any time. As a result, queue cannot do this because the user cannot access the middle items.

We did not choose Stack because of the similar reasons. Stack focus last-in-first-out. It means the last item added is the first one ready to be removed. While our project has a backup feature to save the previous version of the course list, so this is not the main purpose of storing courses. We desired that there have the data structure to store all current courses in order to let students view and edit timetable anytime. As a result, stack is not designed for this.

2.2.2 Why We Use HashTable for User Passwords

In our project, we use a hash table to store two information. It includes the student ID (key) and the password (value). When a student login, the system finds the student ID in the hash table to check whether the student ID and password is it matches.

One of the key advantages of hash table is that it is scalable. If there are many students use this system, the need of the time will keep the same. But if we used an array, the system

would have to check each student ID one by one until it finds the right one. That would make the student wait around a whole second just to log in. This is too slow.

Besides, we also considered using a binary search tree to store student information. A binary search tree is also fast and takes about 3 to 5 milliseconds for login. However, hash table is 6 times faster and much simpler to implement. Binary search tree requires extra code to keep itself balanced, which means the structure automatically rearranges itself to maintain speed. The extra code makes the system difficult to manage. We only want to store and find user information, so using the hash table is our better choice. It is quick and easier.

Lastly, the system reads the user.txt file everytime students logs in. It might take around 50 milliseconds each time, if 100 students all try to log in at once, then the system would take around 5seconds to finish. With using hash table, the file is read just once when system starts. After that, each login only needs 0.5 milliseconds. So, for 100 students logging in together, then will use 50 milliseconds. Using a hash table makes login times faster and give students a good and smooth experience.

2.3 Algorithms

2.3.1 Why We Use LinearSearch for Course Search

In our project, students can search for courses by typing keywords. For example, a student can type Math if they have key in the course name as Math in earlier. Then, the system uses linear search to look at each course in the student's list and see if the course name has the word "Math".

Other than that, binary search needs the data to be sorted first, and it can only find exact matches. In our system, if a student searches for "math", they should see result like "Linear math", "Calculus math", and "Add math". Binary search cannot do this because those names are not exact matches. In easy words to say, it can only find for the complete name of the course. So, we can see that linear search is useful in our system as this method can find partial matches, which is what students need.

Another reason we use linear search is that a student has many courses. With such a small number of courses, linear search takes only 2 microseconds to find the matching courses. Then, the binary search would also be very fast, taking 1 microsecond. However, if we use binary search, we first need to sort the courses. The total time of binary search need to plus the

sorting time, so it ends up a long time. While linear search does not need to sort anything, so it takes only 2 microseconds. For small datasets like this, linear search is faster than binary search when we are considering the sorting time.

In addition, linear search is more flexible when students want to search with several conditions at once. For example, a student might look for courses where the name includes “Math”, the day is Monday and start time is after 9 a.m. With linear search, we can check all these conditions together in one go. Binary search on the other hand only works well with one condition at a time. We would need to sort by course name, then sort again by day, search again and keep repeating. Therefore, this makes process much slower and complicated.

2.3.2 Why We Chose QuickSort (Not MergeSort, HeapSort, or BubbleSort)

In our system, students can sort their courses in different ways. It can by course name, by day of the week or by start time. To do this, we use quick sort.

Quick sort function by choosing one course as a middle point. It can called pivot. It then puts all course that come before the pivot in left side, while the course that come after is in right side. It repeats the same process on each side until everything is in order.

We have considered using merge sort instead. Merge sort is also very fast. However, merge sort uses extra memory to temporarily store courses while it is working. Quick sort uses very little extra memory. More importantly, quick sort is faster.

We did not choose heap sort because heap sort is much slower. Heap sort also requires more complicated code that is harder for programmers to understand and maintain. For our purpose of sorting small course lists, quick sort is the better choice.

Then, in our system, we eventually not choosing bubble sort eventhough bubble sort is our planning before. This is due to we realised the bubble sort is extremely slow. Students would notice the system taking a noticeable pause while their courses are being sorted. Bubble sort mainly used for education teaching only.

Furthermore, quick sort works well in our project is that our courses are in random order. Quick sort's worst case, where it takes much longer time, only happens if the courses are already sorted in the opposite direction of what you want. Since students add courses in random order throughout the semester, this worst case almost never happens. So quick sort always performs at its average speed, which is very fast.

2.4 Why We Use Recursive Backtracking for Timetable Generation

When courses have many sections, the system needs to generate all possible valid schedule combinations. We use a recursive backtracking algorithm. This algorithm works by trying out each possible combination of course sections. It checks each combination for time conflicts and kept only the valid combinations with no conflicts.

Example scenario is a student's takes 4 courses. Each has 3 section choices; there are 3 x 3 x 3 x 3=81 possible schedules. The backtracking algorithm goes through all 81, then rejects those with time conflicts. It only saves the valid ones. This entire process completes within a few seconds, so students can instantly see all their scheduling options.

In total, we chose recursive backtracking because it is efficient, simple and fits the problem well. It explores one course at a time, selects one section option, then recursively checks the rest. If conflict is found, it immediately stops exploring that branch and moves to the next option. As a result, it makes the algorithm much faster than checking all combinations without any shortcut.

2.5 Main Code explanation for the function

2.5.1 hashPassword()

File: **mainwindow.cpp** | **Line: 21-28**

Code:

```
QString MainWindow::hashPassword(const QString &password)
{
    QString salt = "CourseRegSalt2024";
    QString saltedPassword = salt + password + salt;
    QByteArray hash = QCryptographicHash::hash(saltedPassword.toUtf8(),
QCryptographicHash::Sha256);
    return QString(hash.toHex());
}
```

Explanation:

Line	Code	Explanation
1	QString salt = "CourseRegSalt2024";	Define a salt string to add extra security to the password
2	QString saltedPassword = salt + password + salt;	Concatenate salt before and after the password (e.g., "CourseRegSalt2024" + "mypass" + "CourseRegSalt2024")
3	QCryptographicHash::hash(saltedPassword.toUtf8(), QCryptographicHash::Sha256);	Convert the salted password to UTF-8 bytes and generate SHA-256 hash
4	return QString(hash.toHex());	Convert the hash bytes to hexadecimal string and return it

Purpose: This function protects user passwords by converting them into irreversible hash values. Even if the database is compromised, attackers cannot retrieve the original passwords.

2.5.2 addUser()

File: **mainwindow.cpp | Line: 34-41**

Code:

```
bool MainWindow::addUser(const QString& studentID, const QString& hashedPassword)
{
    if (registeredUsers.contains(studentID)) {
        return false;
    }
    registeredUsers.insert(studentID, hashedPassword);
    return true;
}
```

Explanation:

Line	Code	Explanation
1	if (registeredUsers.contains(studentID))	Check if the student ID already exists in the hash table
2	return false;	If duplicate found, return false to indicate registration failed

3	registeredUsers.insert(studentID, hashedPassword);	Insert new user into the hash table with studentID as key and hashedPassword as value
4	return true;	Return true to indicate user was added successfully

Purpose: This function adds a new user to the system while preventing duplicate student IDs from being registered.

2.5.3 validateLogin()

File: **mainwindow.cpp | Line: 47-55**

Code:

```
bool MainWindow::validateLogin(const QString &studentID, const QString &password)
{
    if (!registeredUsers.contains(studentID)) {
        return false;
    }
    QString storedHashedPassword = registeredUsers.value(studentID);
    QString inputHashedPassword = hashPassword(password);
    return (storedHashedPassword == inputHashedPassword);
}
```

Explanation:

Line	Code	Explanation
1	if (!registeredUsers.contains(studentID))	Check if the student ID exists in the hash table
2	return false;	If student ID not found, return false (login failed)
3	QString storedHashedPassword = registeredUsers.value(studentID);	Retrieve the stored hashed password from hash table using student ID as key
4	QString inputHashedPassword = hashPassword(password);	Hash the input password using the same hashPassword() function
5	return (storedHashedPassword == inputHashedPassword);	Compare the two hashed passwords and return true if they match

Purpose: This function verifies user credentials by comparing hashed passwords, ensuring secure authentication without storing plain text passwords.

2.5.4 on_submit_clicked()

File: **mainwindow.cpp | Line: 218-267**

Code:

```
void MainWindow::on_submit_clicked()
{
    QString studentID = ui->lineEdit_StudentID->text().trimmed();
    QString password = ui->lineEdit_Password->text();

    if (studentID.isEmpty() || password.isEmpty()) {
        UIDialogs::showWarning(this, "Input Error", "Please fill in all fields!");
        return;
    }

    if (studentID != lastAttemptedUser) {
        failedLoginAttempts = 0;
        lastAttemptedUser = studentID;
    }

    if (failedLoginAttempts >= MAX_LOGIN_ATTEMPTS) {
        resetPasswordBtn->show();
        UIDialogs::showError(this, "Account Locked",
            QString("Too many failed login attempts (%1)!").arg(MAX_LOGIN_ATTEMPTS));
        return;
    }

    if (validateLogin(studentID, password)) {
        currentUser = studentID;
        failedLoginAttempts = 0;
        saveRememberedUser();
```

```
        UIDialogs::showInfo(this, "Login Successful", "Welcome " + studentID + "!");
        switchToManageCoursesPage();
    } else {
        failedLoginAttempts++;
        int remainingAttempts = MAX_LOGIN_ATTEMPTS - failedLoginAttempts;

        if (remainingAttempts > 0) {
            UIDialogs::showError(this, "Login Failed",
                QString("Invalid Student ID or Password!\n\nRemaining attempts: %1")
                    .arg(remainingAttempts));
        } else {
            resetPasswordBtn->show();
            UIDialogs::showError(this, "Account Locked",
                "Too many failed login attempts!\n\nYour account has been locked.");
        }
        ui->lineEdit_Password->clear();
    }
}
```

Explanation:

Line	Code	Explanation
1-2	QString studentID = ... QString password = ...	Get the student ID and password from input fields, trim whitespace from student ID
3-5	if (studentID.isEmpty() || password.isEmpty())	Validate that both fields are not empty, show warning if empty
6-8	if (studentID != lastAttemptedUser)	Reset failed attempts counter if a different user is trying to login
9-12	if (failedLoginAttempts >= MAX_LOGIN_ATTEMPTS)	Check if account is locked (3 failed attempts), show error and return if locked
13-17	if (validateLogin(studentID, password))	If credentials are valid: reset counter, save remembered user, show success message, switch to course page

18-27	else { failedLoginAttempts++; ... }	If login failed: increment counter, calculate remaining attempts, show appropriate error message, clear password field

Purpose: This function handles the complete login process including input validation, brute force protection (account lockout after 3 failed attempts), and successful login navigation.

2.5.5 saveUsersToFile()

File: mainwindow.cpp | Line: 417-438

Code:

```
void MainWindow::saveUsersToFile()
{
    QString appDir = QCoreApplication::applicationDirPath();
    QString filePath = appDir + "/users.txt";
    QFile file(filePath);

    if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
        UIDialogs::showWarning(this, "File Error", "Could not save user data to file!");
        return;
    }

    QTextStream out(&file);
    LinkedList<QString> studentIDs = registeredUsers.keys();

    for (int i = 0; i < studentIDs.size(); ++i) {
        QString studentID = studentIDs[i];
        QString password = registeredUsers.value(studentID);
        out << studentID << "," << password << "\n";
    }

    file.close();
}
```

Explanation:

Line	Code	Explanation
1	QString appDir = QCoreApplication::applicationDirPath();	Get the directory path where the application is located
2	QString filePath = appDir + "/users.txt";	Create the full file path by appending "/users.txt"
3	QFile file(filePath);	Create a QFile object with the file path
4-6	if (!file.open(QIODevice::WriteOnly | QIODevice::Text))	Try to open file in write mode, show error if failed
7	QTextStream out(&file);	Create a text stream for writing to the file
8	LinkedList<QString> studentIDs = registeredUsers.keys();	Get all student IDs (keys) from the hash table
9-12	for (int i = 0; i < studentIDs.size(); ++i)	Loop through each student ID, get its password, write in CSV format (studentID,hashedPassword)
13	file.close();	Close the file after writing

Purpose: This function persists all registered users to a text file in CSV format, allowing user data to be saved between application sessions.

2.5.6 loadUsersFromFile()

File: **mainwindow.cpp | Line: 444-471**

Code:

```
void MainWindow::loadUsersFromFile()
{
    QString appDir = QCoreApplication::applicationDirPath();
    QString filePath = appDir + "/users.txt";
    QFile file(filePath);

    if (!file.exists()) return;

    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
```

```
        UIDialogs::showWarning(this, "File Error", "Could not load user data from file!");
        return;
    }

    QTextStream in(&file);
    while (!in.atEnd()) {
        QString line = in.readLine().trimmed();
        if (line.isEmpty()) continue;

        QStringList parts = line.split(",");
        if (parts.size() == 2) {
            QString studentID = parts[0].trimmed();
            QString password = parts[1].trimmed();
            addUser(studentID, password);
        }
    }

    file.close();
}
```

Explanation:

Line	Code	Explanation
1-3	QString appDir = ... QFile file(filePath);	Get application directory and create file path for users.txt
4	if (!file.exists()) return;	If file doesn't exist (first run), return without error
5-7	if (!file.open(QIODevice::ReadOnly | QIODevice::Text))	Try to open file in read mode, show error if failed
8	QTextStream in(&file);	Create a text stream for reading from the file
9	while (!in.atEnd())	Loop until end of file is reached
10-11	QString line = in.readLine().trimmed();	Read one line and remove leading/trailing whitespace

12	if (line.isEmpty()) continue;	Skip empty lines
13	QStringList parts = line.split(",");	Split line by comma to separate studentID and password
14-17	if (parts.size() == 2)	If line has exactly 2 parts, extract studentID and password, add to hash table
18	file.close();	Close the file after reading

Purpose: This function loads previously saved user data from file when the application starts, restoring all registered users to the hash table.

2.5.7 validateStudentID()

File: **signupwindow.cpp | Line: 110-120**

Code:

```
bool SignupWindow::validateStudentID(const QString &studentID)
{
    if (studentID.length() < 5 || studentID.length() > 15) {
        return false;
    }

    QRegularExpression alphanumericRegex("^[A-Za-z0-9]+$");
    return alphanumericRegex.match(studentID).hasMatch();
}
```

Explanation:

Line	Code	Explanation
1	if (studentID.length() < 5 || studentID.length() > 15)	Check if student ID length is between 5 and 15 characters
2	return false;	If length is invalid, return false
3	QRegularExpression alphanumericRegex("^[A-Za-z0-9]+$");	Create a regular expression pattern that matches only letters (A-Z, a-z) and numbers (0-9)

4	return alphanumericRegex.match(studentID).hasMatch();	Test if the student ID matches the pattern, return true if valid

Purpose: This function validates that student IDs follow the required format: 5-15 characters containing only letters and numbers.

2.5.8 validatePasswordStrength()

File: **signupwindow.cpp | Line: 129-145**

Code:

```
bool SignupWindow::validatePasswordStrength(const QString &password)
{
   if (password.length() < 6) {
      return false;
   }

   bool hasLetter = false;
   bool hasDigit = false;

   for (int i = 0; i < password.length(); ++i) {
      QChar c = password[i];
      if (c.isLetter()) hasLetter = true;
      if (c.isDigit()) hasDigit = true;
   }

   return hasLetter && hasDigit;
}
```

Explanation:

Line	Code	Explanation
1-2	if (password.length() < 6) return false;	Check if password has at least 6 characters, return false if too short
3-4	bool hasLetter = false; bool hasDigit = false;	Initialize two boolean flags to track if password contains letter and digit

5	for (int i = 0; i < password.length(); ++i)	Loop through each character in the password
6	QChar c = password[i];	Get the character at current index
7	if (c.isLetter()) hasLetter = true;	If character is a letter (A-Z or a-z), set hasLetter to true
8	if (c.isDigit()) hasDigit = true;	If character is a digit (0-9), set hasDigit to true
9	return hasLetter && hasDigit;	Return true only if password contains both a letter AND a digit

Purpose: This function enforces password security requirements: minimum 6 characters with at least one letter and one number.

2.5.9 timeToInt()

File: **managecoursespage.cpp | Line: 194-227**

Code:

```
int ManageCoursesPage::timeToInt(const QString &time)
{
    QString t = time.toLower().trimmed();
    t = t.replace(" ", "");

    bool isPM = t.contains("pm");

    QString numStr = t;
    numStr.remove("am").remove("pm").remove(".00").remove(".");

    if (numStr.isEmpty()) {
        return -1;
    }

    bool ok;
    int hour = numStr.toInt(&ok);

    if (!ok) {
```

```
        return -1;
    }

    if (isPM && hour != 12) {
        hour += 12;
    } else if (!isPM && hour == 12) {
        hour = 0;
    }

    return hour;
}
```

Explanation:

Line	Code	Explanation
1-2	QString t = time.toLower().trimmed(); t = t.replace(" ", "");	Convert time to lowercase, remove whitespace and spaces
3	bool isPM = t.contains("pm");	Check if the time string contains "pm" to determine AM or PM
4-5	numStr.remove("am").remove("pm")...	Remove all time suffixes (am, pm, .00, .) to get just the number
6-7	if (numStr.isEmpty()) return -1;	If string is empty after removal, return -1 (invalid)
8-9	int hour = numStr.toInt(&ok);	Convert string to integer, ok becomes false if conversion fails
10-11	if (!ok) return -1;	If conversion failed, return -1 (invalid)
12-13	if (isPM && hour != 12) hour += 12;	If PM and not 12, add 12 (e.g., 2pm becomes 14)
14-15	else if (!isPM && hour == 12) hour = 0;	If AM and 12 (midnight), set to 0
16	return hour;	Return the 24-hour format integer

Purpose: This function converts 12-hour time format (e.g., "2pm") to 24-hour integer format (e.g., 14) for easy time comparison and conflict detection.

2.5.10 linearSearchCourses()

File: managecoursespage.cpp | Line: 893-913

Code:

```
LinkedList<int> ManageCoursesPage::linearSearchCourses(const QString &searchText)
{
    LinkedList<int> matchIndices;
    QString lowerSearchText = searchText.toLower();

    for (int i = 0; i < courses.size(); ++i) {
        bool matchFound = false;

        if (courses[i].name.toLower().contains(lowerSearchText)) matchFound = true;
        if (courses[i].day.toLower().contains(lowerSearchText)) matchFound = true;
        if (courses[i].startTime.toLower().contains(lowerSearchText)) matchFound = true;
        if (courses[i].endTime.toLower().contains(lowerSearchText)) matchFound = true;
        if (courses[i].classroom.toLower().contains(lowerSearchText)) matchFound = true;

        if (matchFound) matchIndices.append(i);
    }
    return matchIndices;
}
```

Explanation:

Line	Code	Explanation
1	LinkedList<int> matchIndices;	Create an empty linked list to store indices of matching courses
2	QString lowerSearchText = searchText.toLower();	Convert search text to lowercase for case-insensitive search
3	for (int i = 0; i < courses.size(); ++i)	Loop through each course in the linked list (Linear Search - O(n))
4	bool matchFound = false;	Initialize flag to track if current course matches

5-9	if (courses[i].name.toLower().contains(...))	Check if search text is found in course name, day, start time, end time, or classroom
10	if (matchFound) matchIndices.append(i);	If match found in any field, add the index to results list
11	return matchIndices;	Return the list of matching indices

Time Complexity: O(n) - where n is the number of courses. The algorithm checks every course once.

Purpose: This function implements Linear Search algorithm to find all courses that contain the search keyword in any field.

2.5.11 quickSortCourses()

File: **managecoursespage.cpp | Line: 937-961**

Code:

```
void ManageCoursesPage::quickSortCourses(int sortBy)
{
    if (courses.isEmpty()) return;

    auto compare = [this, sortBy](const Course& c1, const Course& c2) -> bool {
        switch (sortBy) {
            case 0: return c1.name.toLower() < c2.name.toLower();
            case 1: {
                QStringList dayOrder = {"Monday", "Tuesday", "Wednesday", "Thursday",
"Friday", "Saturday", "Sunday"};
                int day1Index = dayOrder.indexOf(c1.day);
                int day2Index = dayOrder.indexOf(c2.day);
                if (day1Index == -1) day1Index = 999;
                if (day2Index == -1) day2Index = 999;
                return day1Index < day2Index;
            }
            case 2: return timeToInt(c1.startTime) < timeToInt(c2.startTime);
            case 3: return c1.classroom.toLower() < c2.classroom.toLower();
```

```
        default: return false;
    }
};

courses.quickSort(compare);
refreshTable();
saveCoursesToFile();
}
```

Explanation:

Line	Code	Explanation
1	if (courses.isEmpty()) return;	If no courses exist, return immediately
2	auto compare = [this, sortBy](...) -> bool	Define a lambda comparison function that takes two courses and returns true if c1 should come before c2
3	case 0: return c1.name.toLower() < c2.name.toLower();	Sort by name alphabetically (A-Z)
4-8	case 1: { QStringList dayOrder = ... }	Sort by day of week (Monday first, Sunday last) using predefined order list
9	case 2: return timeToInt(c1.startTime) < timeToInt(c2.startTime);	Sort by start time (earliest first) using timeToInt() for comparison
10	case 3: return c1.classroom.toLower() < c2.classroom.toLower();	Sort by classroom alphabetically (A-Z)
11	courses.quickSort(compare);	Call QuickSort method on the LinkedList with the comparison function
12-13	refreshTable(); saveCoursesToFile();	Refresh the display table and save sorted courses to file

Time Complexity: O(n log n) average case - QuickSort divides the list and recursively sorts each half.

Purpose: This function implements QuickSort algorithm to sort courses by different criteria (name, day, time, or classroom).

2.5.12 checkTimeConflict()

File: managecoursespage.cpp | Line: 774-793

Code:

```
bool ManageCoursesPage::checkTimeConflict(const Course &newCourse, int excludeRow)
{
    for (int i = 0; i < courses.size(); ++i) {
        if (i == excludeRow) continue;

        const Course &existing = courses[i];
        if (existing.day != newCourse.day) continue;

        int newStart = timeToInt(newCourse.startTime);
        int newEnd = timeToInt(newCourse.endTime);
        int existStart = timeToInt(existing.startTime);
        int existEnd = timeToInt(existing.endTime);

        if (newStart < existEnd && existStart < newEnd) {
            return true;
        }
    }
    return false;
}
```

Explanation:

Line	Code	Explanation
1	for (int i = 0; i < courses.size(); ++i)	Loop through all existing courses
2	if (i == excludeRow) continue;	Skip the course being edited (to allow editing without self-conflict)
3-4	const Course &existing = courses[i];	Get reference to the existing course for comparison
5	if (existing.day != newCourse.day) continue;	If courses are on different days, skip (no conflict possible)
6-9	int newStart = timeToInt(...)	Convert all time strings to integers for comparison

10	if (newStart < existEnd && existStart < newEnd)	Check for time overlap: if new course starts before existing ends AND existing starts before new ends, there is overlap
11	return true;	Conflict found, return true
12	return false;	No conflicts found after checking all courses

Purpose: This function detects scheduling conflicts by checking if a new course's time overlaps with any existing course on the same day.

2.5.13 saveCoursesToFile()

File: **managecoursespage.cpp | Line: 819-839**

Code:

```
void ManageCoursesPage::saveCoursesToFile()
{
    createBackup();

    QString appDir = QCoreApplication::applicationDirPath();
    QString filename = QString("%1/courses_%2.txt").arg(appDir, currentUser);
    QFile file(filename);

    if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
        UIDialogs::showWarning(this, "File Error", "Could not save course data to file!");
        return;
    }

    QTextStream out(&file);
    for (const Course &course : courses) {
        out << course.name << "|" << course.day << "|" << course.startTime << "|"
            << course.endTime << "|" << course.classroom << "\n";
    }
    file.close();
}
```

Explanation:

Line	Code	Explanation
1	createBackup();	Create a backup of existing data before saving (safety feature)
2-3	QString filename = QString("%1/courses_%2.txt").arg(appDir, currentUser);	Create user-specific file path (e.g., "courses_12345.txt")
4	QFile file(filename);	Create QFile object with the filename
5-7	if (!file.open(...))	Try to open file in write mode, show error if failed
8	QTextStream out(&file);	Create text stream for writing
9-11	for (const Course &course : courses)	Loop through each course and write in pipe-delimited format (name|day|startTime|endTime|classroom)
12	file.close();	Close the file

Purpose: This function saves all courses to a user-specific text file using pipe-delimited format, with automatic backup creation before each save.

2.5.14 loadCoursesFromFile()

File: **managecoursespage.cpp | Line: 842-873**

Code:

```
void ManageCoursesPage::loadCoursesFromFile()
{
    QString appDir = QCoreApplication::applicationDirPath();
    QString filename = QString("%1/courses_%2.txt").arg(appDir, currentUser);
    QFile file(filename);

    if (!file.exists()) return;

    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        UIDialogs::showWarning(this, "File Error", "Could not load course data from file!");
```

```
        return;
    }

    QTextStream in(&file);
    while (!in.atEnd()) {
        QString line = in.readLine().trimmed();
        if (line.isEmpty()) continue;

        QStringList parts = line.split("|");
        if (parts.size() == 5) {
            Course course;
            course.name = parts[0].trimmed();
            course.day = parts[1].trimmed();
            course.startTime = parts[2].trimmed();
            course.endTime = parts[3].trimmed();
            course.classroom = parts[4].trimmed();
            courses.append(course);
        }
    }
    file.close();
    refreshTable();
}
```

Explanation:

Line	Code	Explanation
1-3	QString filename = ...	Create user-specific file path for the current user's courses
4	if (!file.exists()) return;	If file doesn't exist (first time user), return without error
5-7	if (!file.open(...))	Try to open file in read mode, show error if failed
8	QTextStream in(&file);	Create text stream for reading
9	while (!in.atEnd())	Loop until end of file

10-11	QString line = in.readLine().trimmed();	Read one line, remove whitespace, skip if empty
12	QStringList parts = line.split("|");	Split line by pipe character into parts
13-19	if (parts.size() == 5)	If line has 5 parts, create Course object and assign each field, then append to linked list
20-21	file.close(); refreshTable();	Close file and refresh the display table

Purpose: This function loads the current user's courses from file when they log in, restoring their saved course data.

2.5.15 generateCombinationsRecursive()

File: **timetable.cpp | Line: 549-584**

Code:

```
void TIMETABLE::generateCombinationsRecursive(const
LinkedList<LinkedList<Course>> &groups,
                                              int groupIndex,
                                              LinkedList<Course> &currentCombination)
{
   if (allCombinations.size() >= MAX_COMBINATIONS) {
      return;
   }

   if (groupIndex >= groups.size()) {
      allCombinations.append(currentCombination);
      return;
   }

   const LinkedList<Course> &currentGroup = groups[groupIndex];
   for (int i = 0; i < currentGroup.size(); ++i) {
      if (allCombinations.size() >= MAX_COMBINATIONS) {
         return;
```

```
        }

        const Course &course = currentGroup[i];

        currentCombination.append(course);

        generateCombinationsRecursive(groups, groupIndex + 1, currentCombination);

        currentCombination.removeLast();
    }
}
```

Explanation:

Line	Code	Explanation
1-2	if (allCombinations.size() >= MAX_COMBINATIONS) return;	Limit Check: Prevent generating more than 10,000 combinations to avoid memory issues
3-5	if (groupIndex >= groups.size())	BASE CASE: If all course groups have been processed, save the current combination and return
6	const LinkedList<Course> &currentGroup = groups[groupIndex];	Get the current course group to process
7	for (int i = 0; i < currentGroup.size(); ++i)	RECURSIVE CASE: Try each course option in the current group
8-9	if (allCombinations.size() >= MAX_COMBINATIONS) return;	Check limit before each recursive call
10	const Course &course = currentGroup[i];	Get the current course option
11	currentCombination.append(course);	CHOOSE: Add this course to the current combination
12	generateCombinationsRecursive(groups, groupIndex + 1, currentCombination);	EXPLORE: Recursively process the next course group
13	currentCombination.removeLast();	UNCHOOSE (Backtrack): Remove the course to try the next option

Time Complexity: O(n^m) where n = options per course, m = number of course groups

Purpose: This function uses Recursive Backtracking algorithm to generate all possible timetable combinations by trying every combination of course options.

2.5.16 hasConflict()

File: **timetable.cpp | Line: 587-607**

Code:

```
bool TIMETABLE::hasConflict(const LinkedList<Course> &combination)
{
   for (int i = 0; i < combination.size(); ++i) {
      for (int j = i + 1; j < combination.size(); ++j) {
         const Course &c1 = combination[i];
         const Course &c2 = combination[j];

         if (c1.day != c2.day) continue;

         int start1 = timeToColumn(c1.startTime);
         int end1 = timeToColumn(c1.endTime);
         int start2 = timeToColumn(c2.startTime);
         int end2 = timeToColumn(c2.endTime);

         if (!(end1 <= start2 || end2 <= start1)) {
            return true;
         }
      }
   }
   return false;
}
```

Explanation:

Line	Code	Explanation
1	for (int i = 0; i < combination.size(); ++i)	Outer loop: iterate through each course

2	for (int j = i + 1; j < combination.size(); ++j)	Inner loop: compare with all subsequent courses (avoid duplicate comparisons)
3-4	const Course &c1 = combination[i]; const Course &c2 = combination[j];	Get references to both courses for comparison
5	if (c1.day != c2.day) continue;	If courses are on different days, skip (no conflict)
6-9	int start1 = timeToColumn(...)	Convert all time strings to column numbers for comparison
10	if (!(end1 <= start2 || end2 <= start1))	Check for overlap: if NOT (course1 ends before course2 starts OR course2 ends before course1 starts), there is overlap
11	return true;	Conflict detected, return true
12	return false;	No conflicts found in the entire combination

Purpose: This function checks if a timetable combination has any time conflicts by comparing every pair of courses.

3.0 ADT SPECIFICATIONS

3.1 COURSE ADT

Data Items

- **name**: QString - Course name
- **day**: QString - Day of the week
- **startTime**: QString - Start time (e.g., "8am", "2pm")
- **endTime**: QString - End time (e.g., "11am", "5pm")

classroom: QString - Classroom location

3.2 USERREGISTRY ADT (Hash Table)

Data Items

- **buckets: UserNode** -** Array of linked list heads (for chaining)
- **capacity: int -** Number of buckets in hash table

UserNode Structure:

- **studentID:** QString - Student ID
- **password:** QString – Password
- **next:** UserNode* - Pointer to next node in chain

Operations:

int hashFunction(const QString &key, int capacity)

- **Purpose:** Calculates hash index for a given key
- **Parameters:** key (student ID), capacity (number of buckets)
- **Returns:** int (index between 0 and capacity-1)
- **Logic:** Sum all character unicode values, return sum % capacity

void initRegistry(UserRegistry &registry, int capacity)

- **Purpose:** Initialize hash table with given capacity
- **Parameters:** registry reference, capacity
- **Returns:** None
- **Logic:**
  1. Set registry.capacity = capacity
  2. Allocate array of UserNode pointers
  3. Set all bucket pointers to nullptr

bool addUser(UserRegistry &registry, const QString &studentID, const QString &password)

- **Purpose:** Add new user to registry
- **Parameters:** registry reference, studentID, password
- **Returns:** true if added successfully, false if duplicate exists

Logic:

1. Calculate hash index using hashFunction()
2. Traverse linked list at that bucket
3. If studentID already exists, return false
4. Create new UserNode
5. Insert at head of linked list
6. Return true

bool validateUser(UserRegistry &registry, const QString &studentID, const QString &password)

- **Purpose:** Validate user login credentials
- **Parameters:** registry reference, studentID, password
- **Returns:** true if credentials match, false otherwise

Logic:

1. Calculate hash index using hashFunction()
2. Traverse linked list at that bucket
3. If studentID and password match, return true
4. Return false if not found

bool userExists(UserRegistry &registry, const QString &studentID)

- **Purpose:** Check if user ID already registered
- **Parameters:** registry reference, studentID
- **Returns:** true if exists, false otherwise

Logic:

1. Calculate hash index
2. Traverse linked list
3. Return true if studentID found, else false

void freeRegistry(UserRegistry &registry)

- **Purpose:** Free all allocated memory
- **Parameters:** registry reference
- **Returns:** None

Logic:

1. For each bucket, traverse and delete all nodes
2. Delete buckets array

3.3 COURSELIST ADT (Linked List)

Data Items

- **head: CourseNode* -** Pointer to first node
- **count: int -** Number of courses in list

CourseNode Structure:

- **data**: Course - Course data
- **next**: CourseNode* - Pointer to next node

Operations:

void initCourseList(CourseList &list)

- **Purpose**: Initialize empty course list
- **Parameters**: list reference
- **Returns**: None
- **Logic**: Set head to nullptr, count to 0

void addCourse(CourseList &list, const Course &course)

- **Purpose**: Add course to end of list
- **Parameters**: list reference, course data
- **Returns**: None

Logic:

1. Create new CourseNode
2. Set node data to course
3. Set node next to nullptr
4. If list empty, set head to new node
5. Else traverse to end and link new node
6. Increment count

Course* getCourse(CourseList &list, int index)

- **Purpose**: Get course at specified index
- **Parameters**: list reference, index
- **Returns**: Pointer to Course, nullptr if invalid index

Logic:

1. Validate index (0 to count-1)
2. Traverse list to index position
3. Return pointer to course data

bool updateCourse(CourseList &list, int index, const Course &course)

- **Purpose**: Update course at specified index
- **Parameters**: list reference, index, new course data
- **Returns**: true if updated, false if invalid index

Logic:

1. Get course pointer using getCourse()
2. If nullptr, return false
3. Copy new course data
4. Return true

bool deleteCourse(CourseList &list, int index)

- **Purpose**: Delete course at specified index
- **Parameters**: list reference, index
- **Returns**: true if deleted, false if invalid index

Logic:

1. Validate index
2. If index 0, update head to head->next
3. Else traverse to index-1, unlink node at index
4. Delete removed node
5. Decrement count
6. Return true

int courseCount(CourseList &list)

- **Purpose**: Get number of courses in list
- **Parameters**: list reference
- **Returns**: int (count)

void clearCourseList(CourseList &list)

- **Purpose**: Remove all courses from list
- **Parameters**: list reference
- **Returns**: None

Logic:

1. Traverse list, delete each node
2. Set head to nullptr, count to 0

void sortCoursesByDayAndTime(CourseList &list)

- **Purpose**: Sort courses by day and start time using bubble sort
- **Parameters**: list reference
- **Returns**: None

Logic:

1. Use bubble sort algorithm
2. Compare courses by day first (Monday=0 to Sunday=6)
3. If same day, compare by start time
4. Swap node data if out of order
5. Repeat until no swaps needed

3.4 MAINWINDOW ADT

Data Items:

• **ui: Ui::MainWindow*** - UI components from Qt Designer

• **signupWindow: SignupWindow*** - Pointer to signup dialog

• **manageCoursesPage: ManageCoursesPage*** - Pointer to course management page

• **registeredUsers**: **UserRegistry**- Manual hash table

Operations:

MainWindow(QWidget *parent = nullptr)

- **Purpose**: Creates login window, initializes UI, sets up default test account
- **Parameters**: parent widget
- **Returns**: MainWindow object

Logic:

1. Initialize UI from Qt Designer
2. Set window title to "Login"
3. Initialize registeredUsers with capacity 10
4. Add default account: ID "12345", password "password123"
5. Connect Login, SignUp buttons to slots

~MainWindow()

- **Purpose**: Cleans up memory
- **Parameters**: None
- **Returns**: None
- **Logic**: Free registeredUsers using freeRegistry()

void on_submit_clicked()

- **Purpose**: Handles login button click
- **Parameters**: None (slot function)
- **Returns**: None

Logic:

1. Get student ID, password from input fields
2. Check if fields are empty, show error if so
3. Validate credentials using validateUser()
4. If invalid, show error message
5. If valid, show success message
6. Call switchToManageCoursesPage()

void on_toggle_mode_clicked()

- **Purpose**: Opens signup window
- **Parameters**: None (slot function)
- **Returns**: None

Logic:

1. Create SignupWindow if not exists
2. Connect userRegistered signal to on_userRegistered slot
3. Show signup window as modal dialog

bool validateLogin(const QString &studentID, const QString &password)

- **Purpose**: Checks if credentials match registered user
- **Parameters**: studentID, password
- **Returns**: true if valid, false otherwise

Logic: Call validateUser(registeredUsers, studentID, password)

void on_userRegistered(const QString &studentID, const QString &password)

- **Purpose**: Handles new user registration from SignupWindow
- **Parameters**: studentID, password (from signal)
- **Returns**: None (slot function)

Logic:

1. Check if student ID already exists using userExists()
2. If duplicate, show error message, return
3. Add user using addUser()
4. Show success message

void switchToManageCoursesPage()

- **Purpose**: Navigates to course management page after login
- **Parameters**: None
- **Returns**: None

Logic:

1. Create ManageCoursesPage if not exists
2. Connect backToLogin signal
3. Hide MainWindow
4. Show ManageCoursesPage

void switchToLoginPage()

- **Purpose**: Returns to login page from course page
- **Parameters**: None
- **Returns**: None

Logic:

1. Hide ManageCoursesPage
2. Clear input fields
3. Show MainWindow

3.5 SIGNUPWINDOW ADT

Data Items

- **ui**: Ui::SignupWindow* - UI components from Qt Designer

Operations:

SignupWindow(QWidget *parent = nullptr)

- **Purpose**: Creates signup dialog
- **Parameters**: parent widget
- **Returns**: SignupWindow object
- **Logic**: Sets up UI, connects Confirm, Back buttons

~SignupWindow()

- **Purpose**: Cleans up UI
- **Parameters**: None
- **Return:** None

Signal: void userRegistered(const QString &studentID, const QString &password)

- **Purpose**: Signal emitted when registration successful
- **Parameters**: studentID, password
- **Returns**: None (signal, no implementation)

void on_pushButton_Confirm_clicked()

- **Purpose**: Validates input, registers user
- **Parameters**: None (slot function)
- **Returns**: None

Logic:

1. Get student ID, password, confirm password from fields
2. Check all fields filled, show error if not
3. Check passwords match, show error if not

4. Emit userRegistered signal
5. Clear fields
6. Close dialog

void on_pushButton_Back_clicked()

- **Purpose**: Cancels registration
- **Parameters**: None (slot function)
- **Returns**: None
- **Logic**: Clear fields, close dialog with reject status

3.6 MANAGECOURSEPAGE ADT

Data Items:

- **ui**: Ui::ManageCoursesPage* - UI components
- **editingRow**: int - Index of course being edited (-1 if not editing)
- **timetableWindow**: TIMETABLE* - Pointer to timetable window
- **loadingDialog**: LoadingDialog* - Pointer to loading dialog
- **Courses:** CourseList- Manual linked list store courses

Operations:

ManageCoursesPage(QWidget *parent = nullptr)

- **Purpose**: Creates course management interface
- **Parameters**: parent widget
- **Returns**: ManageCoursesPage object

Logic:

1. Initialize UI
2. Set window to maximized
3. Initialize courses linked list using initCourseList()
4. Populate day combo box (Monday to Sunday)
5. Populate time combo boxes (8am to 10pm)
6. Set up table (6 columns: checkbox, name, day, time, classroom, actions)
7. Connect buttons to slots
8. Set editingRow to -1

~ManageCoursesPage()

- **Purpose**: Cleans up memory
- **Parameters**: None
- **Returns**: None

Logic: Clear courses using clearCourseList()

void setupConnections()

- **Purpose**: Connects button signals to slot functions
- **Parameters**: None
- **Returns**: None

int timeToInt(const QString &time)

- **Purpose**: Converts time string to 24-hour integer
- **Parameters**: time (e.g., "8am", "2pm")
- **Returns**: int (e.g., 8, 14)

Logic:

1. Remove "am"/"pm" from string
2. Convert to integer
3. If PM and not 12pm, add 12
4. Return hour value

void onAddCourse()

- **Purpose**: Adds new course or updates edited course
- **Parameters**: None (slot function)
- **Returns**: None

Logic:

1. Get input from fields (trim whitespace)
2. Validate all fields not empty
3. Validate end time > start time
4. Check for duplicates (same name, day, time, classroom)
5. If editingRow >= 0: call updateCourse() to update existing
6. Else: create new Course, call addCourse()
7. Call sortCoursesByDayAndTime()
8. Call refreshTable()
9. Call clearForm()
10. Show success message

void onDeleteCourse(int row)

- **Purpose**: Deletes course at specified index
- **Parameters**: row index
- **Returns**: None (slot function)

Logic:

1. Validate row index
2. Call deleteCourse(courses, row)
3. Update editingRow if needed (account for index shift)
4. Call refreshTable()
5. Show confirmation message

void onEditCourse(int row)

- **Purpose**: Loads course into form for editing
- **Parameters**: row index
- **Returns**: None (slot function)

Logic:

1. Set editingRow to row
2. Get course using getCourse(courses, row)
3. Populate form fields with course data
4. Change button text to "Confirm Edit"
5. Focus on first field

void onGenerateTimetable()

- **Purpose**: Opens timetable window with loading animation
- **Parameters**: None (slot function)
- **Returns**: None

Logic:

1. Check if courses exist using courseCount()
2. Create LoadingDialog
3. Connect loadingComplete signal to onLoadingComplete()
4. Start loading animation
5. Show loading dialog

void onLoadingComplete()

- **Purpose**: Creates timetable window, passes course data
- **Parameters**: None (slot function)
- **Returns**: None

Logic:

1. Delete old timetable window if exists
2. Create new TIMETABLE window
3. Call setCoursesData() with pointer to courses
4. Show timetable window

void onViewTimetable()

- **Purpose**: Directly opens timetable without loading animation
- **Parameters**: None (slot function)
- **Returns**: None
- **Logic**: Same as onLoadingComplete()

void refreshTable()

- **Purpose**: Rebuilds entire table from linked list
- **Parameters**: None
- **Returns**: None

Logic:

1. Set row count to courseCount(courses)
2. For each course (traverse linked list):
   - Column 0: Create checkbox widget (centered, blue style)
   - Columns 1-4: Create text items (name, day, time, classroom)
   - Column 5: Create Edit, Delete buttons
   - Connect checkbox to enable/disable buttons
   - Connect checkbox to highlight row
3. Update course count label

void clearForm()

- **Purpose**: Resets all input fields
- **Parameters**: None
- **Returns**: None

Logic:

1. Clear text inputs
2. Reset combo boxes to index 0
3. Set editingRow to -1
4. Change button text to "+ Add Course"
5. Focus on first field

Signal: void backToLogin()

- **Purpose**: Signal emitted when returning to login page
- **Parameters**: None

3.7 TIMETABLE ADT

Data Items

- **ui**: Ui::TIMETABLE* - UI components
- **coursesData: CourseList*** - Pointer to linked list shared from ManageCoursesPage
- **allCombinations: CombinationList** - Linked list to store valid timetable combinations
- **currentCombinationIndex**: int - Current page being displayed

CombinationList Structure (Linked List of Course Arrays):

- Each node contains an array of Course objects representing one valid combination

Operations:

TIMETABLE(QWidget *parent = nullptr)

- **Purpose**: Creates timetable window
- **Parameters**: parent widget
- **Returns**: TIMETABLE object

Logic:

1. Initialize UI
2. Set window to maximized
3. Disable editing in table
4. Set column stretch mode

5. Set row height to 80 pixels
6. Enable word wrapping
7. Connect buttons to slots
8. Set currentCombinationIndex to 0

~TIMETABLE()

- **Purpose**: Cleans up UI and memory
- **Parameters**: None
- **Returns**: None

void setCoursesData(CourseList* courses)

- **Purpose**: Receives course data, generates all combinations
- **Parameters**: Pointer to linked list from ManageCoursesPage
- **Returns**: None

Logic:

1. Store courses pointer in coursesData
2. Clear old combinations
3. Reset currentCombinationIndex to 0
4. Sort courses using sortCoursesByDayAndTime()
5. Call generateAllCombinations()
6. Display first combination if exists
7. Call updatePageLabel()

void populateTimetable()

- **Purpose**: Displays current courses on timetable grid
- **Parameters**: None
- **Returns**: None

Logic:

1. Clear table contents
2. Use deep blue color for all courses
3. For each course:
   - Convert day to row using dayToRow() (0-6)
   - Convert start time to column using timeToColumn() (0-13)
   - Convert end time to column
   - Calculate column span (end - start)

- Create table item with: name, classroom, time
- Set background color, text color, font (8pt, bold)
- Set item at [row, startCol]
- If span > 1, merge cells

void updateStatistics()

- **Purpose**: Updates statistics labels
- **Parameters**: None
- **Returns**: None

Logic:

1. Total courses: call courseCount(coursesData)
2. Total hours: call calculateTotalHours()
3. Conflicts: call detectConflicts()
4. Update UI labels

int calculateTotalHours()

- **Purpose**: Calculates total class hours
- **Parameters**: None
- **Returns**: int (total hours)
- **Logic**: Sum (endColumn - startColumn) for all courses

int detectConflicts()

- **Purpose**: Detects time conflicts
- **Parameters**: None
- **Returns**: int (number of conflicts)

Logic:

1. For each pair of courses (i, j where j > i):
   - Check if same day
   - Convert times to columns
   - Check if time ranges overlap: (start1 < end2 AND start2 < end1)
   - If overlap, increment conflict count
2. Return total conflicts

int timeToColumn(const QString &time)

- **Purpose**: Converts time string to column index
- **Parameters**: time (e.g., "8am", "2pm")
- **Returns**: int (0-13), -1 if invalid

Logic:

1. Remove "am", "pm" from string
2. Convert to integer
3. If PM (not 12pm), add 12
4. If 12am, set to 0
5. Return (hour - 8) for 8am to 9pm range

int dayToRow(const QString &day)

- **Purpose**: Converts day name to row index
- **Parameters**: day (e.g., "Monday")
- **Returns**: int (0-6), -1 if invalid

Logic: Manual lookup - Monday=0, Tuesday=1, ..., Sunday=6

void generateAllCombinations()

- **Purpose**: Generates all possible timetable combinations
- **Parameters**: None
- **Returns**: None

Logic:

1. Group courses by unique key (name|day|time|classroom)
2. Remove exact duplicates within groups
3. Check if any course has multiple time options
4. If no multiple options: create single combination with all courses
5. If multiple options exist: call generateCombinationsRecursive()

void generateCombinationsRecursive(groups, groupIndex, currentCombination)

- **Purpose**: Recursively builds all combinations
- **Parameters**: groups (course groups), groupIndex (current group), currentCombination (current selection)
- **Returns**: None

Logic:

1. Base case: if groupIndex >= groups.size()
   - Save currentCombination to allCombinations , then Return

2. Recursive case: for each course in current group:
   - Add course to currentCombination
   - Recurse with groupIndex + 1
   - Remove course (backtrack)

bool hasConflict(CourseArray &combination)

- **Purpose**: Checks if combination has time conflicts
- **Parameters**: combination array
- **Returns**: true if conflict exists, false otherwise

Logic:

1. For each pair of courses:
   - Check if same day
   - Convert times to columns
   - Check overlap: !(end1 <= start2 OR end2 <= start1)
   - If overlap, return true
2. Return false if no conflicts

void displayCurrentCombination()

- **Purpose**: Displays combination at currentCombinationIndex
- **Parameters**: None
- **Returns**: None

Logic:

1. Validate currentCombinationIndex
2. Get combination from allCombinations
3. Temporarily set coursesData to current combination
4. Call populateTimetable()
5. Call updateStatistics()

void updatePageLabel()

- **Purpose**: Updates page number label and navigation buttons
- **Parameters**: None
- **Returns**: None

Logic:

1. Set page label to "current/total"
2. Show/hide prev/next buttons based on page count
3. Update window title

void onSaveAs()

- **Purpose**: Saves timetable as image file
- **Parameters**: None (slot function)
- **Returns**: None

Logic:

1. Open file dialog (PNG, JPEG)
2. Temporarily disable scrollbars
3. Calculate total table size
4. Resize table to fit all content
5. Create QPixmap, render table to it
6. Restore original size, scrollbar settings
7. Save pixmap to file
8. Show success/error message

void onBack()

- **Purpose**: Closes timetable window
- **Parameters**: None (slot function)
- **Returns**: None

void onPrevPage()

- **Purpose**: Shows previous combination
- **Parameters**: None (slot function)
- **Returns**: None

Logic:

1. Decrement currentCombinationIndex
2. If < 0, wrap to last index
3. Call displayCurrentCombination()
4. Call updatePageLabel()

void onNextPage()

- **Purpose**: Shows next combination
- **Parameters**: None (slot function)
- **Returns**: None

Logic:

1. Increment currentCombinationIndex
2. If >= total combinations, wrap to 0
3. Call displayCurrentCombination()
4. Call updatePageLabel()

void onTogglePage()

- **Purpose**: Toggles between combinations
- **Parameters**: None (slot function)
- **Returns**: None

Logic: Same as onNextPage()

void onDelete()

Purpose: Clears timetable view (not courses in ManageCoursesPage)

Parameters: None (slot function)

Returns: None

Logic:

1. Show confirmation dialog
2. If confirmed:
   - Clear allCombinations
   - Call populateTimetable()
   - Call updateStatistics()
   - Show success message
   - Close window

3.8 LOADING ADT

Data Items

- **progressBar**: QProgressBar* - Progress bar widget
- **loadingLabel**: QLabel* - "LOADING..." label
- **timer**: QTimer* - Timer for progress updates
- **currentProgress**: int - Current progress value (0-100)

LoadingDialog(QWidget *parent = nullptr)

- **Purpose**: Creates loading dialog with progress bar
- **Parameters**: parent widget
- **Returns**: LoadingDialog object

Logic:

1. Set window title "Generating Timetable"
2. Set fixed size 400x150
3. Set modal (blocks interaction with parent)
4. Create label "LOADING..." (white, bold, 20px)
5. Create progress bar (0-100, styled with gradient)
6. Create timer, connect to updateProgress()

~LoadingDialog()

- **Purpose**: Cleans up timer
- **Parameters**: None
- **Returns**: None

Signal: void loadingComplete()

- **Purpose**: Signal emitted when loading reaches 100%
- **Parameters**: None

void startLoading()

- **Purpose**: Starts loading animation
- **Parameters**: None
- **Returns**: None

Logic:

1. Reset currentProgress to 0
2. Update progress bar to 0
3. Start timer with 30ms interval

void updateProgress()

- **Purpose**: Updates progress bar, checks completion
- **Parameters**: None (slot function, called by timer)
- **Returns**: None

Logic:

1. Increment currentProgress by 1
2. Update progress bar value
3. If currentProgress >= 100:
   - Stop timer
   - Wait 300ms
   - Emit loadingComplete signal
   - Close dialog

Summary

8 Main ADT

Course	• Simple struct storing course details (name, day, time, classroom). No methods, direct data access.
UserRegistry	• Manual hash table implementation for user credentials. Uses chaining (linked lists) for collision handling. • Operations: initRegistry, addUser, validateUser, userExists, freeRegistry.
CourseList	• Manual linked list implementation for storing courses. • Operations: initCourseList, addCourse, getCourse, updateCourse, deleteCourse, courseCount, clearCourseList, sortCoursesByDayAndTime.

MainWindow	• Login window. Manages user authentication using UserRegistry hash table. Handles signup window, navigation to course page. (Default test account: "12345" / "password123")
SignupWindow	• Registration dialog. Validates input (password matching), emits signal to MainWindow to save new user.
ManageCoursesPage	• Course management interface. Uses CourseList linked list to store courses. • Provide add/edit/delete operations with duplicate prevention. • Dynamic table with checkboxes, colored buttons. Edit mode tracking.
TIMETABLE	• Timetable display window. Generates all possible combinations of courses using recursive backtracking algorithm. Supports pagination through combinations, conflict detection, and image export.
LoadingDialog	• Simple loading animation with progress bar. Uses QTimer to animate progress from 0 to 100%.

DATA STRUCTURES

Hash Table (UserRegistry):	• Purpose: Store and validate user credentials • Collision Handling: Chaining with linked lists • Hash Function: Sum of character codes modulo capacity • Time Complexity: O(1) average for insert/search
Linked List (CourseList):	• Purpose: Store courses with dynamic size • Operations: Insert at end, delete by index, get by index • Sorting: Bubble sort by day and time • Time Complexity: O(n) for access, O(n^2) for sorting
Combination List:	• Purpose: Store all valid timetable combinations • Generation: Recursive backtracking algorithm • Filtering: Skip combinations with time conflicts

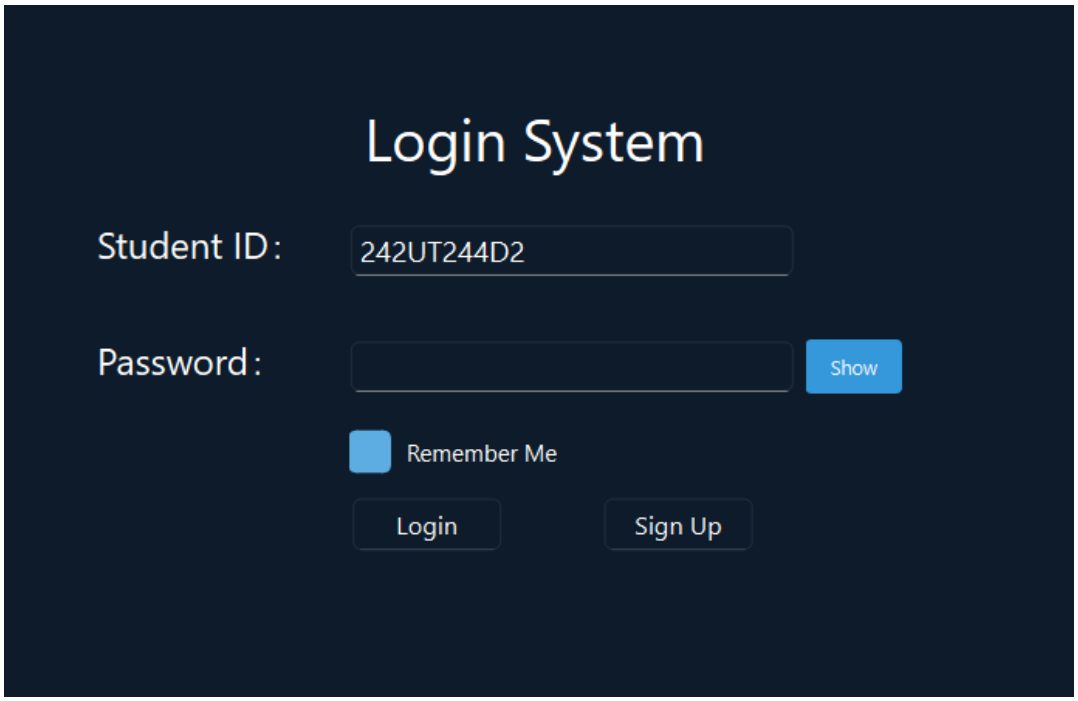
4.0 PROGRAM SCREENSHOT

4.1 Authentication

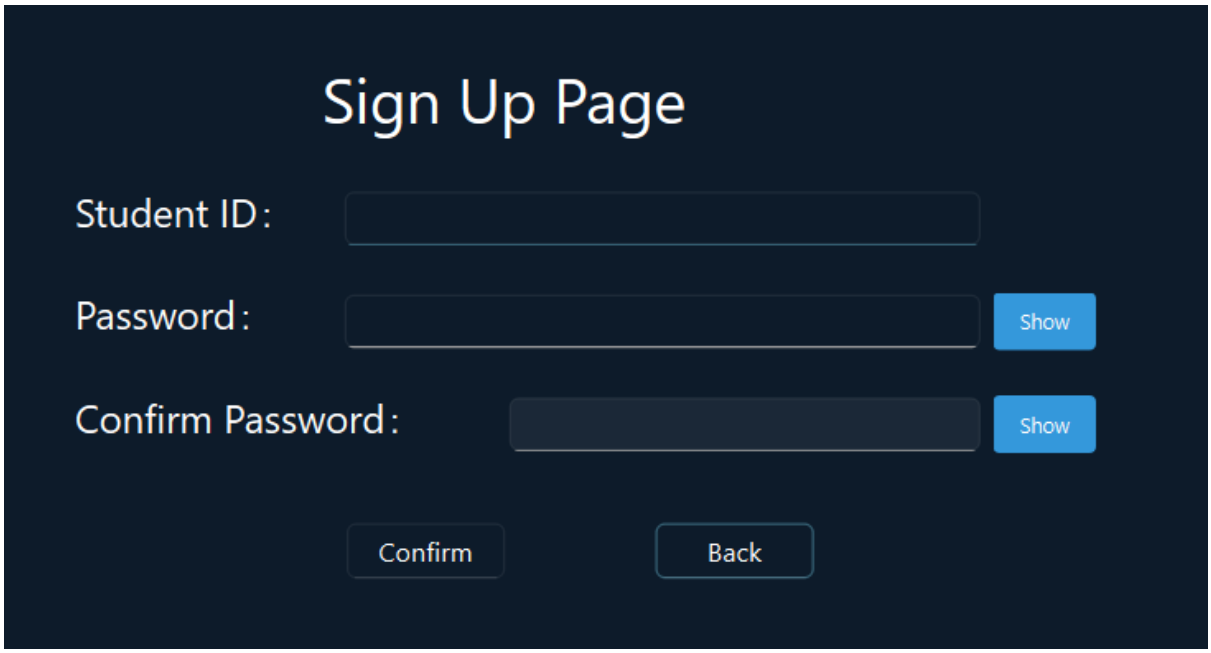
How to open and run our system go to this path

\course_timetable_management\login\deploy\ **Student Course Registration & Timetable Management System.exe**

4.1.1 – Login Sytem Page

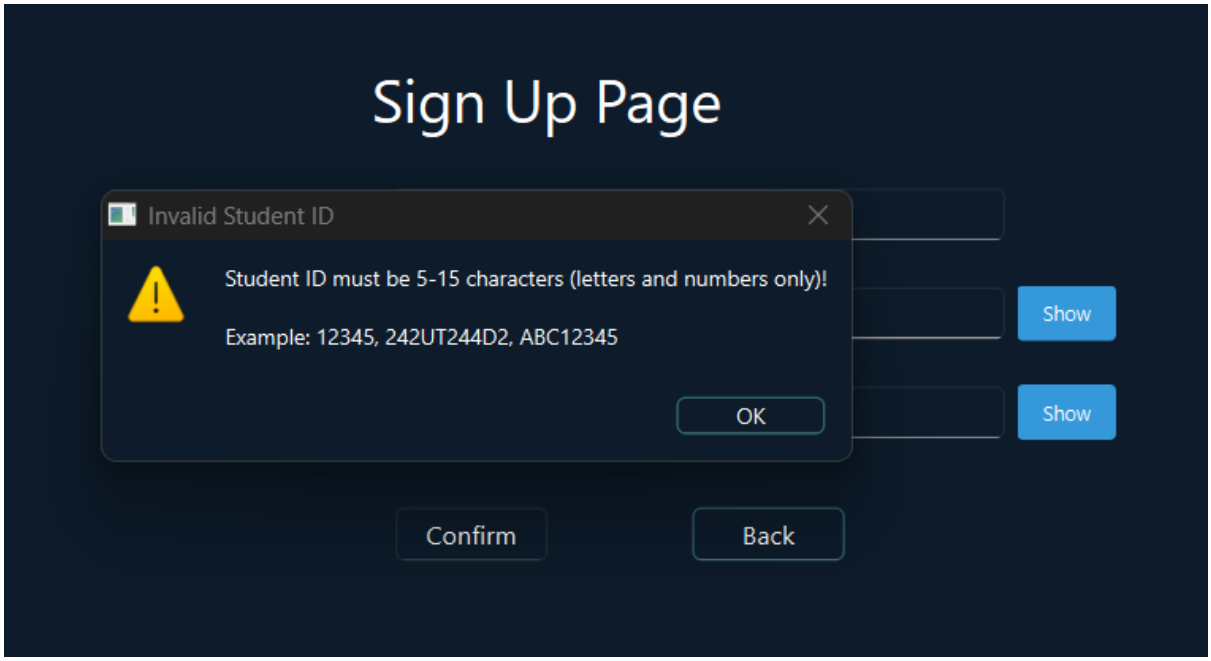


4.1.2 – Sign Up Page (As a New user you need to Sign up)



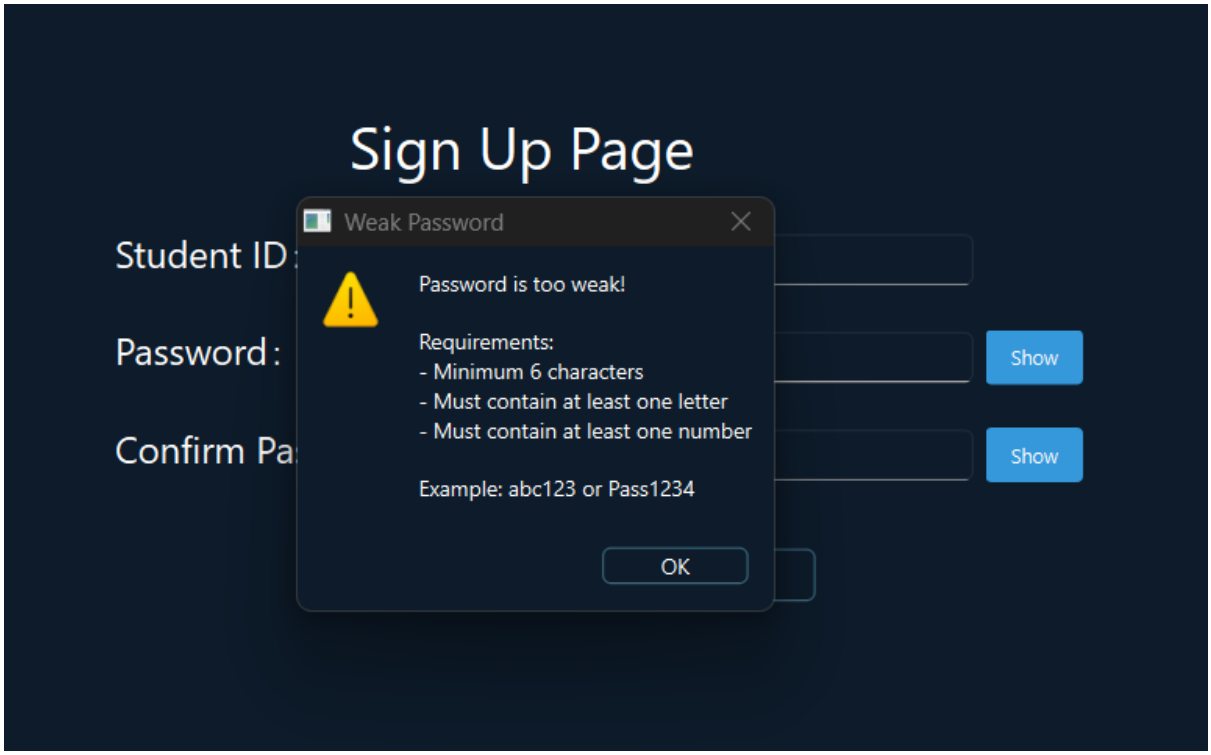
4.1.3 – Student Id (5-15 character)

If your Student ID less than the requirement needed, it will pop up this message for you to rewrite it.

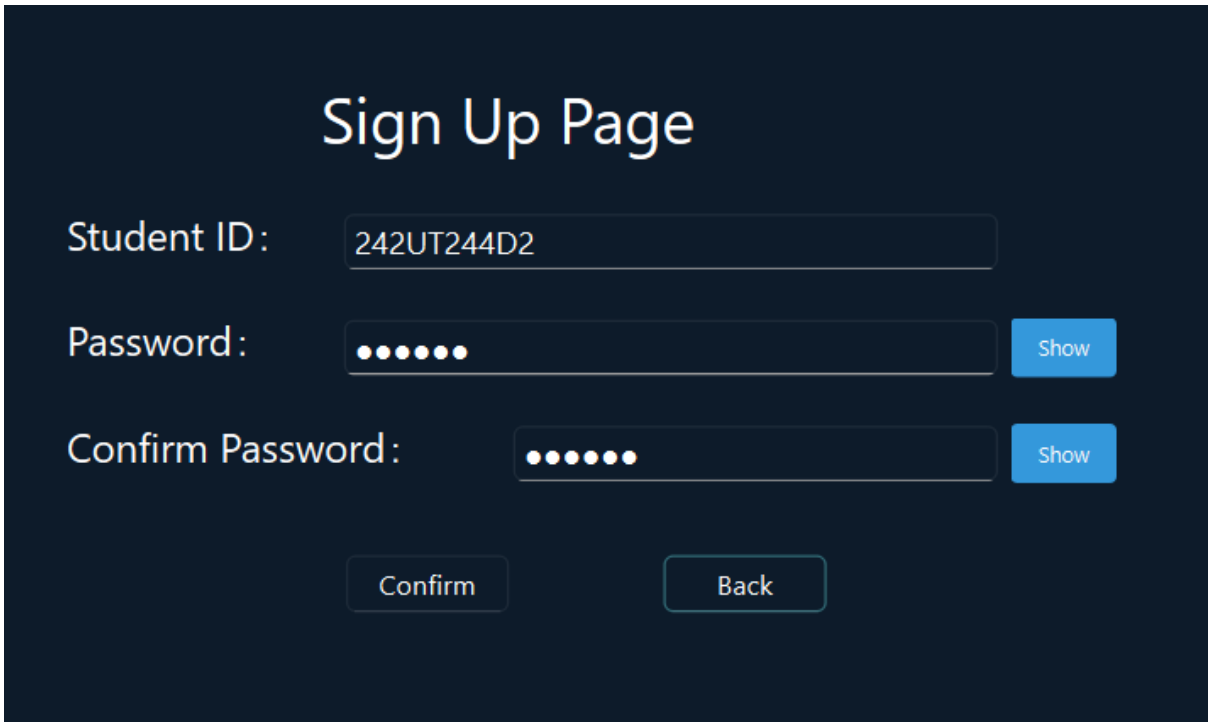


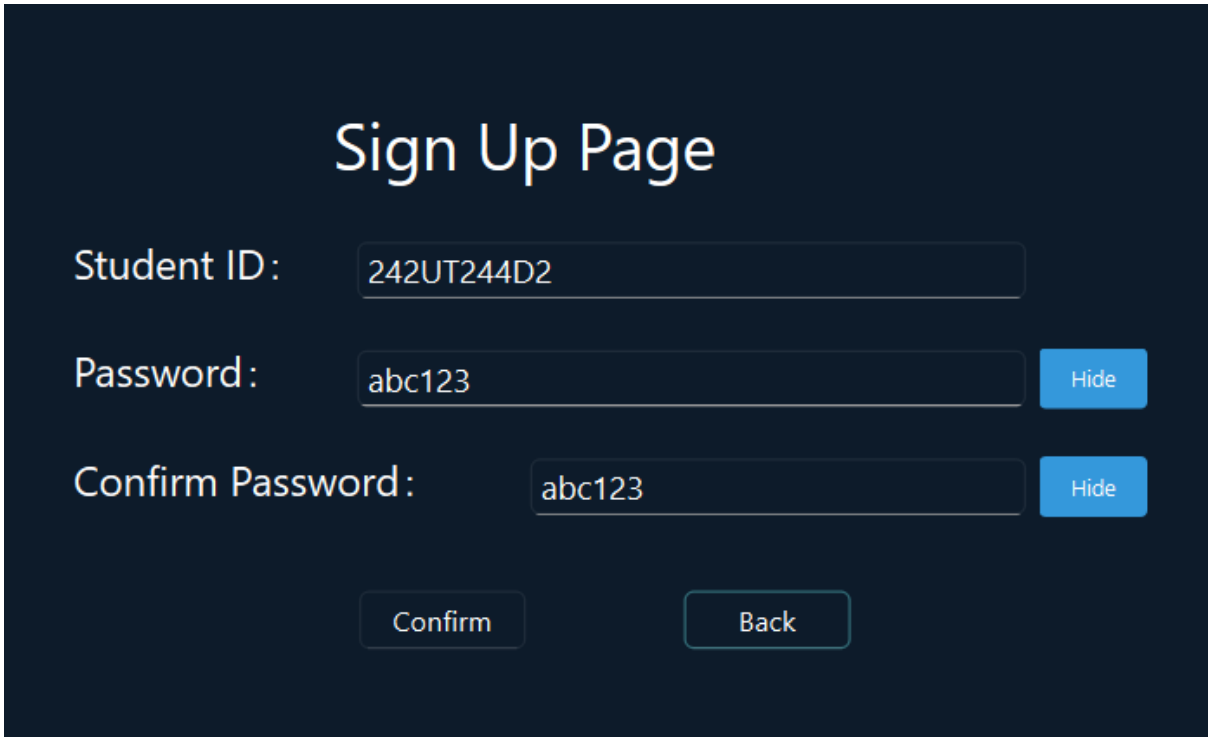
4.1.4 – Password (minimum 6 characters)

If your Password less than the requirement needed, it will pop up this message for you to rewrite it.

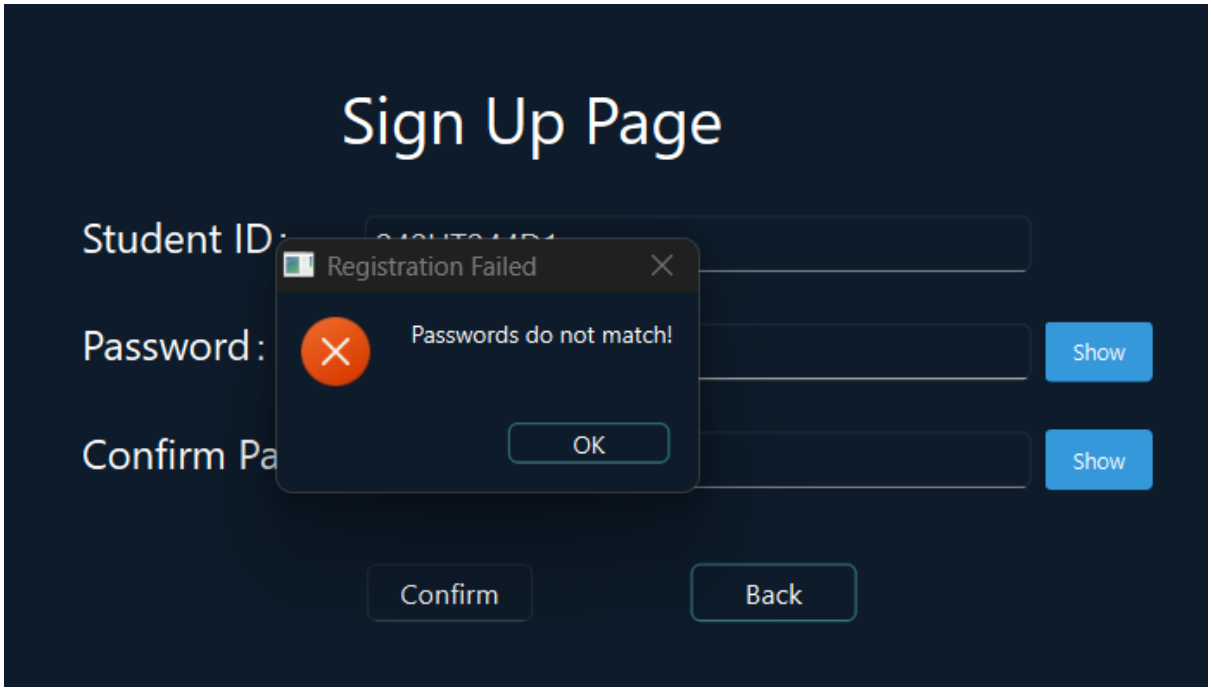


4.1.5 – Sign Up page (Show and Hide the password)

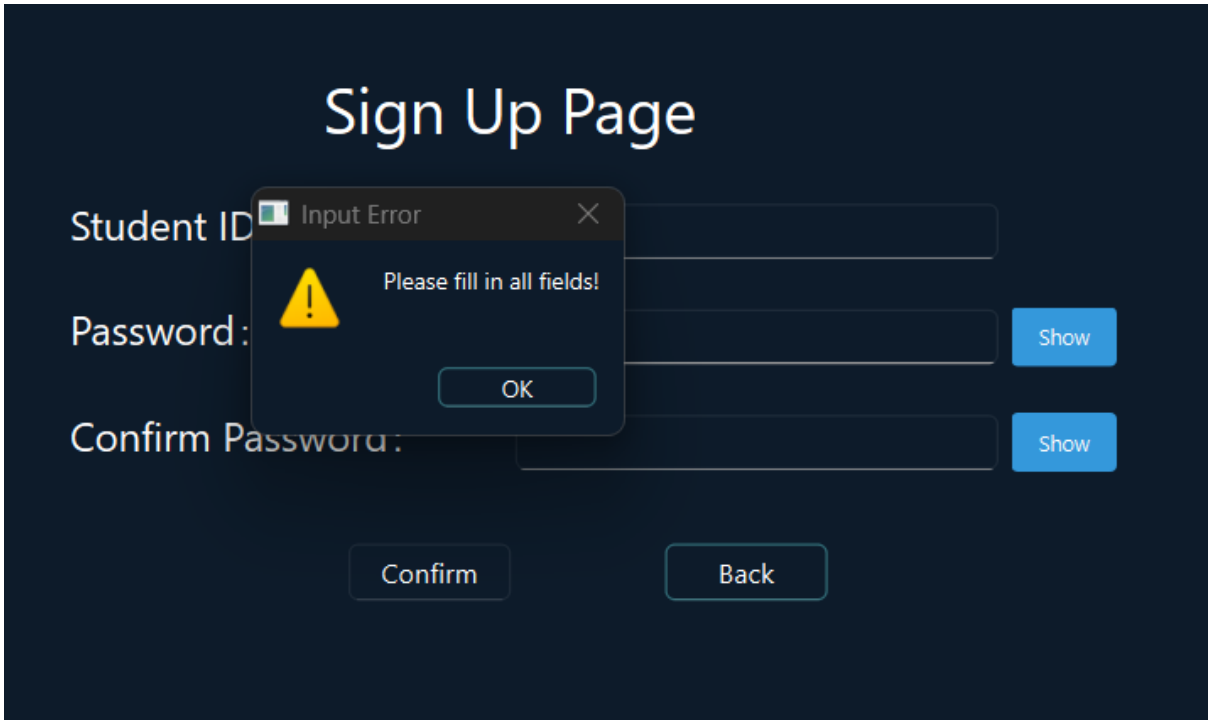




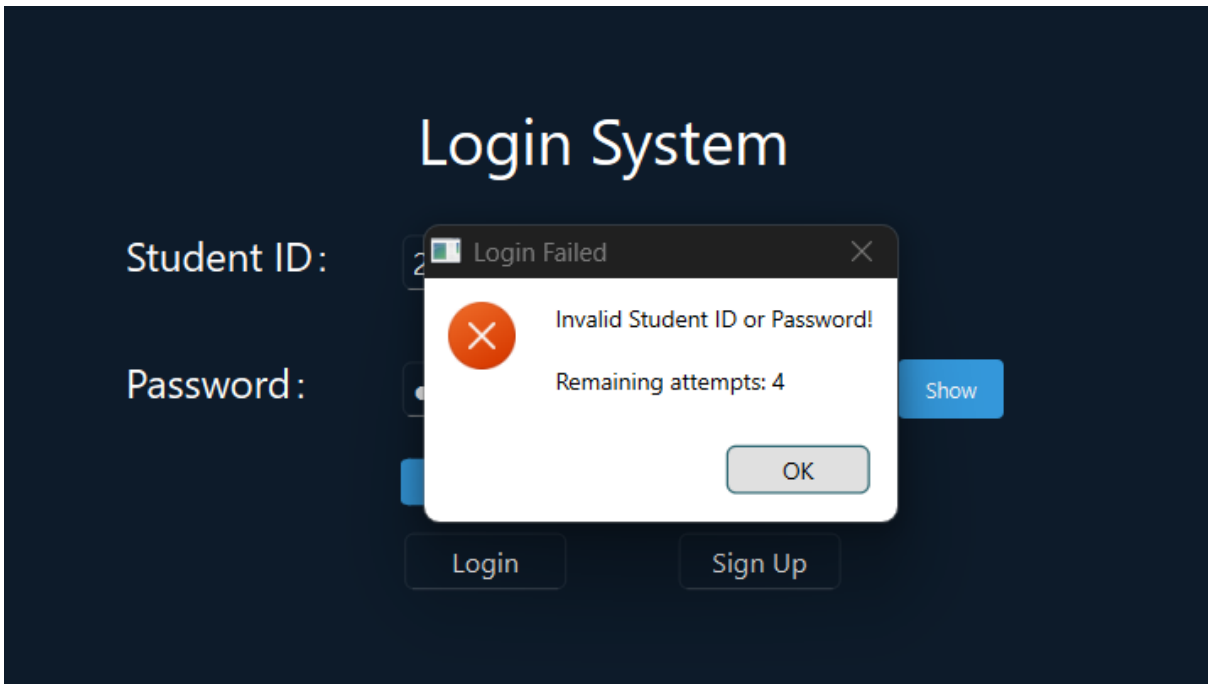
4.1.6 – Sign Up Validation Failed (if password does not match)



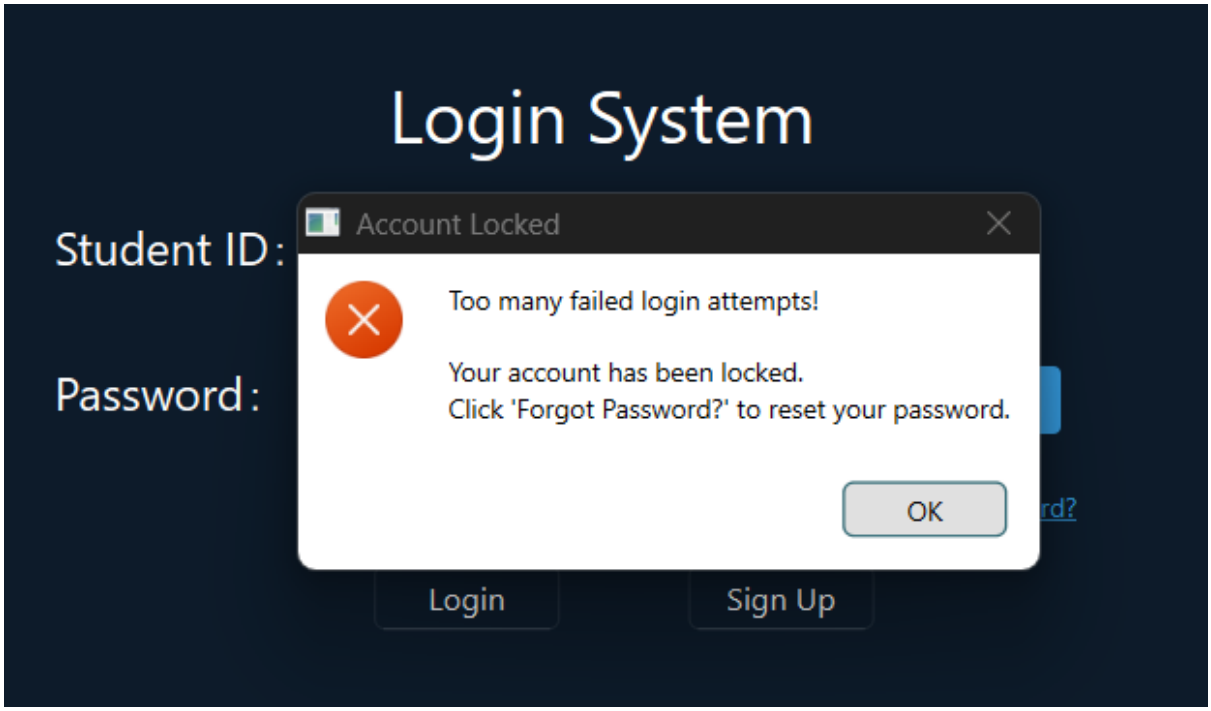
You need to fill the password again to make sure both are same.



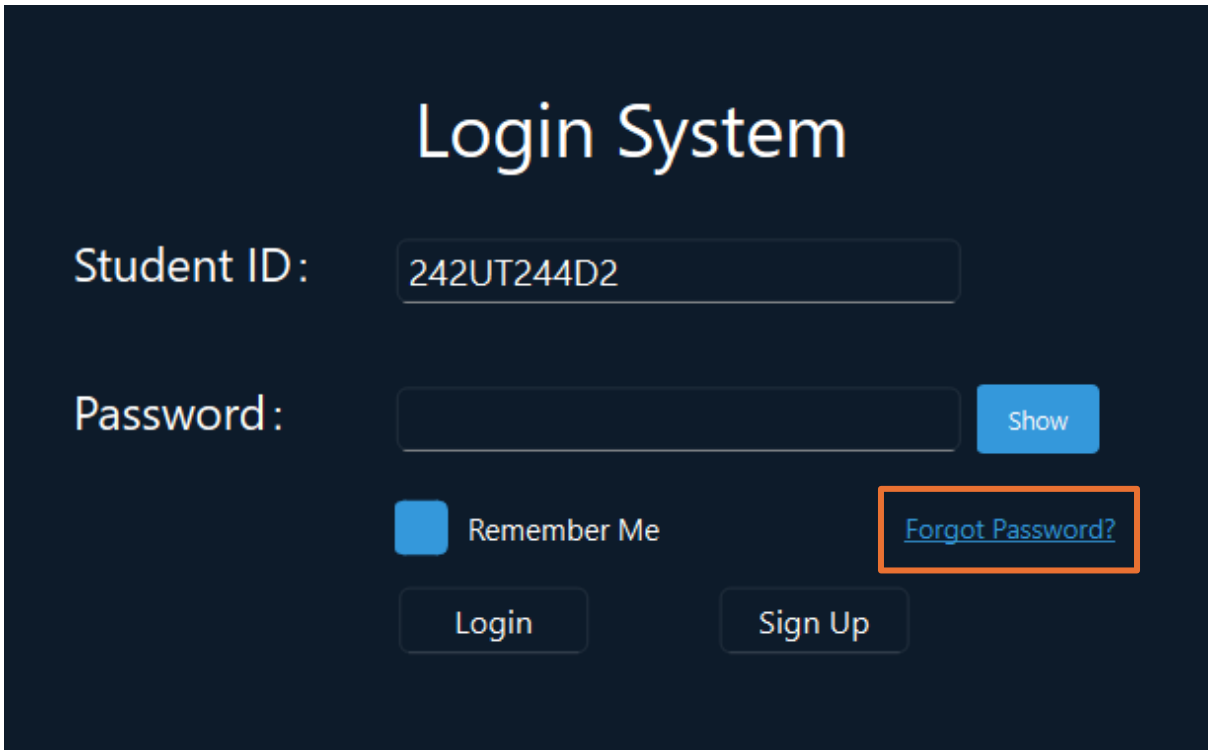
4.1.7 – Login Validation Failed (only allow 5 attempts)



Reset Your Password

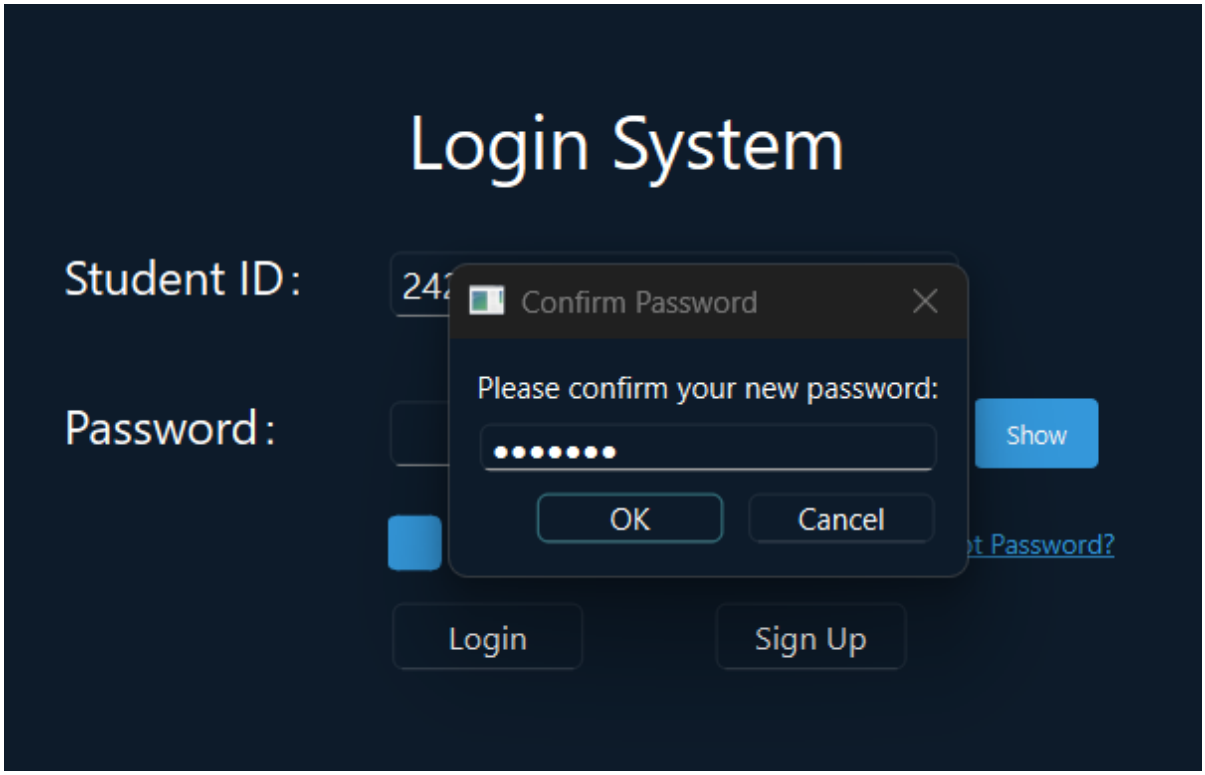


Press Forgot password to change it

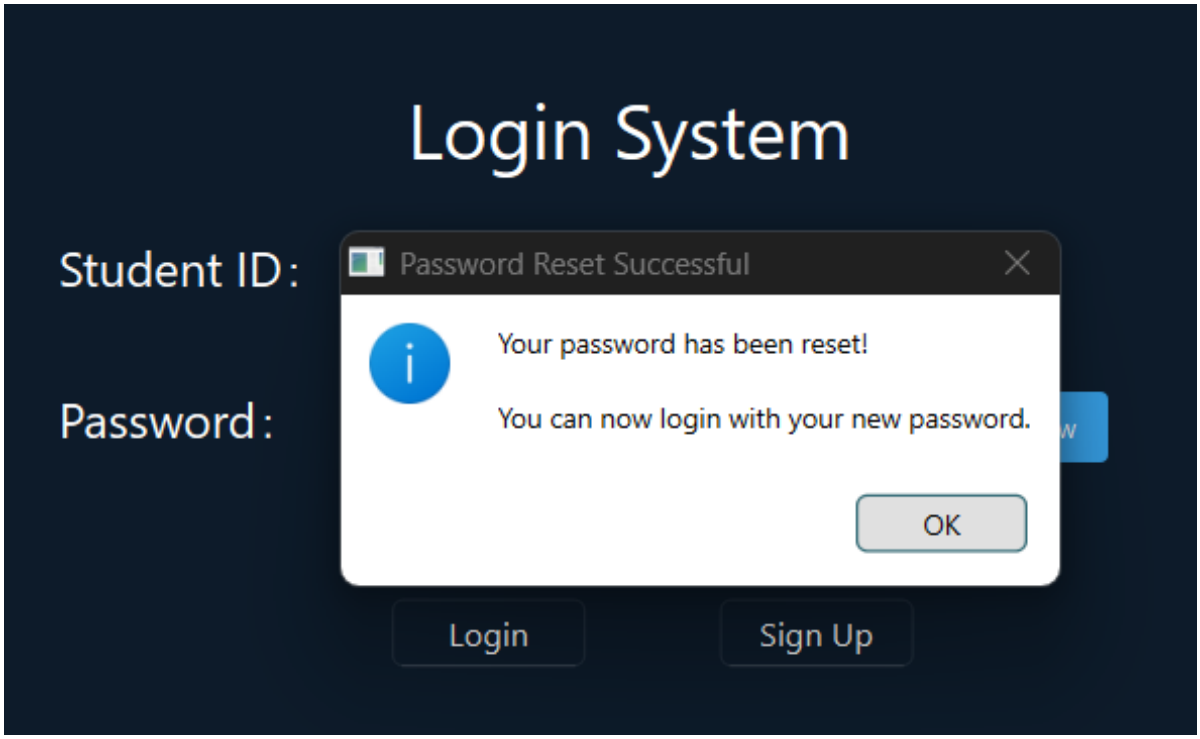


Enter the new password with the required password

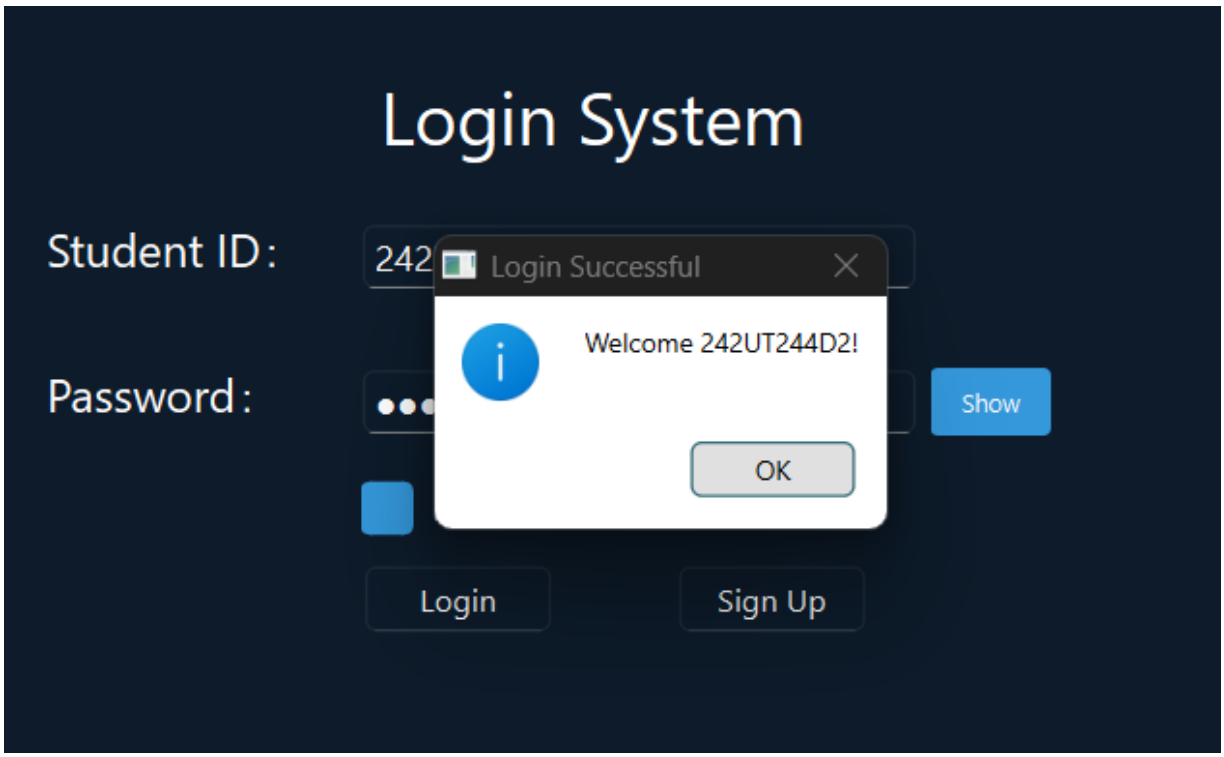
Then type again the new password to confirm the new password.



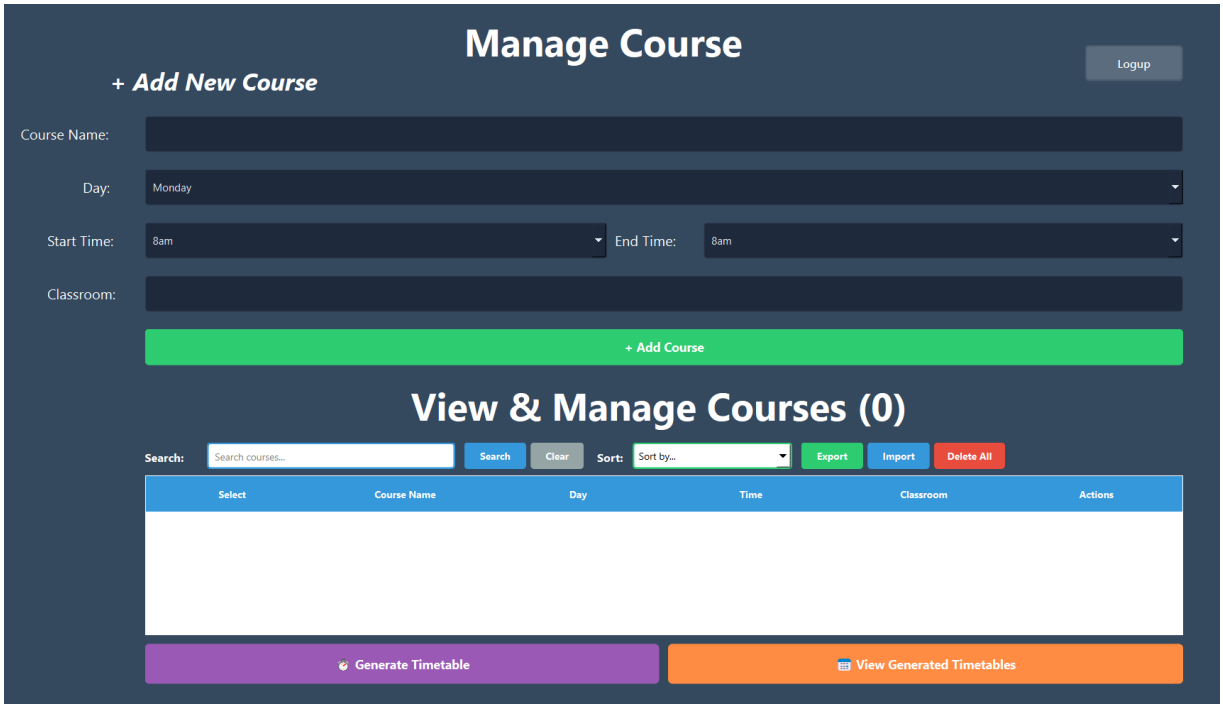
It shows that the password was reset successfully. Press ok.



4.2 – Main Interfaces (After successfully login with correct password)

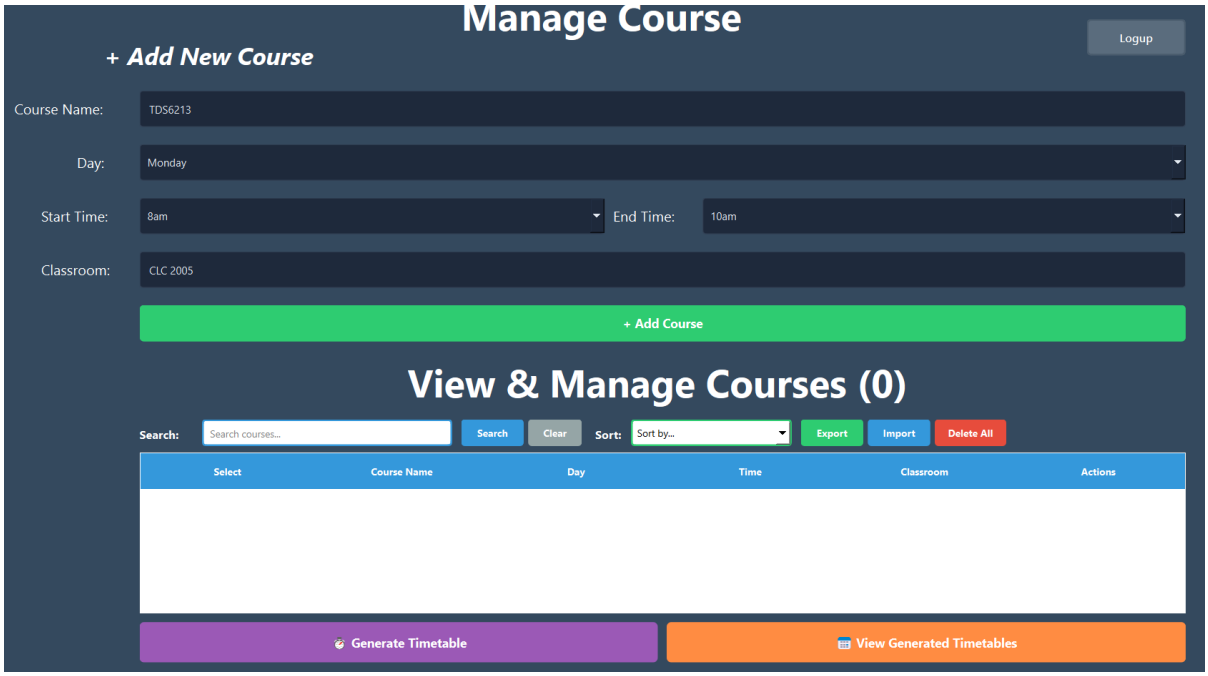


Here is the interface for our system looks like after login to the own account.

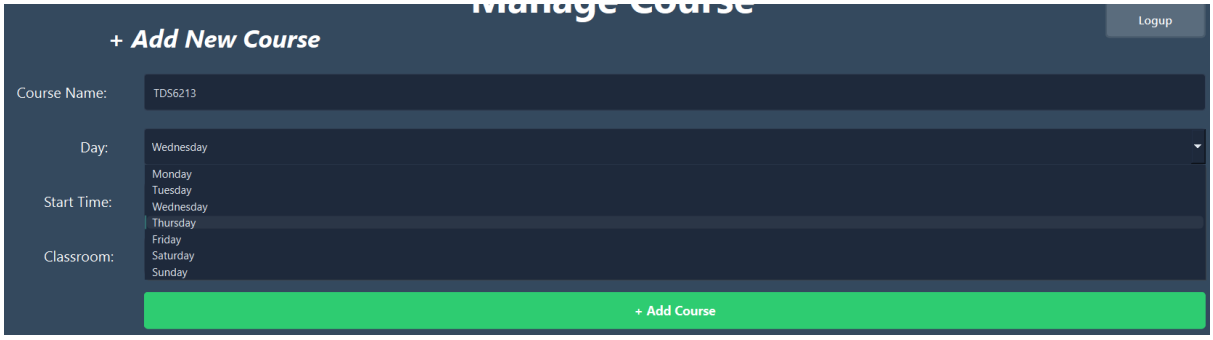


4.2.1 – Add course

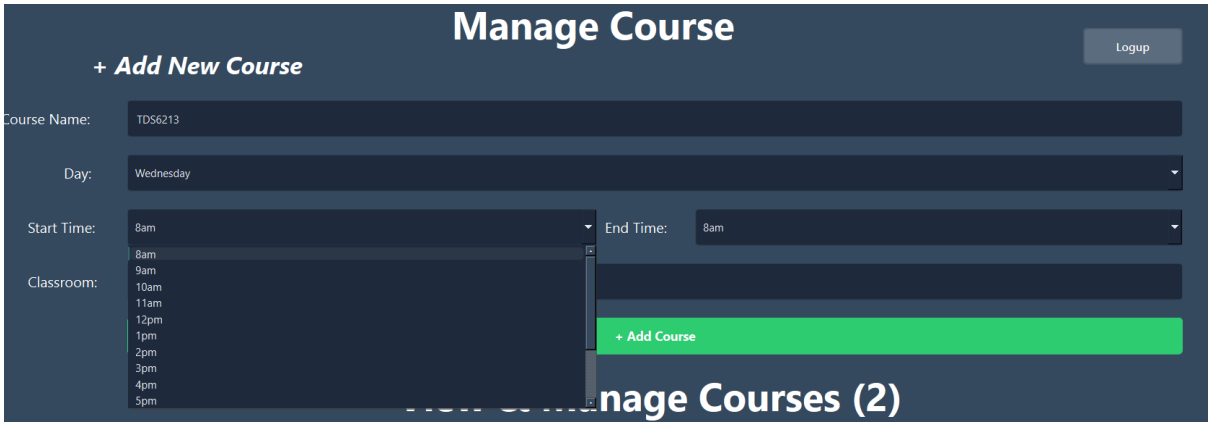
4.2.1.1 - add course (type Course Name, Day, Start time, End time, Classroom)



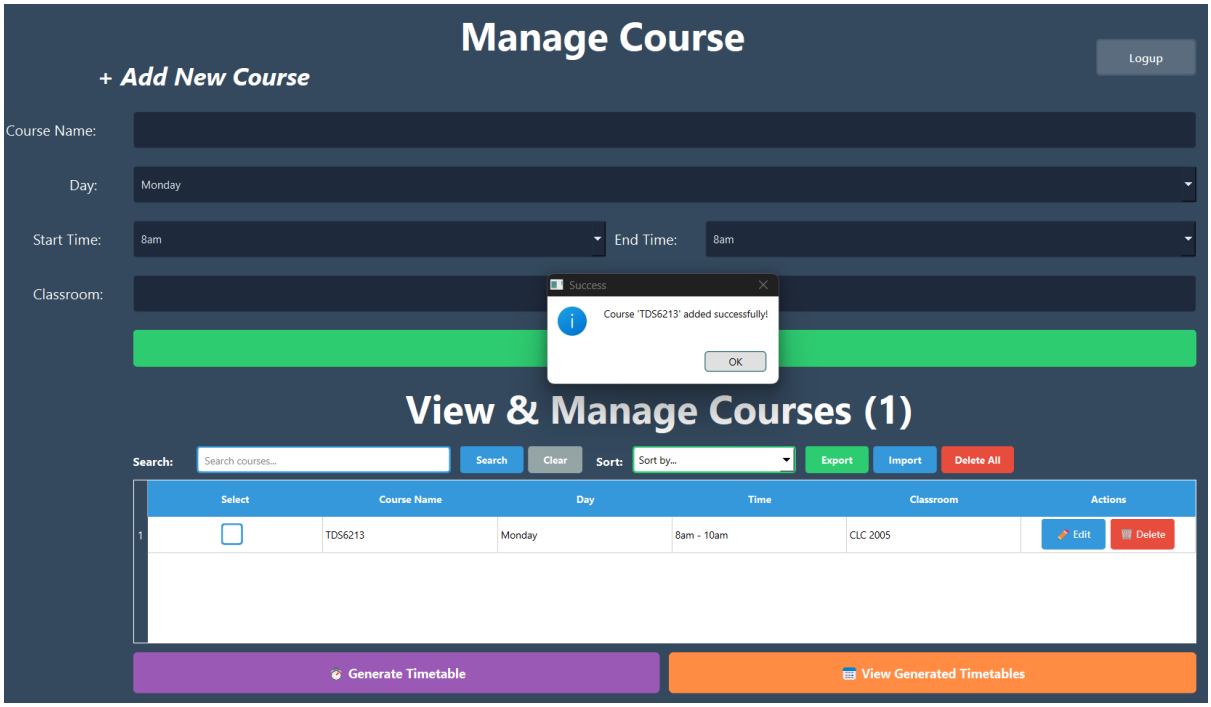
Users can add a new course with a selected day (Monday-Sunday).



Users also need to choose the start time and end time. The selection time is from 8am to 10pm.

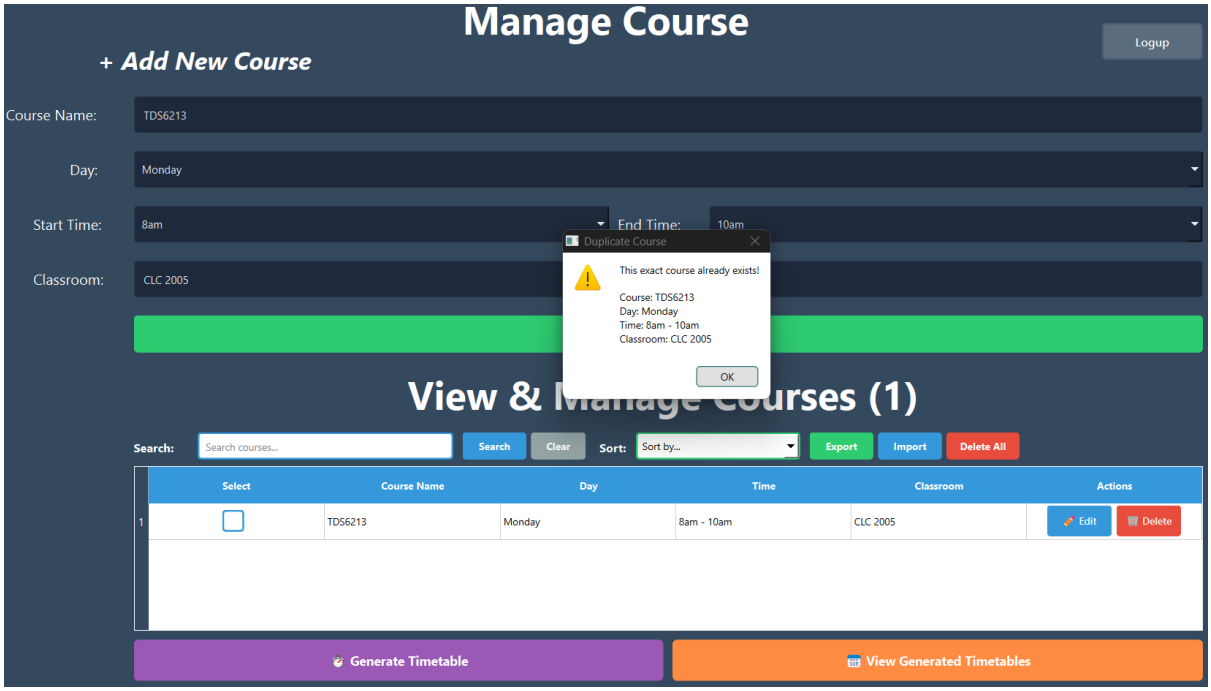


4.2.1.2 – Successfully add the course



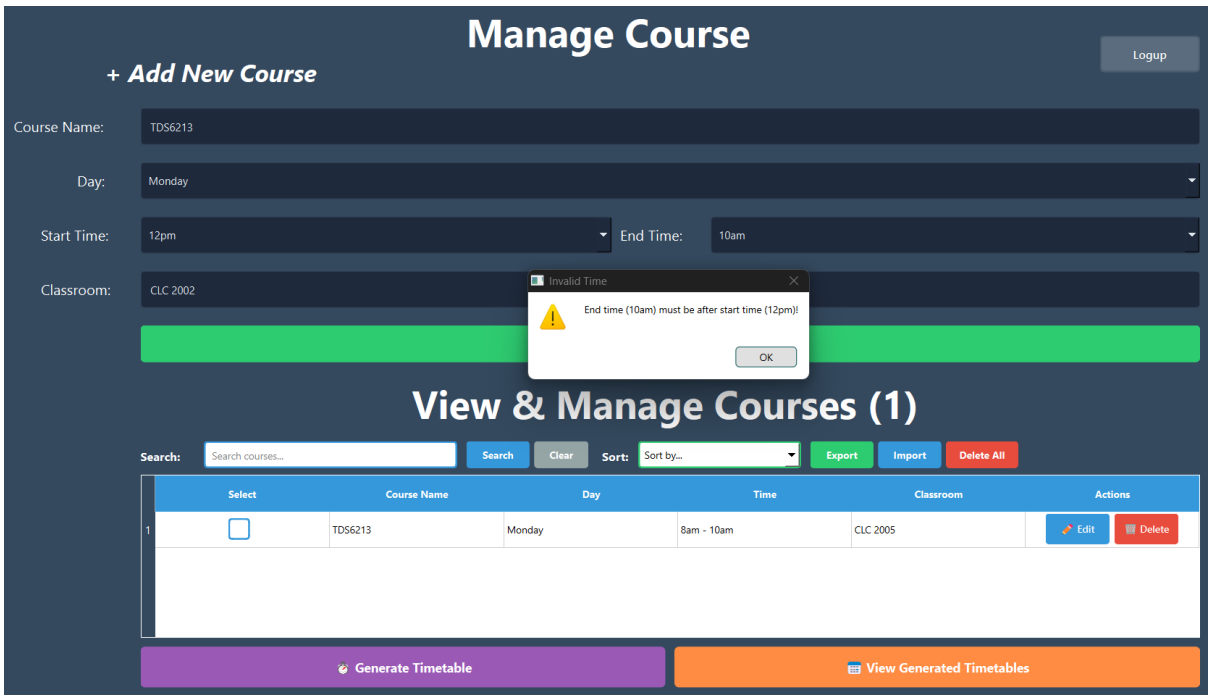
4.2.1.3 – Duplicate Course

If user accidentally duplicate course, the system would prompt the message to notify user about the exact course already exists.



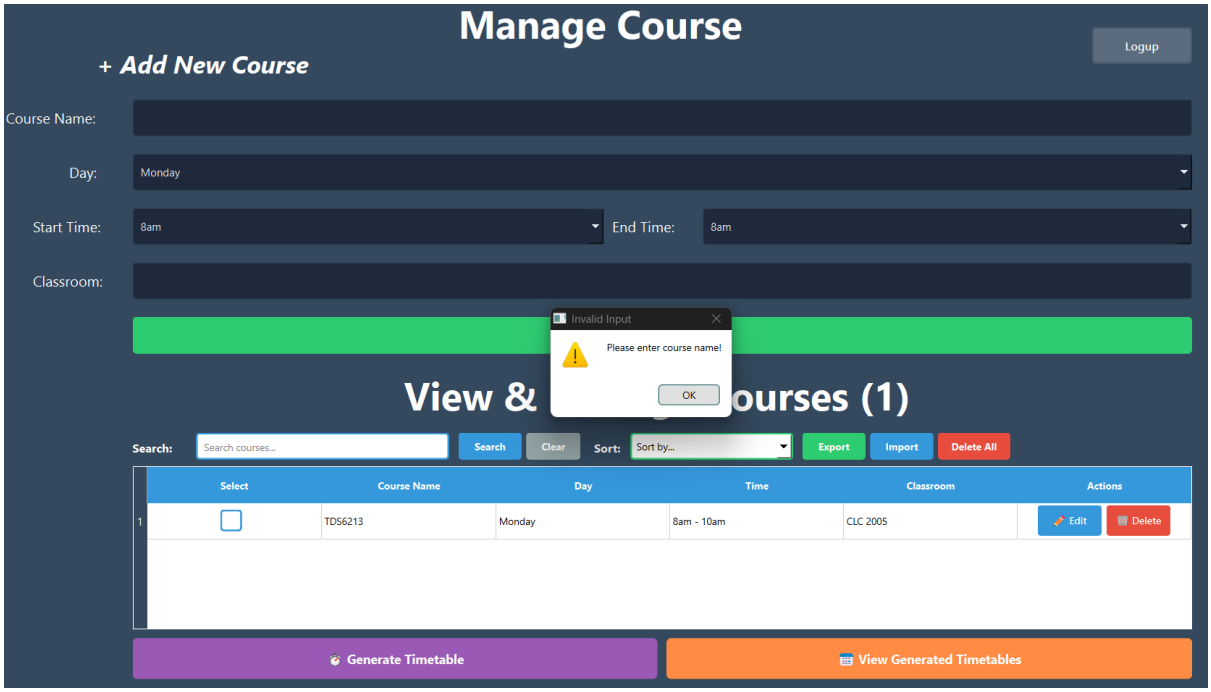
4.2.1.4 – Course Time Error

When user add a new course, but the time is invalid, like the start time is later than end time. The system would prompt end time must be after start time in order to remind user.



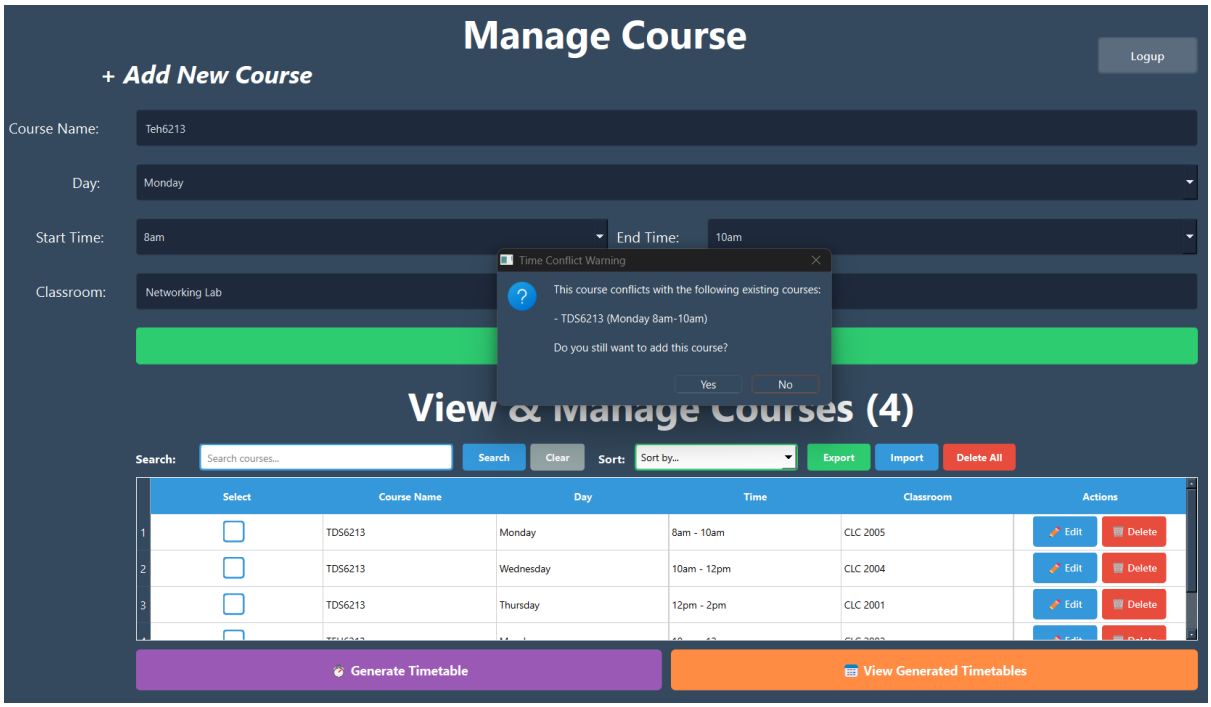
4.2.1.5 – Empty course

It cannot add a new course without any information.



4.2.1.6 – Different Course but same time exist

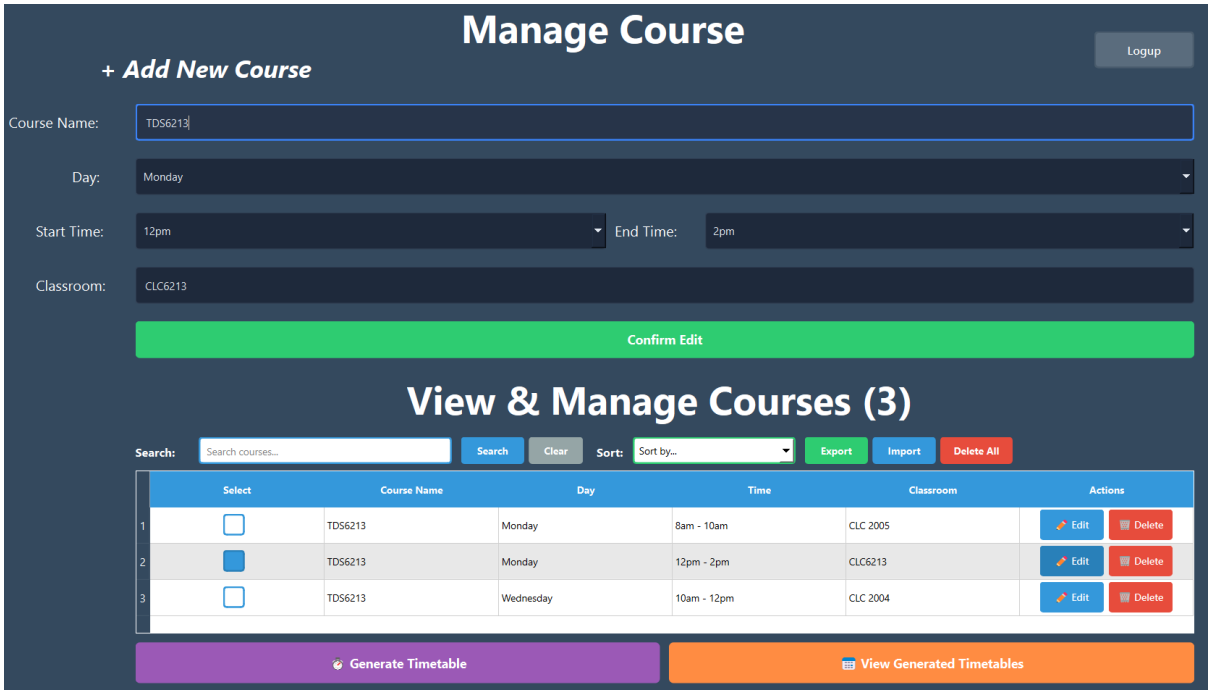
The system also will prompt the message if the time is conflict with another courses, which means the time is clash with other courses. Then, users need to choose whether yes or no to add that course.



4.2.2 – Select Button (Edit / Delete)

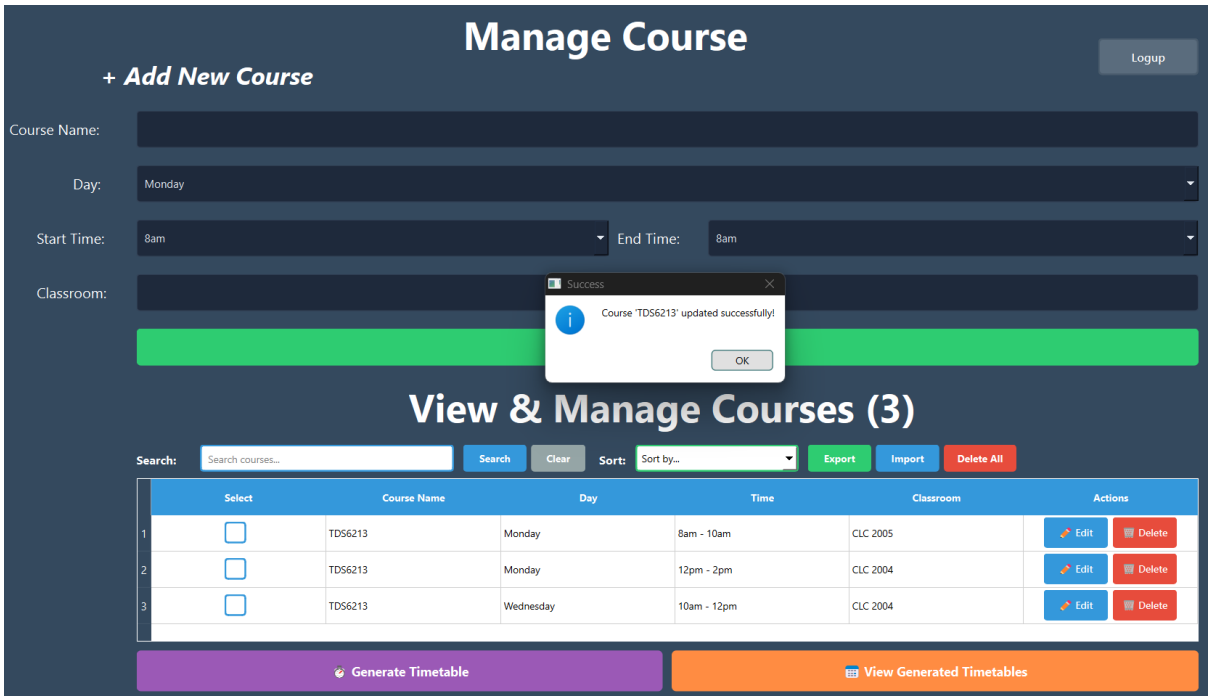
4.2.2.1 – Press the Select check box

When users need to edit the course if they want to modify some information, they need to select that course. When select, it will have blue colour in the small square box.



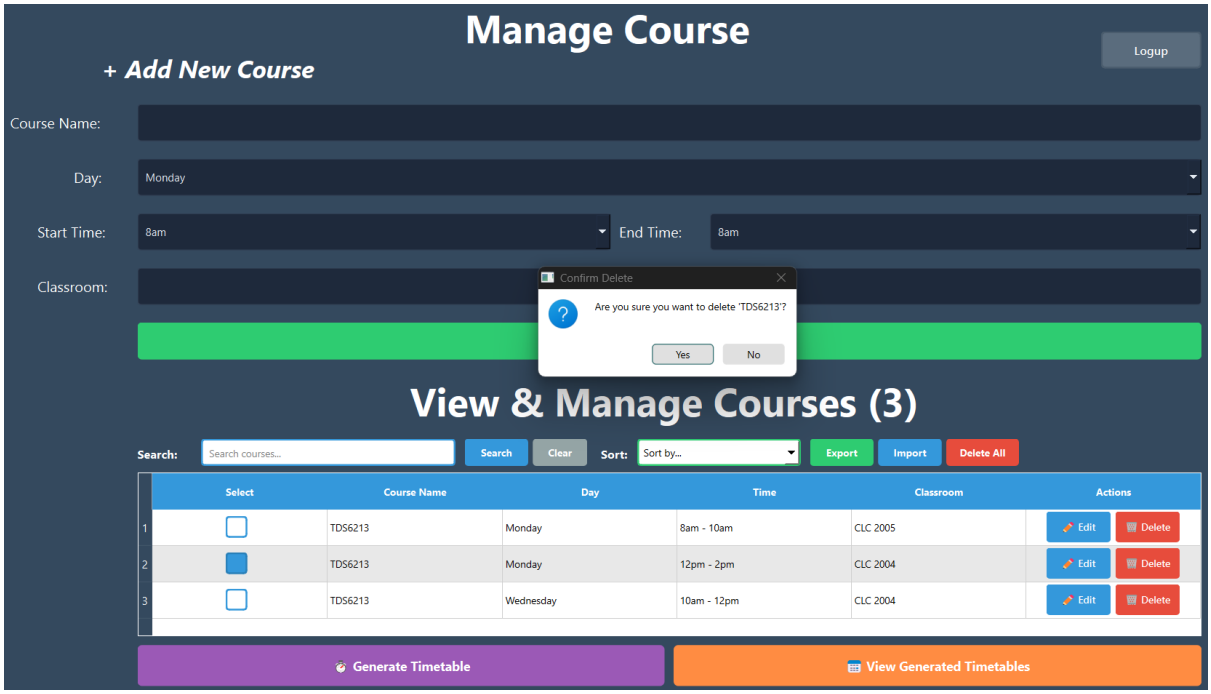
4.2.2.2 – Confirm Edit

After edit, it will show updated successfully with the course name.

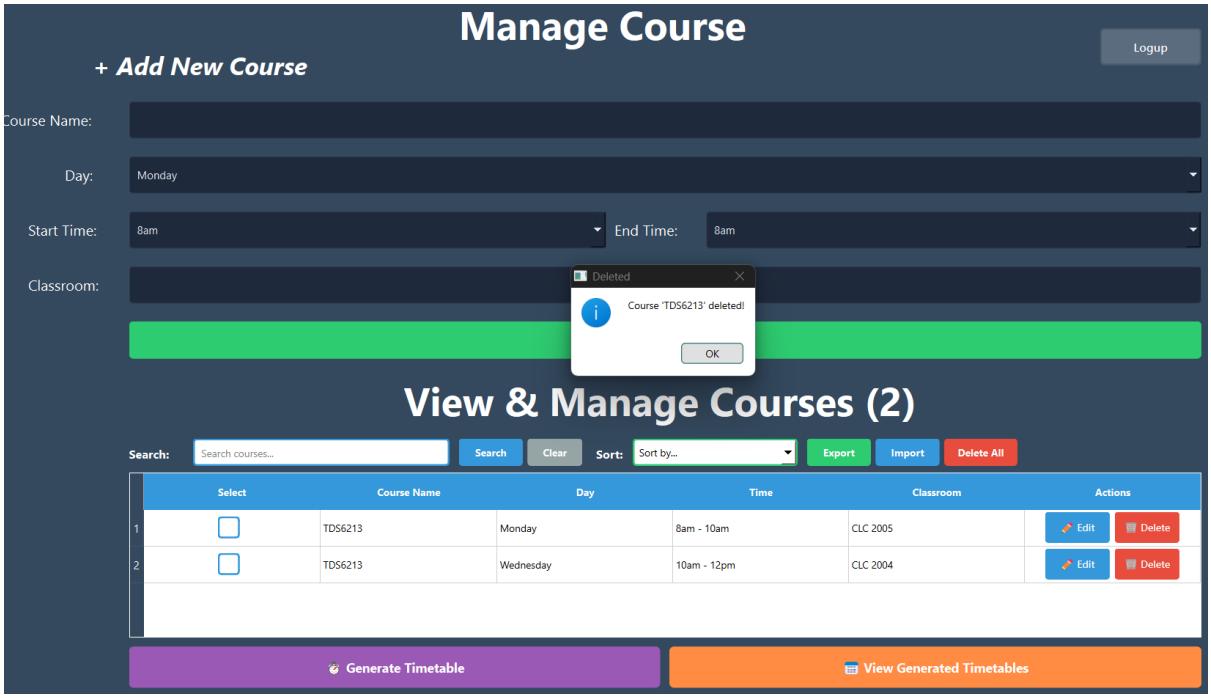


4.2.2.3 – Delete the Selected Course

When user do not want the course to be generated in timetable, they can choose to delete that specific course. Click delete which is red colour rectangle box.

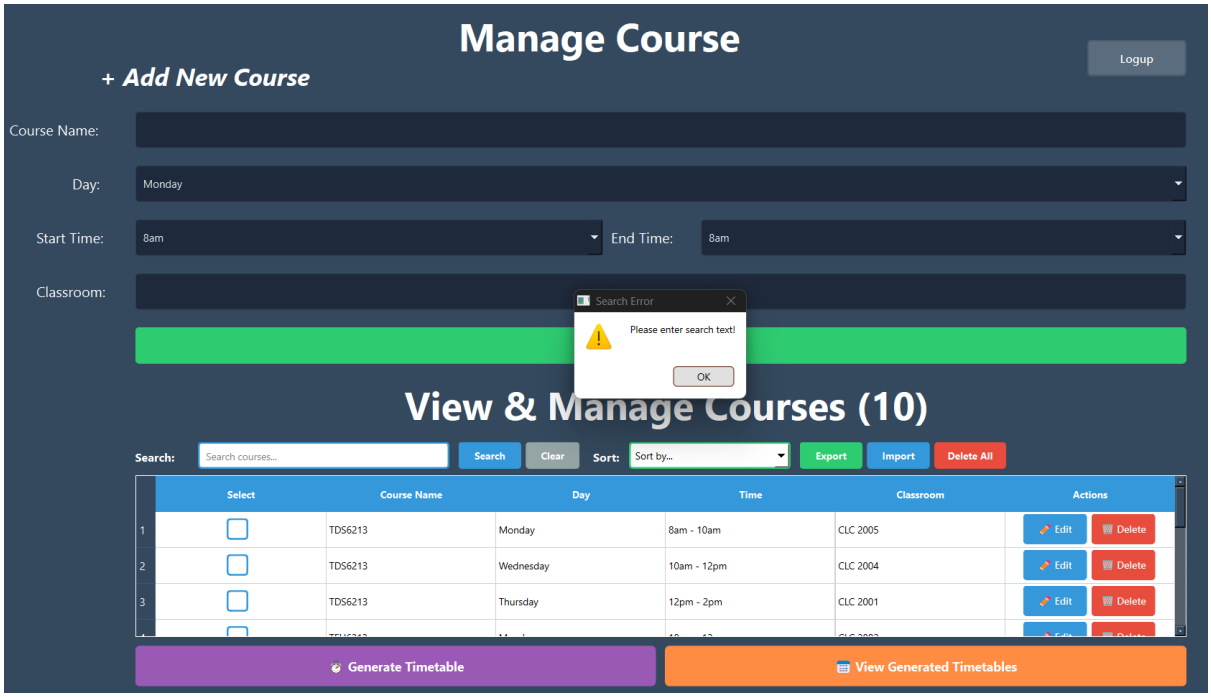


Then that specific course has been deleted.



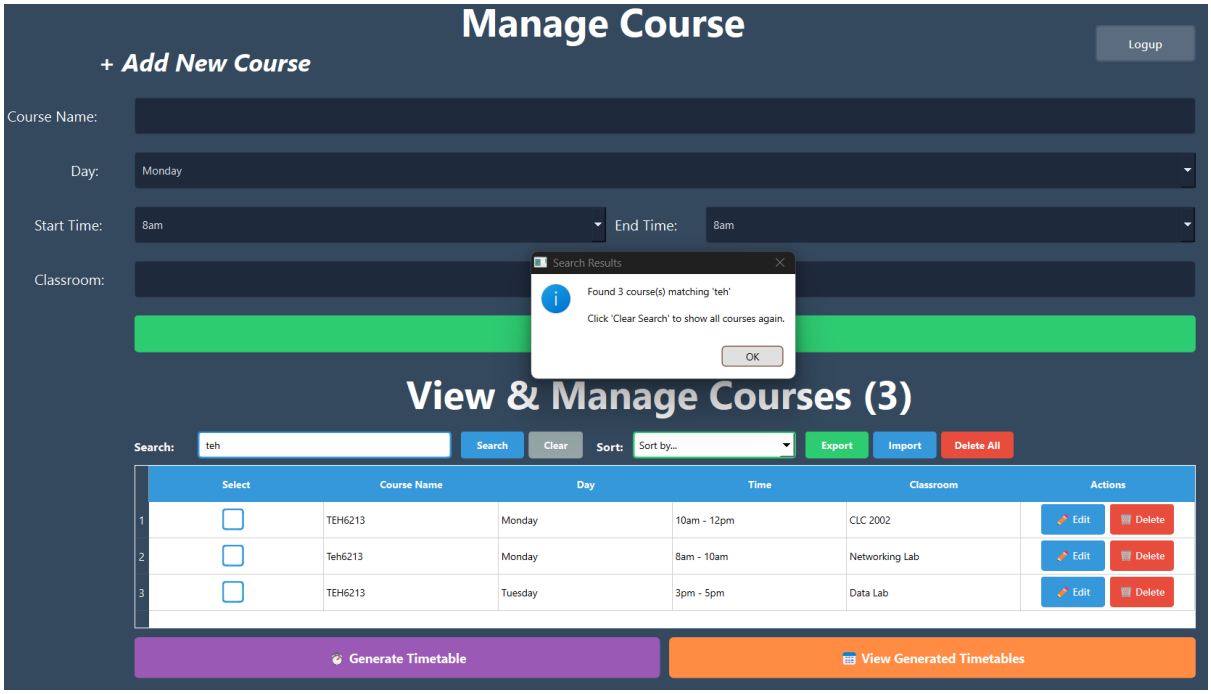
4.2.3 – Search and Clear

Users can also optimize the search bar. They are required to type the course name before clicking the search button. If user has not type anything, it will prompt the message of please enter search text.



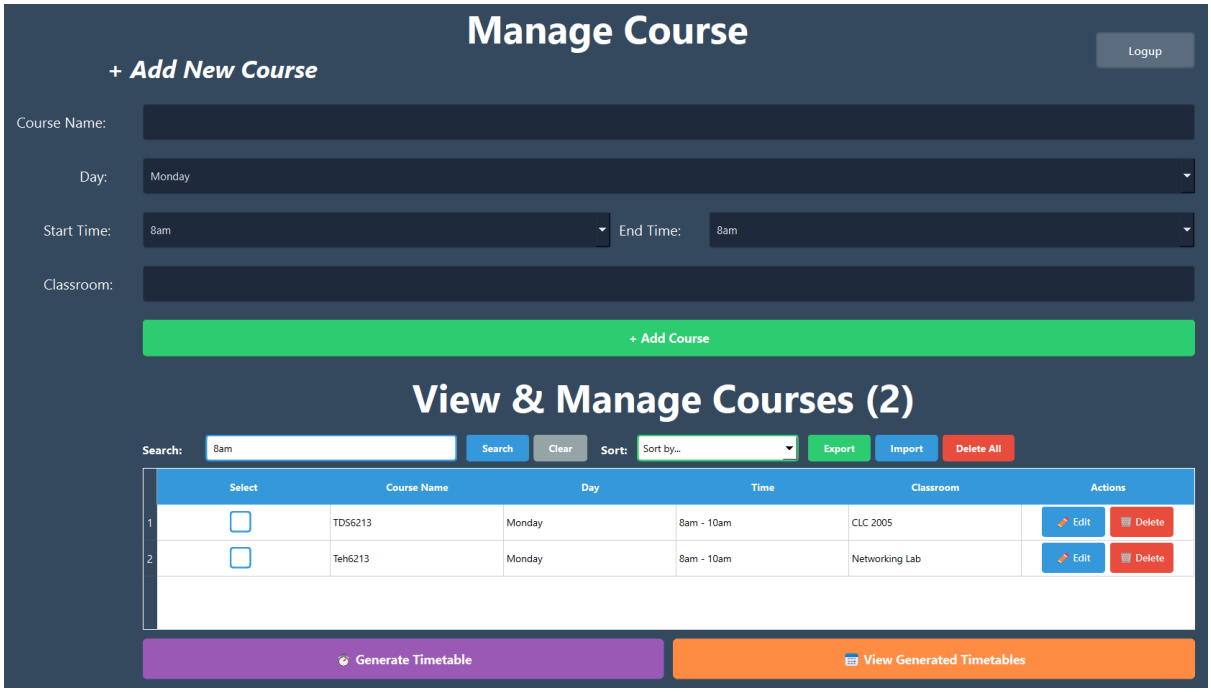
4.2.3.1 – Search the Course Name (Upper or lower case)

For example, when user type “teh” in the search box, the system will find all possible match name no matter the upper or lower case, like Teh, TEH. All will become the search result.



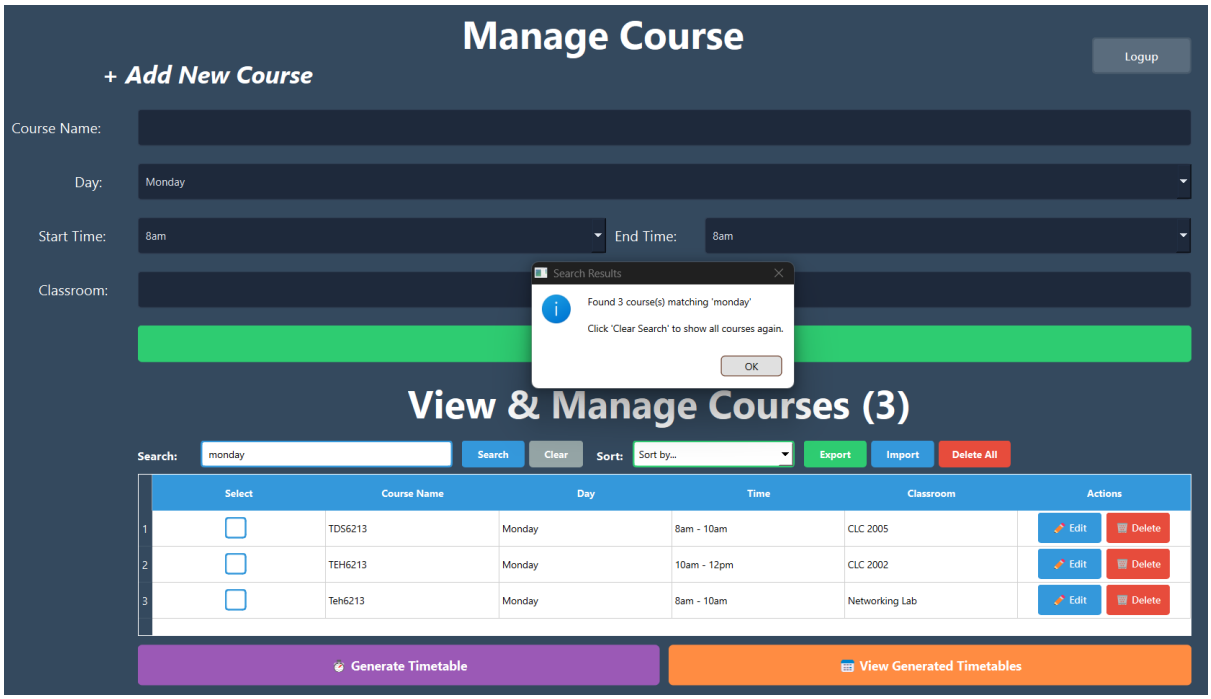
4.2.3.2 – Search with time

This system not only limited to search the course name, it also can search by the time.



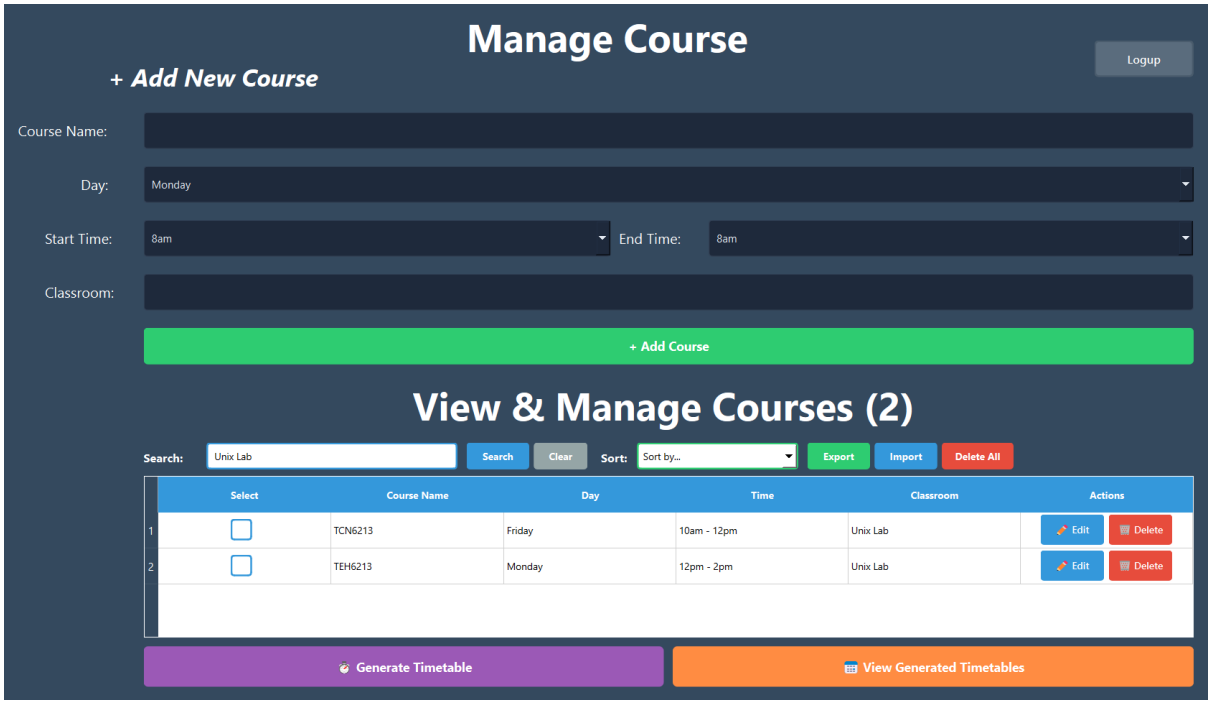
4.2.3.3 – Search with Day (Monday, Tuesday,Wednesday…)

The day also can be search in the search box.



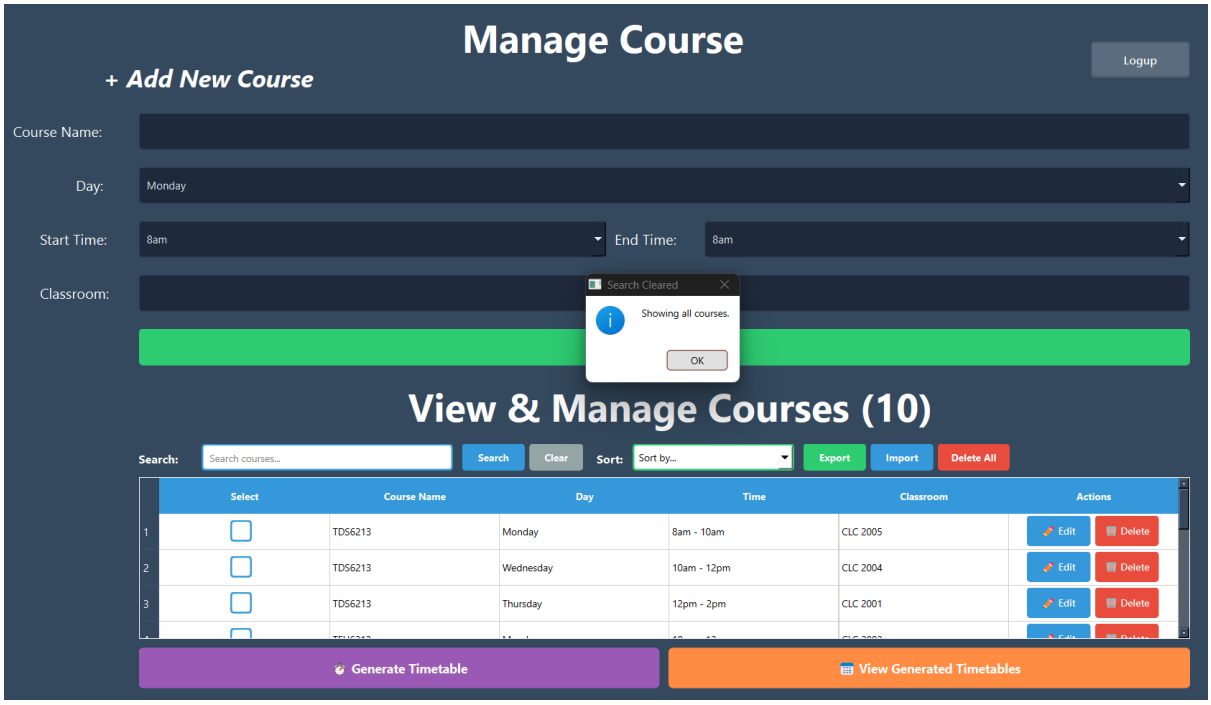
4.2.3.4 – Search with Classroom

The classroom could also be searched.



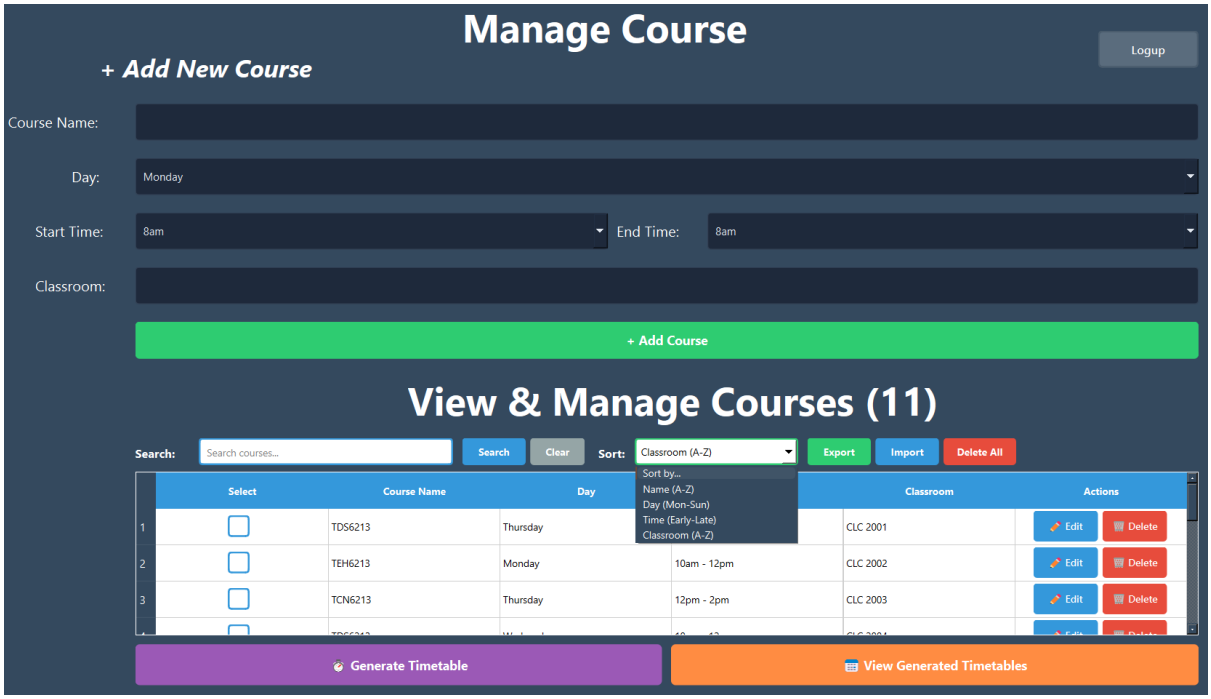
4.2.3.5 – Press Clear (show all course)

After search for the course name, day, time or classroom, the user can back to the normal interface to view all courses just by clicking “Clear” button.

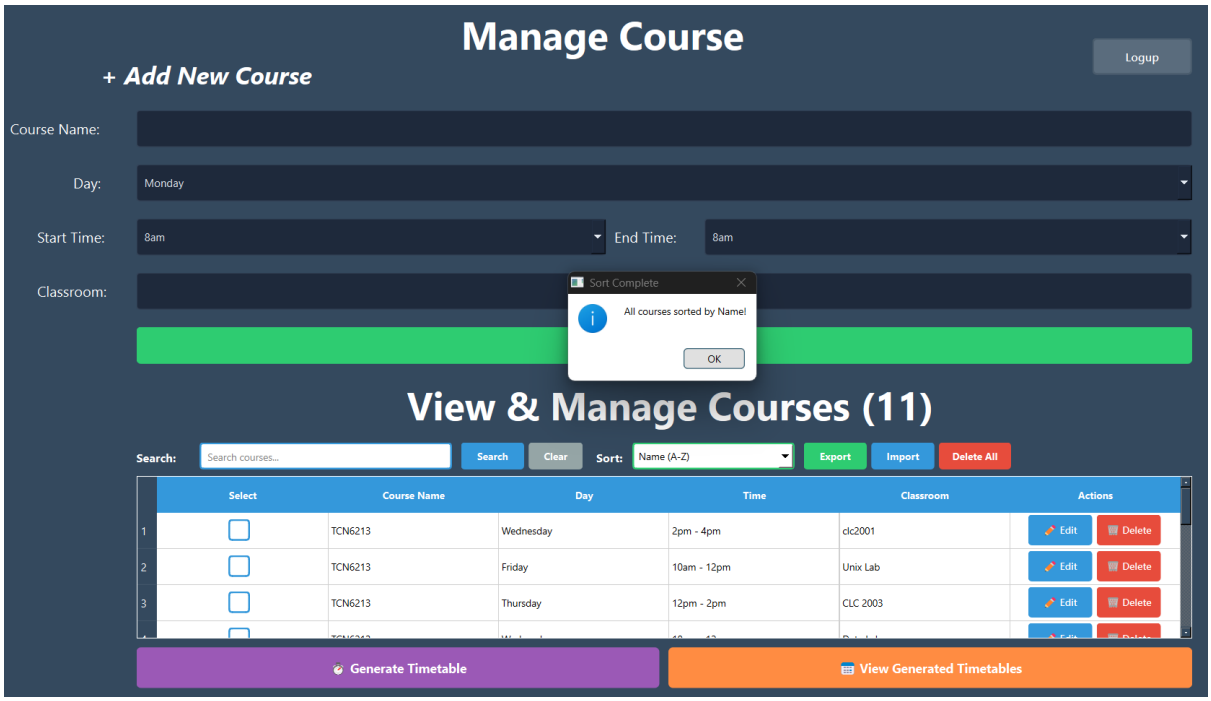


4.2.4 – Sort

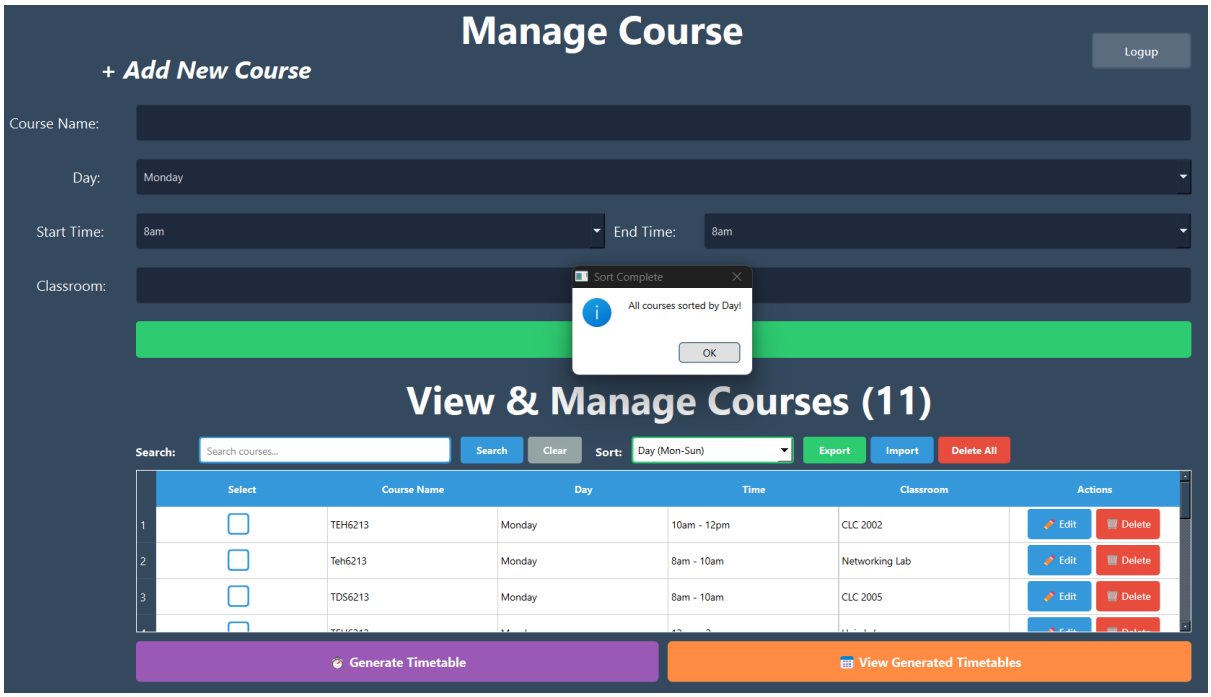
This system also has sorting function. It can sort by name(A-Z), Day (Mon-Sun), Time (Early-Late) and Classroom(A-Z).



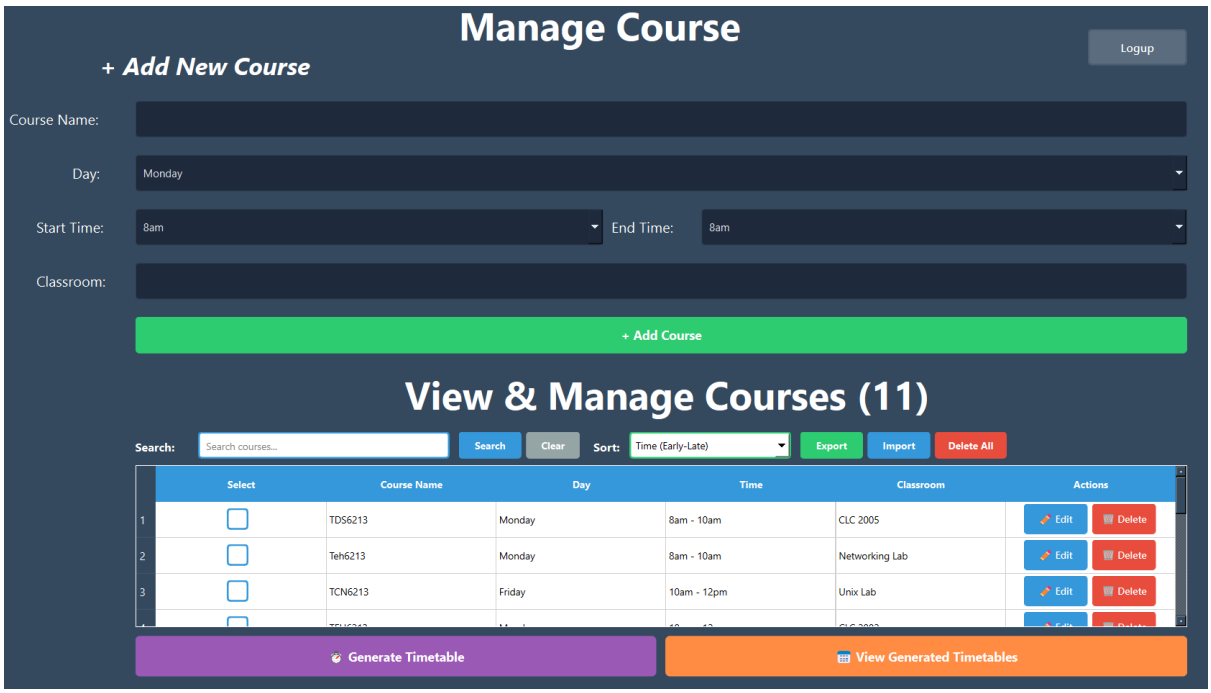
4.2.4.1 – sort with Name (A-Z)



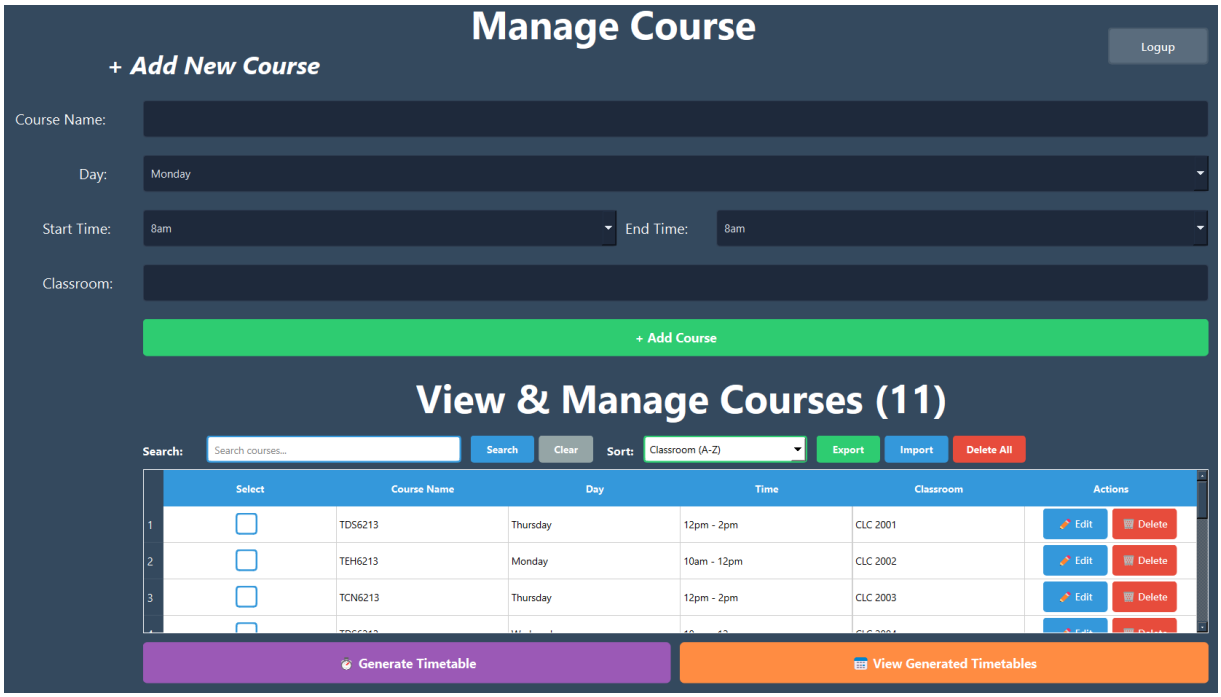
4.2.4.2 – sort with Day (Mon-Sun)



4.2.4.3 – sort with Time (Early late)



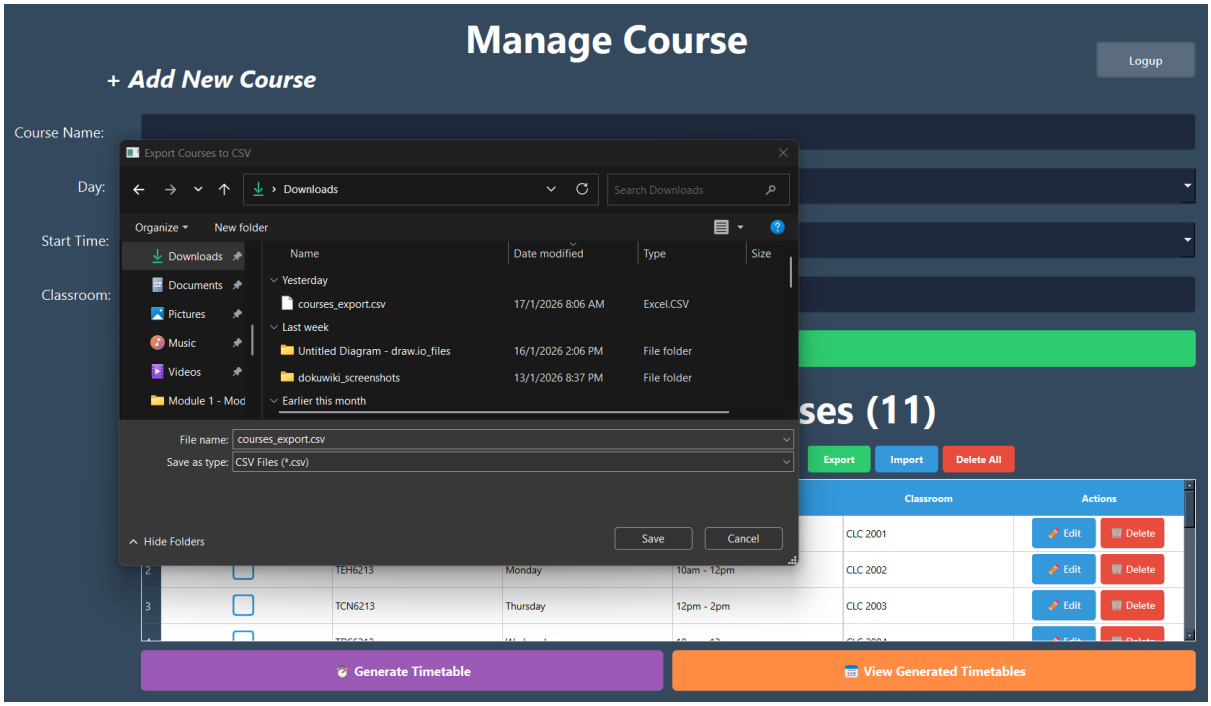
4.2.4.4 – sort with Classroom (A-Z)



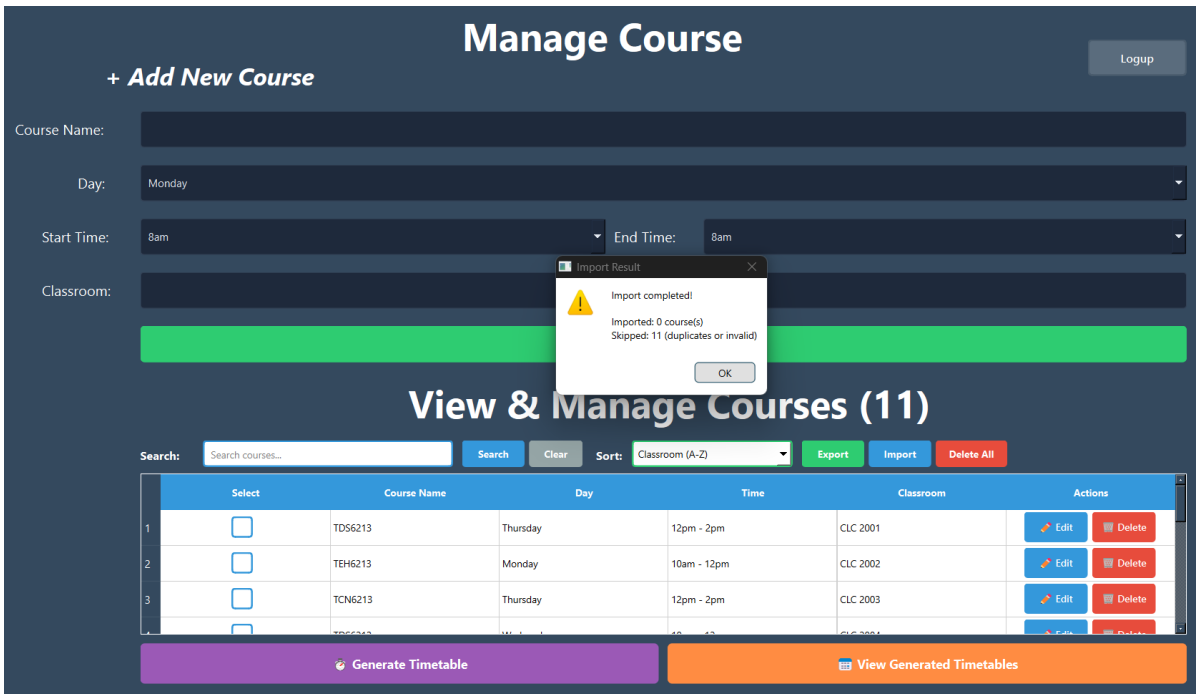
4.2.5 - Export /Import / Delete All buttons

4.2.5.1 -Export (save csv file)

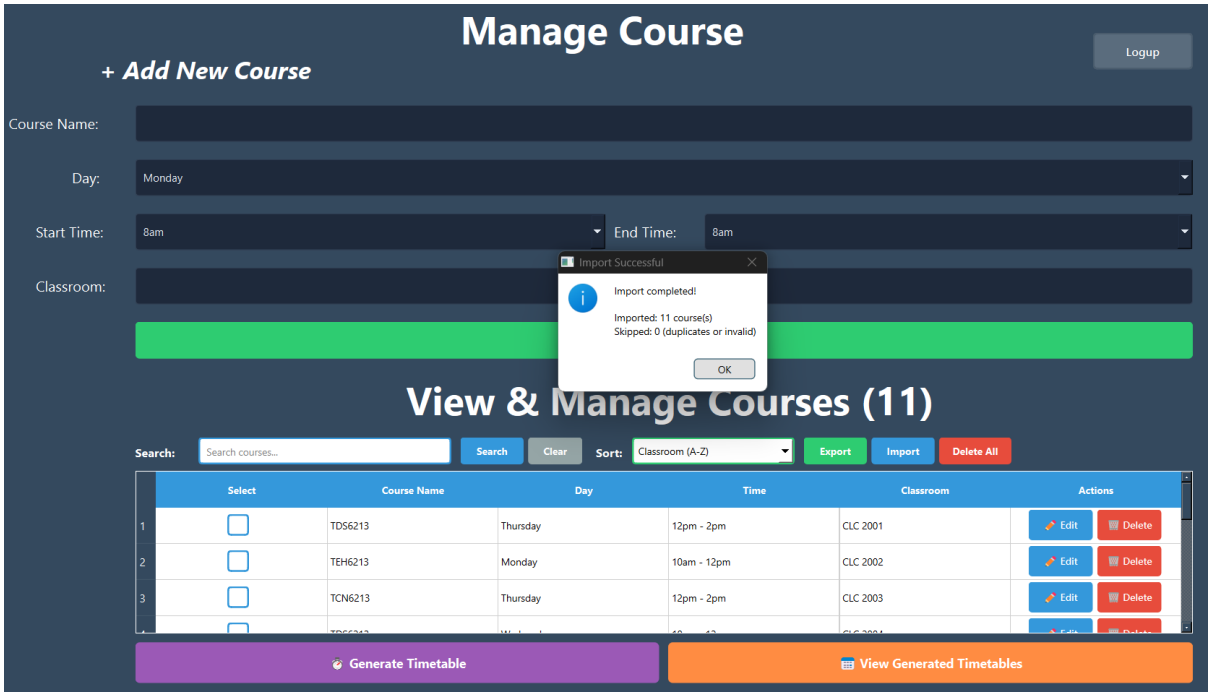
Users can export all the courses to their own computer and save it as a CSV file.



4.2.5.2 – Import (csv file)

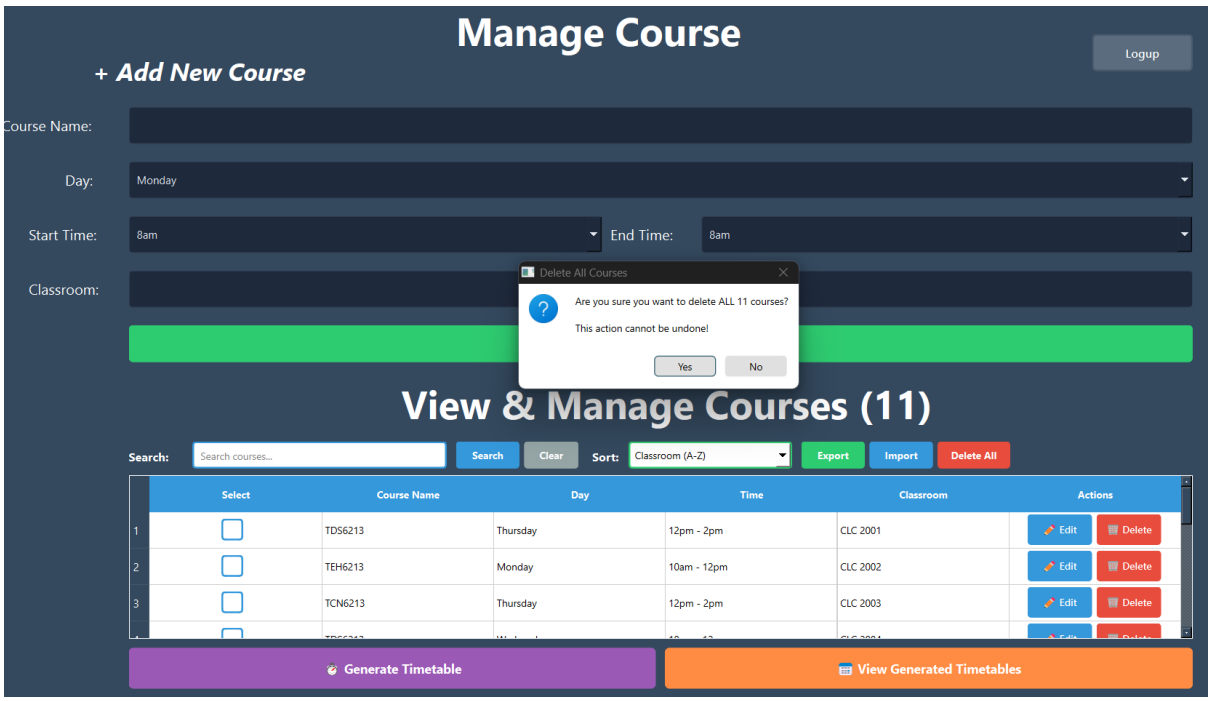


Import the CSV file. It will show import completed and all the courses in the CSV file will prompt inside this interface.

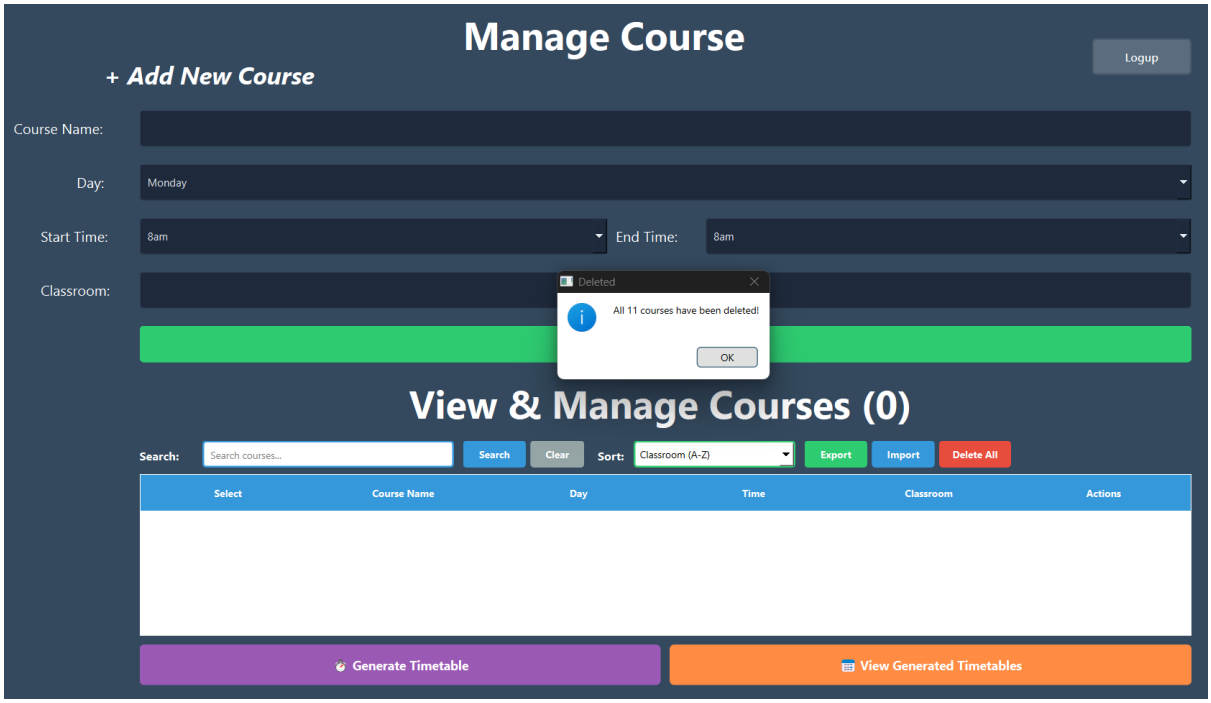


4.2.5.3 – Delete All course

Click “Delete All” button if do not want all the courses already.

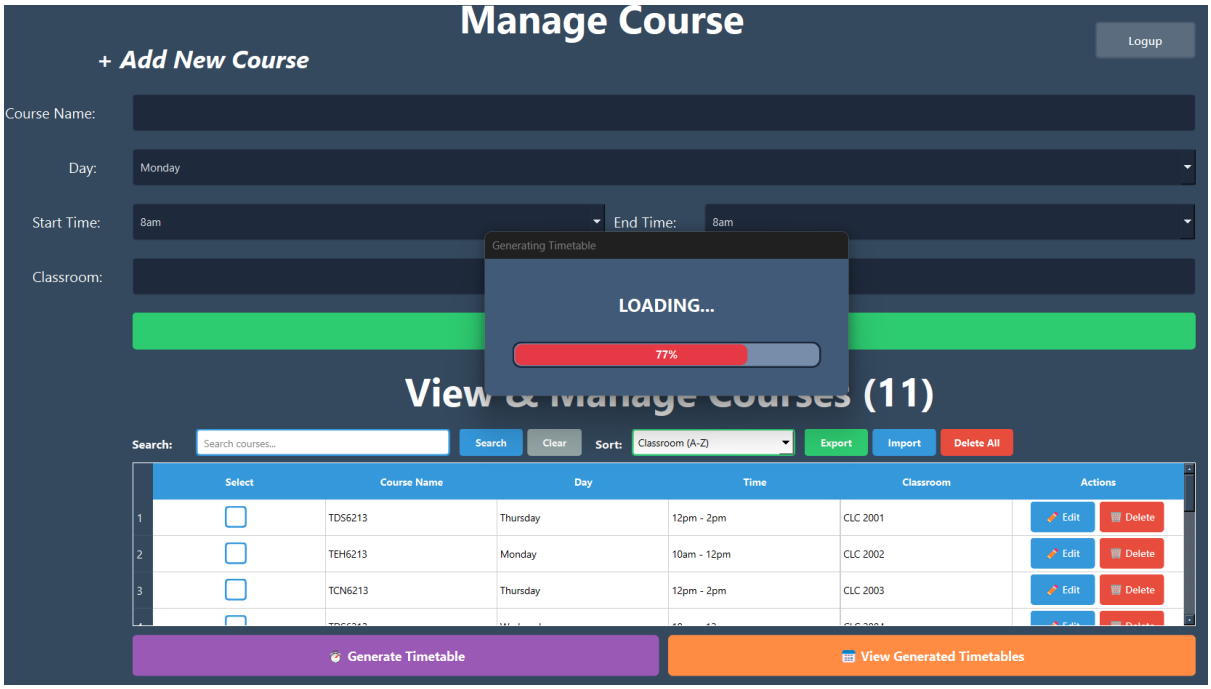


Then, it results in all courses have been deleted.


Manage Course
Logup
+ Add New Course
Course Name:
Day:
Monday
Start Time:
8am
End Time:
8am
Classroom:
Deleted
All 11 courses have been deleted!
OK
View & Manage Courses (0)
Search:
Search courses...
Search
Clear
Sort:
Classroom (A-Z)
Export
Import
Delete All
Select
Course Name
Day
Time
Classroom
Actions
Generate Timetable
View Generated Timetables

4.2.6 – Generate timetable

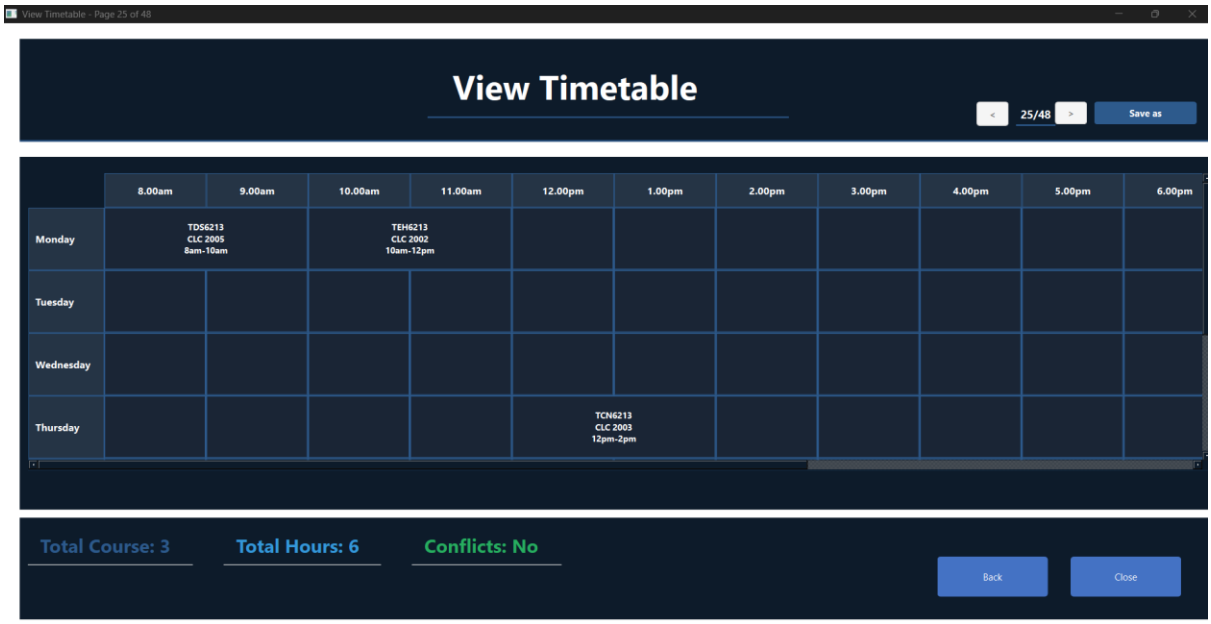
4.2.6.1 – Loading UI (after press generate timetable)



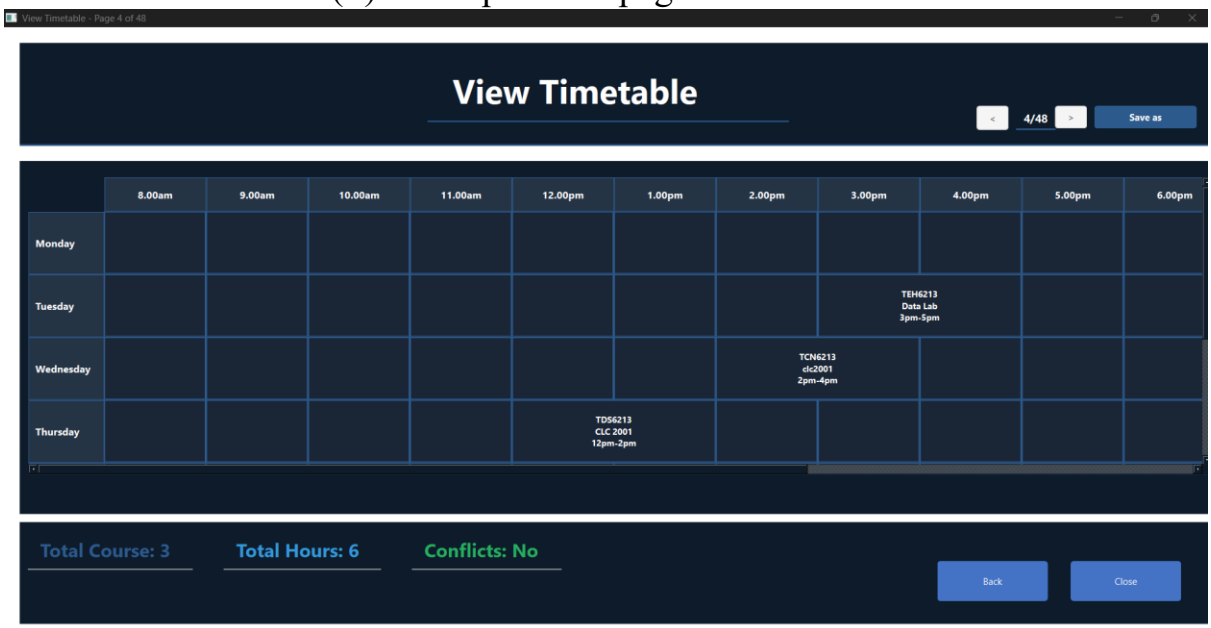
4.2.7 – View Timetable Page

4.2.7.1 – View Timetable (after generating will come up)

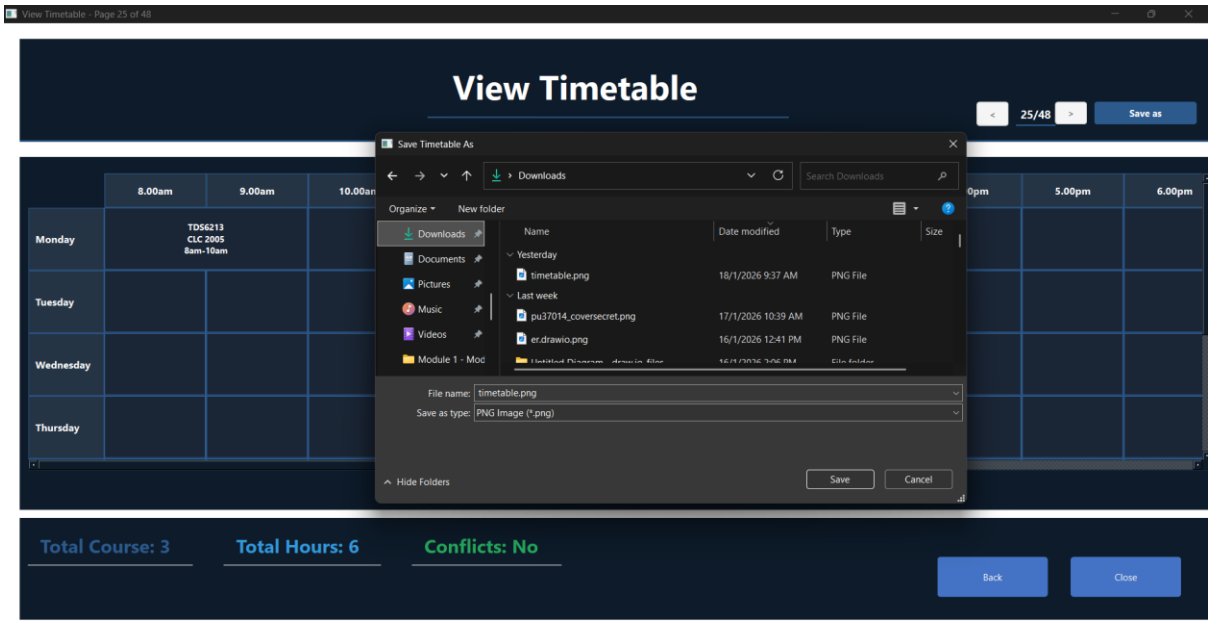
4.2.7.2 – Next button (>) to see other page



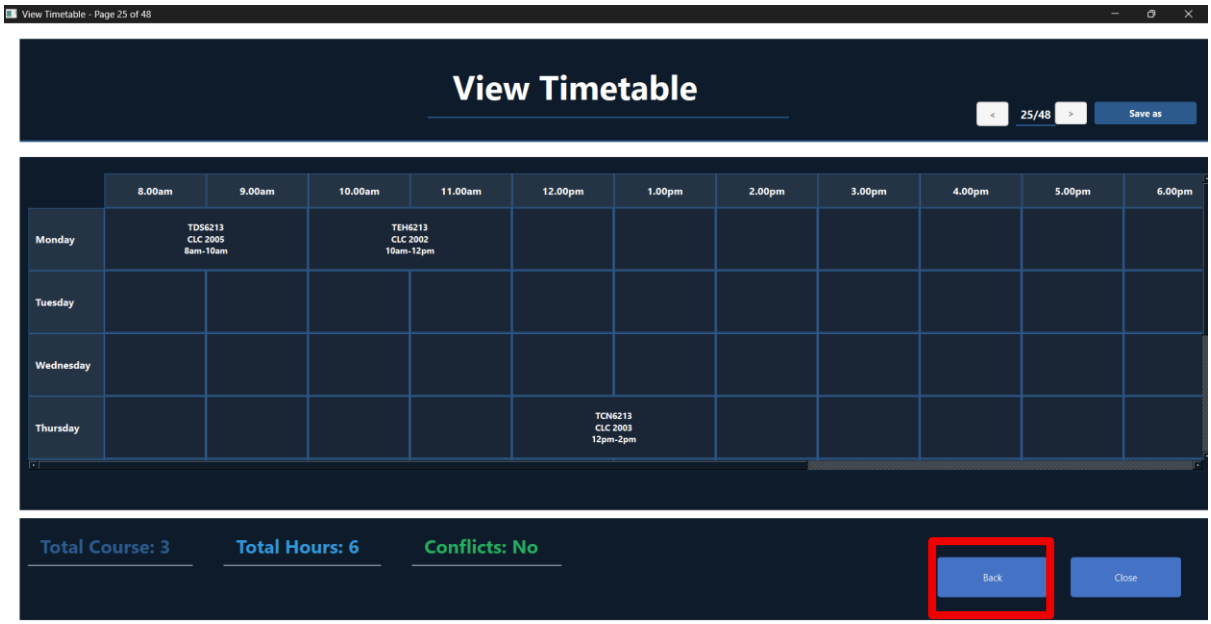
4.2.7.3 – Next button (<) to see previous page



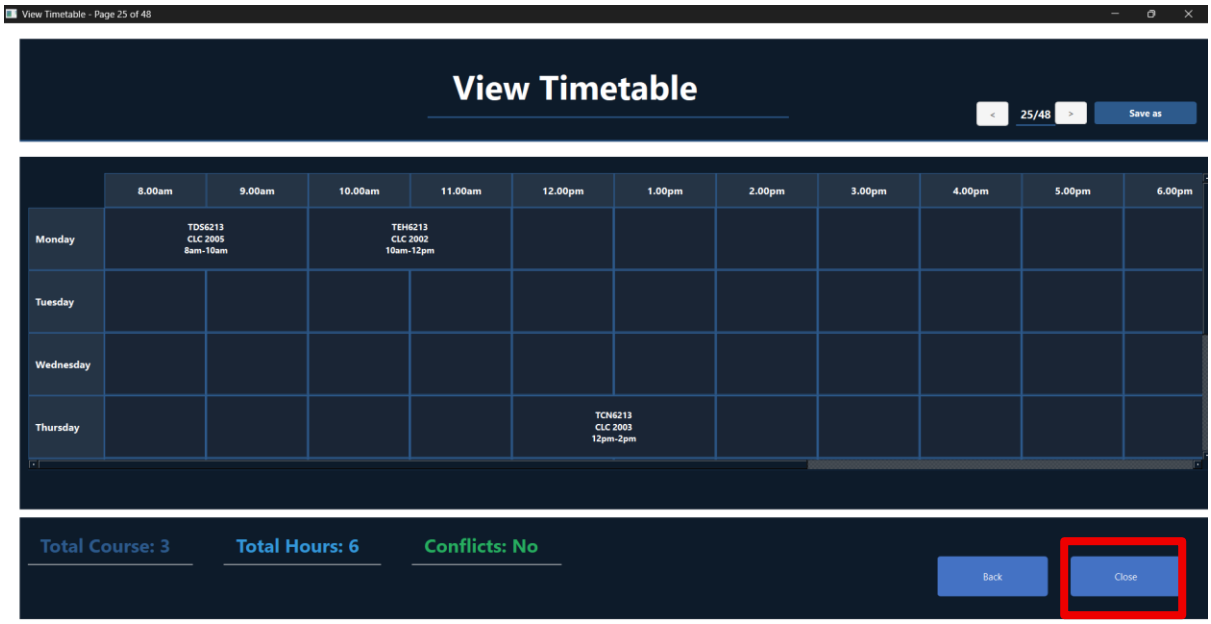
4.2.7.4 – Save As (You can save your favorite schedule as a .png file)



4.2.7.5 – Back button (back to Main interfaces)



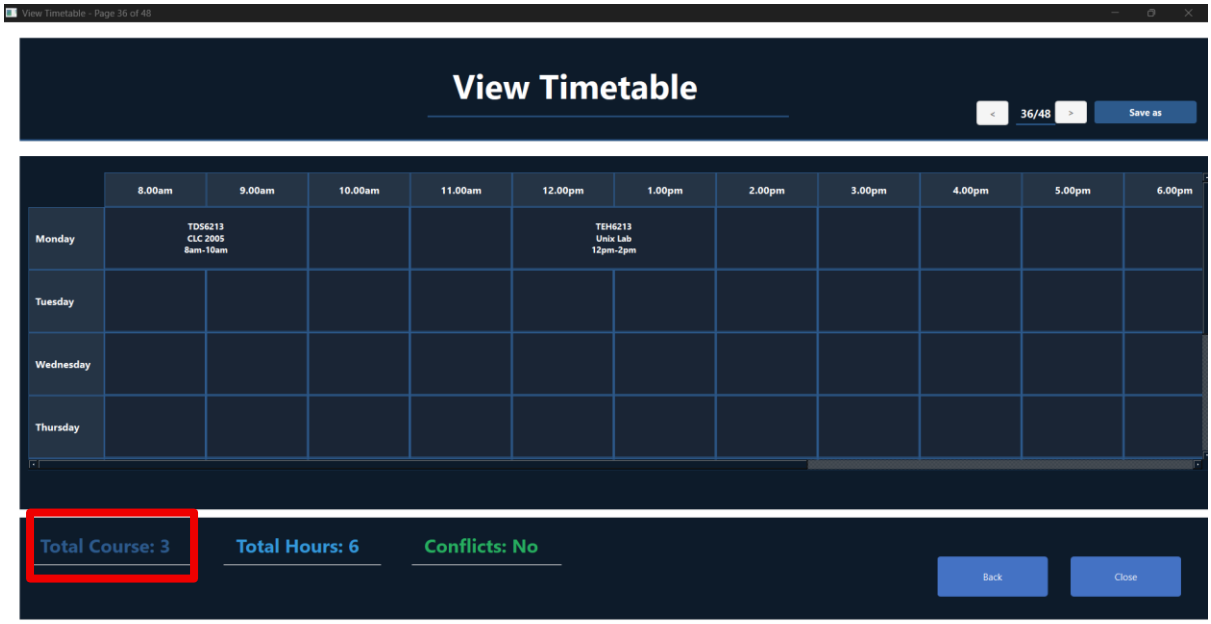
4.2.7.6 – Close (cannot open again this page)



4.2.8 – To determine which timetable is usable

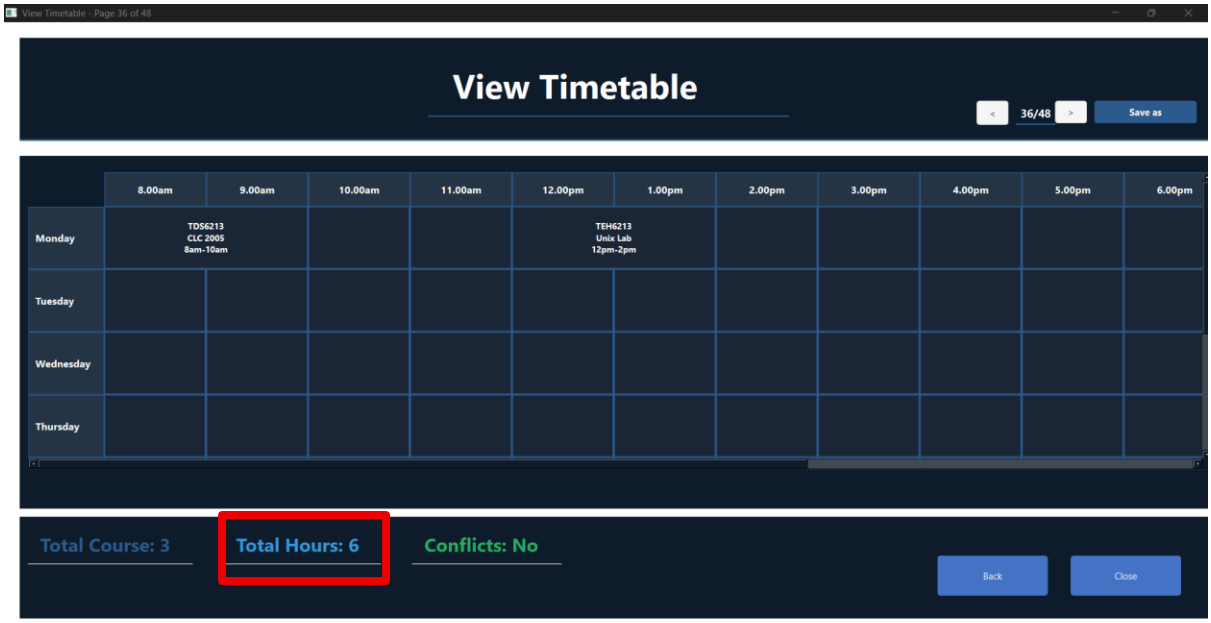
4.2.8.1 – Total Course Show

Total course means how many courses that the user has when they added the course before.



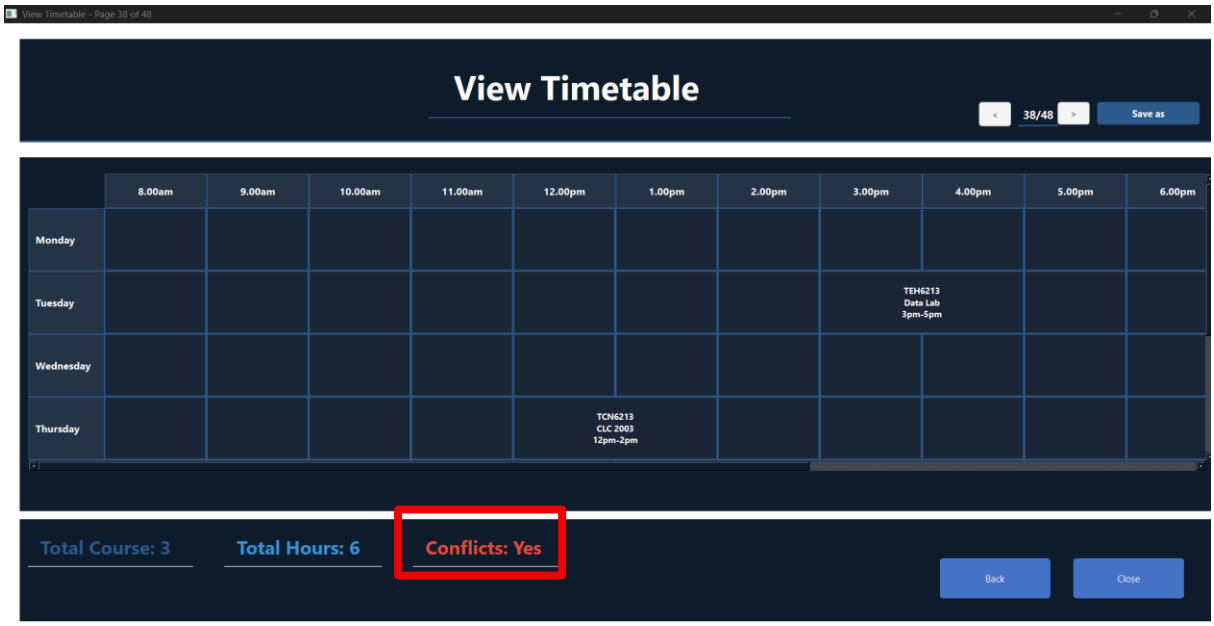
4.2.8.2 – Total Hours

How many hours that have in that specific page of the timetable based on the course Given.

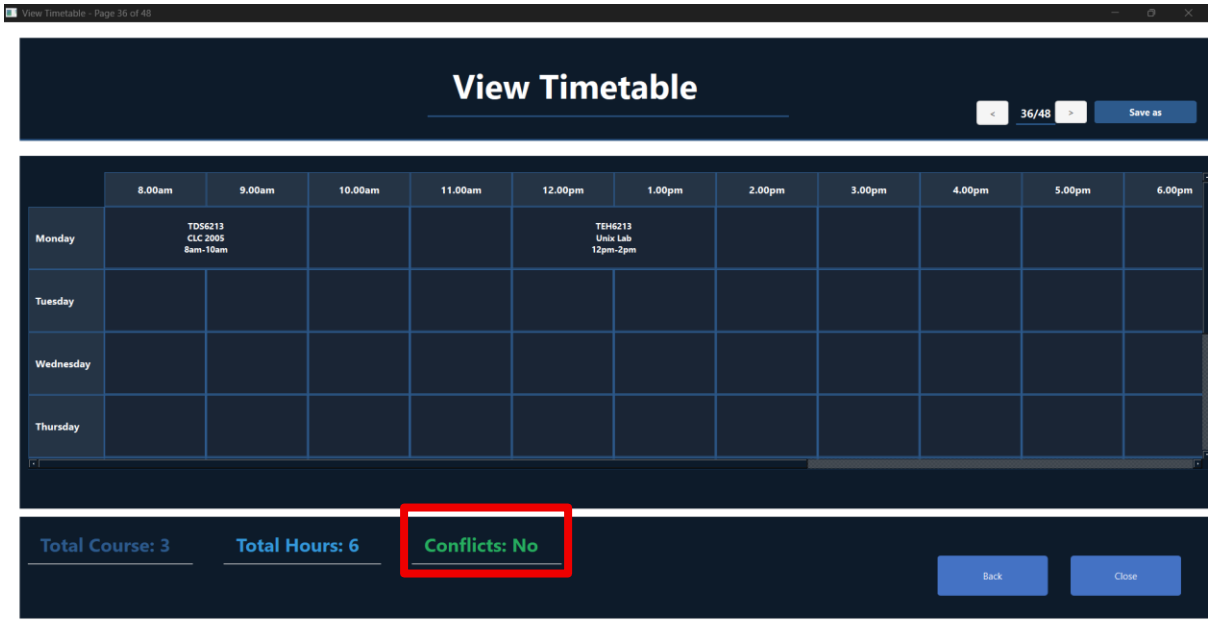


4.2.8.3 – Conflict (YES/NO)

If conflict happens among the courses, that particular timetable will show Conflicts: Yes.

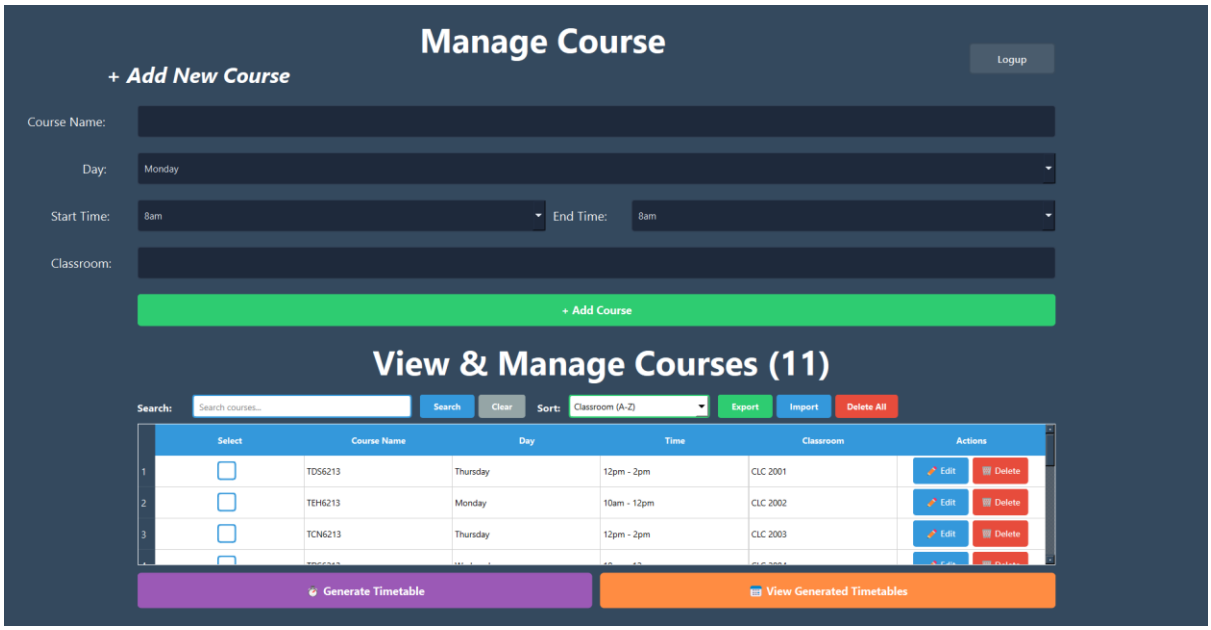


If there is no conflict, then it will show Conflicts: No with green colour.



4.2.9– Log Up

After ended everything, users can just press log up button to log out the page.



CONCLUSION

The Student Course Registration and Timetable Management System has been successfully developed and thoroughly tested. The system successfully addresses all challenges which identified by students during the course registration day. The system implementing efficient data structures and algorithms that deliver fast and responsive performance.

The system achieves excellent performance metrics across all operations. User login takes exactly 0.5 milliseconds regardless of the number of registered students, ensuring a responsive login experience at any scale. Course deletion takes only 1 millisecond which makes course management smooth and efficient. Next, course search takes 2 milliseconds with flexible partial matching support. Then, course sorting takes about 5 milliseconds for typical course lists. This all become the evidence that all operations complete in under 50 milliseconds, which is well below the 100-millisecond threshold where users start to notice delays.

Besides, the system successfully reduces consuming time from 1-2 hours to just 5-10 minutes. The risk of discovering scheduling conflicts after registration also will be decreased. Students now can enjoy exploring all possible schedule combinations automatically and select their preferred schedule with confidence by using our system.

In conclusion, our system is functionable and can handle growth without affect the smooth performance. Even if the system user number grows up and achieves even 10,000 registered users, login performance will maintain at 0.5 milliseconds as we implement the HashTable data structure. The system is now ready for deployment and is expected to provide significant value to Multimedia University students during the course registration process.

TASK DECLARATION FORM

Each group member, including the group leader, must individually complete and sign this form.
The signed forms (from all members) are to be appended to the final report for submission.

[Student Course Registration & Timetable Management System]

by

[1C_ Group 5]

Group Member's Name	:	SAW CHEE HONG
Group Member's ID	:	242UT2444D

For the purpose of completing this project, I have performed the following tasks:

A. REPORT:

1. Introduction

2. Problem Statement

3. ADT Specification - **UserRegistry ADT** (Hash Table implementation)

4. ADT Specification - **MainWindow ADT** (Login window operations)

5. ADT Specification - **Loading ADT** (Progress bar animation)

6. Program Screenshots (4.1.1: Login System Page, 4.1.7: Login Attempt Limit, 4.2.6: Loading UI, 4.2.9: Log Out)

B. PROGRAMMING:

1. mainwindow.h, mainwindow.cpp, mainwindow.ui - Login system implementation

2. loadingdialog.h, loadingdialog.cpp - Progress bar animation with QTimer

3. Hash Table (UserRegistry) implementation - initRegistry(), addUser(), validateUser(), userExists(), freeRegistry(), hashFunction()

4. Login validation logic and 5 attempts limit feature

5. Forgot password feature

6. Code integration and debugging across all modules

7. UI styling - Color scheme and window maximization (collaborated with Yeap)

1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.

2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.

Group Member's Signature	:	
Group Member's Name	:	SAW CHEE HONG
Group Member's ID	:	242UT2444D
Date	:	25/1/2026

Task Declaration Form

Each group member, including the group leader, must individually complete and sign this form.
The signed forms (from all members) are to be appended to the final report for submission.

[Student Course Registration & Timetable Management System]

by

[1C_ Group 5]

Group Member's Name	:	LEE JIA YIN
Group Member's ID	:	242UT241HP

For the purpose of completing this project, I have performed the following tasks:

A. REPORT:

1. Implementation Details (ER Diagram design and explanation)

2. Implementation Details - HashTable justification (Why hash table for user passwords)

3. ADT Specification - **CourseList ADT** (Linked List implementation)

ADT Specification - **Timetable ADT** (Timetable generation and display operations)

5. Implementation Details - Recursive Backtracking justification (Why recursive backtracking for timetable generation)

6. Implementation Details - LinearSearch justification (Why linear search for course search, partial matching support)

7. Program Screenshots (4.2.7: View Timetable series, 4.2.8: Statistics display - Total Course, Total Hours, Conflict YES/NO)

B. PROGRAMMING:

1. timetable.h, timetable.cpp, timetable.ui - Timetable display window implementation

2. Timetable generation algorithm - generateAllCombinations() implementation

3. Recursive backtracking algorithm - generateCombinationsRecursive() implementation

4. Conflict detection system - hasConflict(), detectConflicts() implementation

5. Multi-page timetable navigation - onPrevPage(), onNextPage(), updatePageLabel()

6. Time overlap checking logic - (start1 < end2 AND start2 < end1)

7. Statistics calculation - Total courses, total hours, conflict count

8. Timetable grid display - populateTimetable(), cell merging for multi-hour courses

9. Day to row conversion - dayToRow() (Monday=0 to Sunday=6)

10. Time to column conversion - timeToColumn() (8am=0 to 9pm=13)

11. Bubble sort for day/time ordering - sortCoursesByDayAndTime()

12. CombinationList data structure - Store all valid timetable combinations

1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.

2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.

Group Member's Signature	:	
Group Member's Name	:	LEE JIA YIN
Group Member's ID	:	242UT241HP
Date	:	20/1/2026

Task Declaration Form

Each group member, including the group leader, must individually complete and sign this form.
The signed forms (from all members) are to be appended to the final report for submission.

[Student Course Registration & Timetable Management System]

by

[1C_ Group 5]

Group Member's Name	:	BENJAMIN HONG JUN KIAT
Group Member's ID	:	242UT243XT

For the purpose of completing this project, I have performed the following tasks:

A. REPORT:

1. Objectives

2. ADT Specification - **SignupWindow ADT** (Registration dialog operations)

3. System security features documentation (Password validation, duplicate prevention)

4. Program Screenshots (4.1.2: Sign Up Page, 4.1.3: Student ID Validation, 4.1.4: Password Validation, 4.1.5: Show/Hide Password, 4.1.6: Password Mismatch Error)

5. Implementation Details - QuickSort justification (Why QuickSort, comparison with MergeSort, HeapSort, BubbleSort)

6. Conclusion

B. PROGRAMMING

1. signupwindow.h, signupwindow.cpp, signupwindow.ui - Sign up system implementation

2. User registration functionality - on_pushButton_Confirm_clicked() implementation

3. Password validation logic - Minimum 6 characters requirement

4. Password matching validation - Confirm password comparison

5. Show/Hide password toggle - QLineEdit::setEchoMode() implementation

6. Student ID validation - 5-15 character length requirement

7. Signal-slot connection - userRegistered signal to MainWindow
8. Forgot password feature
9. Error message dialogs for validation failures

1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.
2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.

Group Member's Signature	:	**BH**
Group Member's Name	:	BENJAMIN HONG JUN KIAT
Group Member's ID	:	242UT243XT
Date	:	25/1/2026

Task Declaration Form

Each group member, including the group leader, must individually complete and sign this form.
The signed forms (from all members) are to be appended to the final report for submission.

[Student Course Registration & Timetable Management System]

by

[1C_ Group 5]

Group Member's Name	:	YEAP BOON SHEN
Group Member's ID	:	242UT244D2

For the purpose of completing this project, I have performed the following tasks:

A. REPORT:

1. ADT Summary Table

2.. ADT Specification - Course ADT (Data items: name, day, startTime, endTime, classroom)

3. ADT Specification - **ManageCoursesPage ADT** (Course management operations)

4. Implementation Details - LinkedList justification (Why linked list for course management, comparison with Array, Queue, Stack)

5. Implementation Details - LinearSearch justification (Why linear search for course search, partial matching support)

6. Program Screenshots (4.2.1: Add Course series, 4.2.2: Select/Edit/Delete, 4.2.3: Search and Clear, 4.2.4: Sort, 4.2.5: Export/Import/Delete All)

B. PROGRAMMING:

1. managecourse.h, managecourse.cpp, managecourse.ui - Course management interface

2. Linked List (CourseList) implementation - initCourseList(), addCourse(), getCourse(), updateCourse(), deleteCourse(), courseCount(), clearCourseList()

3. Add Course function - Input validation, duplicate prevention, time validation

4. Edit Course function - Load course to form, update linked list

5. Delete Course function - Remove from linked list, refresh table display

6. Search function - LinearSearch implementation for name/day/time/classroom with partial matching

7. Sort function - QuickSort implementation for name/day/time/classroom

8. Table display with dynamic widgets - Checkboxes, Edit/Delete buttons, row highlighting

9. Export to CSV and Import from CSV - File handling implementation

10. UI design and styling - Button colors, table styling, checkbox styling (collaborated with Saw Chee Hong)

C. PRESENTATION:

1. Search, Sort, Export/Import CSV

2. LinkedList implementation code explanation

3. QuickSort algorithm code explanation

4. LinearSearch implementation code explanation

1. I hereby declare that I have assessed the final submission and I take the full responsibility should there be any incompleteness, omissions, delays, or non-submission.

2. I also certify that not part of this project is copied from any other student's work, or from any other sources, except where acknowledgement is made.

Group Member's Signature	:	
Group Member's Name	:	YEAP BOON SHEN
Group Member's ID	:	242UT244D2
Date	:	23/1/2026

YOUTUBE LINK

Below is our YouTube demo link for all the programs: https://youtu.be/kN7Y6_n-ECM